RĪGAS VALSTS TEHNIKUMS

DATORIKAS NODAĻA

Izglītības programma: Programmēšana

KVALIFIKĀCIJAS DARBS

“MarketPulse: Inteliģenta smaržu cenu uzraudzības un analīzes sistēma”

Paskaidrojošais raksts 62 lpp.

Audzēknis: Ervīns Ubiņins DP4-3

Prakses vadītājs: Igors Litvjakovs

Nodaļas vadītājs: Normunds Barbāns

Rīga 2026

ANOTĀCIJA

Kvalifikācijas darba tēma ir **“MarketPulse: Inteliģenta smaržu cenu uzraudzības un analīzes sistēma”**. Izstrādātā sistēma ir paredzēta kā autonoms vīriešu parfimērijas tirgus monitoringa un padziļinātas analīzes rīks, kas nodrošina datu ieguvi no pieciem vadošajiem Latvijas e-komercijas resursiem: **Douglas**, **Notino**, **Makeup**, **220.lv** un **1a.lv**. Sistēmas unikalitāte slēpjas **hibrīdā datu ieguves dzinēja** izmantošanā, kas apvieno asinhronus pieprasījumus ar pārlūkprogrammas emulāciju, kā arī **Google Gemini AI** integrācijā "pašārstēšanās" (self-healing) mehānismiem, kas automātiski atjauno parsēšanas noteikumus vietņu struktūras izmaiņu gadījumā. Tehnoloģiskais steks ietver **Python 3.12**, **FastAPI** ietvaru, **PostgreSQL** datubāzi un **Redis** kešatmiņai. Sistēma ir pilnībā konteinerizēta ar **Docker** un izvietota **DigitalOcean** mākoņserverī, nodrošinot nepārtrauktu datu apstrādi un automātisku izvietošanu (CI/CD).

Paskaidrojošais raksts sastāv no deviņām nodaļām, kurās secīgi aprakstīts viss programmatūras izstrādes cikls. Ievadā tiek analizēta e-komercijas vide un pamatota automatizēta monitoringa rīka nepieciešamība, savukārt uzdevuma nostādnē definēti sistēmas funkcionālie mērķi un lietotāju lomas. Prasību specifikācijā detalizēti aprakstīta ieejas un izejas informācija, kā arī funkcionālās un nefunkcionālās prasības atbilstoši standartiem. Uzdevuma risināšanas līdzekļu izvēles nodaļā sniegts visaptverošs izmantoto tehnoloģiju, bibliotēku un mākoņpakalpojumu pamatojums. Projektēšanas nodaļa satur sistēmas arhitektūras hierarhiju, konceptuālo **ER-modeli** un septiņu galveno procesu datu plūsmu diagrammas (**DFD**), tostarp AI noteikumu ģenerēšanas un produktu sasaistes (Chaining) algoritmus. Darbā iekļauts detalizēts datu struktūru apraksts, lietotāja ceļvedis ar instrukcijām sistēmas uzstādīšanai un tīmekļa saskarnes lietošanai, kā arī testēšanas dokumentācija un nobeigums ar secinājumiem.

Kvalifikācijas darbs sastāv no **125** lappusēm, kurās ietilpst **61** attēli, **20** tabulas un **9** pielikumi. Pielikumā ir iekļauti programmas pirmkoda fragmenti.

ANNOTATION

The topic of the qualification work is **"MarketPulse: Intelligent Fragrance Price Monitoring and Analysis System"**. The developed system serves as an autonomous tool for monitoring and in-depth analysis of the men's perfume market, performing data extraction from five leading Latvian e-commerce platforms: **Douglas**, **Notino**, **Makeup**, **220.lv**, and **1a.lv**. The uniqueness of the solution lies in the implementation of a **hybrid data extraction engine** that combines asynchronous HTTP requests with browser emulation, alongside **Google Gemini AI** integration for "self-healing" mechanisms that automatically update scraping rules when website structures change. The technology stack includes **Python 3.12**, **FastAPI** framework, **PostgreSQL** database, and **Redis** for caching. The system is fully containerized using **Docker** and deployed on a **DigitalOcean** cloud server, ensuring continuous data processing and automated deployment (CI/CD).

The explanatory note is structured into nine chapters, detailing the full software development lifecycle. The introduction analyzes the e-commerce environment and justifies the need for an automated monitoring tool, while the task statement defines functional goals and user roles. The requirements specification provides a detailed description of input and output information, as well as functional and non-functional requirements according to standards. The justification of solution tools offers a comprehensive analysis of the technologies, libraries, and cloud services used. The design chapter includes the system architecture hierarchy, conceptual **ER-model**, and data flow diagrams (**DFD**) for seven key processes, including AI rule generation and product chaining algorithms. The report features a detailed description of data structures, a user guide with instructions for system installation and web interface interaction, as well as testing documentation and a conclusion with final insights.

The qualification work consists of **125** pages, including **61** images, **20** tables, and **9** appendices. The appendix contains fragments of the program source code.

SATURS

IEVADS **5**
1. UZDEVUMA NOSTĀDNE **7**
2. PRASĪBU SPECIFIKĀCIJA **9**
2.1. Ievades un izvades informācija 9
2.1.1. Ieejas informācijas apraksts 9
2.1.2. Izvades informācijas apraksts 11
2.2. Funkcionālās prasības 12
2.3. Nefunkcionālās prasības 17
3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS **21**
4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA **23**
4.1. Sistēmas struktūras modelis 23
4.1.1. Sistēmas arhitektūra 23
4.1.2. Sistēmas ER modelis 26
4.2. Funkcionālais sistēmas modelis 27
4.2.1. Datu plūsmu modelis 27
5. DATU STRUKTŪRU APRAKSTS **32**
6. LIETOTĀJA CEĻVEDIS **36**
6.1. Sistēmas prasības aparatūrai un programmatūrai 36
6.2. Sistēmas instalācija un palaišana 37
6.3. Programmas apraksts 39
7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA **57**
NOBEIGUMS **60**
INFORMĀCIJAS AVOTI **61**
PIELIKUMI **63**
1. pielikums. Funkcionālās dekompozīcijas diagramma 63
2. pielikums. Sistēmas ER-diagrama 64
3. pielikums. Datubāzes tabulu shēma 65
4. pielikums. Pamācība Nr. 5 66
5. pielikums. Datu atlāse 67
6. pielikums. Telegram savienojums 68
7. pielikums. PDF fails (Kataloga eksports) 69
8. pielikums. PDF fails (Produkta eksports) 70
9. pielikums. Programmas pirmteksts 71

IEVADS

Mūsdienu digitālajā laikmetā e-komercija Latvijā ir kļuvusi par neatņemamu ikdienas sastāvdaļu. Patērētāji arvien biežāk izvēlas iegādāties preces tiešsaistē, meklējot labākos piedāvājumus un zemākās cenas. Parfimērijas tirgus ir viens no dinamiskākajiem e-komercijas segmentiem, kurā tādi lielie spēlētāji kā **Douglas**, **Notino**, **Makeup**, **220.lv** un **1a.lv** regulāri atjaunina savu sortimentu un piedāvā dažādas akcijas. Tomēr patērētājam, kurš vēlas atrast izdevīgāko cenu konkrētam produktam, piemēram, "Dior Sauvage 100ml", nākas saskarties ar vairākiem izaicinājumiem.

Galvenā problēma, kas pastāv pirms šīs sistēmas izstrādes, ir informācijas fragmentācija un datu standartizācijas trūkums starp dažādiem veikaliem. Katrs e-komercijas resurss izmanto savu preču aprakstīšanas formātu: nosaukumi var atšķirties (piemēram, vienā vietnē lietots latviskais "pudelēs pildīts", bet citā - angļu "Bottled"), koncentrācijas (EDP, EDT) un tilpumi tiek norādīti dažādās vietās. Manuāla cenu salīdzināšana starp pieciem vai vairāk resursiem ir laikietilpīgs process, kas bieži vien neļauj pieņemt objektīvu lēmumu, jo dati par cenām mainās vairākas reizes diennaktī.

Veiktā tirgus izpēte liecina, ka Latvijā pieejamie vispārīgie cenu salīdzināšanas rīki, piemēram, *Salidzini.lv* vai *Kurpirkt.lv*, ne vienmēr nodrošina precīzu datu ieguvi no specifiskiem parfimērijas katalogiem. Bieži vien šie rīki neattēlo visus preču variantus (tilpumus) vienā kartītē vai nespēj operatīvi reaģēt uz vietņu struktūras izmaiņām. Globālās analītikas sistēmas, savukārt, ir dārgas un paredzētas lieliem uzņēmumiem, nevis individuāliem lietotājiem vai lokāliem tirgus analītiķiem. Tas norāda uz būtisku nepilnību digitālajos risinājumos, kas būtu piemēroti tieši Latvijas parfimērijas tirgus specifikai.

Sistēma **"MarketPulse"** ir paredzēta, lai aizpildītu šo nišu, piedāvājot autonomu un inteliģentu risinājumu. Projekta aktualitāti pamato modernāko tehnoloģiju izmantošana: **hibrīdais datu ieguves dzinējs** nodrošina stabilitāti pret bloķēšanu, savukārt **Google Gemini AI** [9] integrācija nodrošina sistēmas "apmacību". Tas nozīmē, ka sistēma pati spēj atpazīt izmaiņas veikalu dizainā un automātiski atjaunināt datu ieguves noteikumus bez programmētāja iejaukšanās. Sistēmā ieviestā **"Saķedēšanas" loģika** ļauj inteliģenti apvienot identiskus produktus no dažādiem avotiem vienotā analītiskā skatā, sniedzot lietotājam patiesu tirgus pārskatu.

Kvalifikācijas darba mērķis ir radīt sistēmu, kas ne tikai fiksē aktuālās cenas, bet arī uzkrāj vēsturiskos datus, ļaujot analizēt cenu dinamiku ilgtermiņā un prognozēt izdevīgākos pirkuma brīžus. Sistēmas mērķauditorija ir gan vīriešu parfimērijas pircēji, kuri vēlas ietaupīt, gan digitālā mārketinga speciālisti, kam nepieciešami dati par konkurentu cenu politiku.

Pateicoties realizācijai uz **DigitalOcean** mākoņservera un automatizētai informēšanai caur **Telegram** [18] un e-pastu, sistēma darbojas kā pilnvērtīgs autonomais analītiķis, kas ir pieejams lietotājiem jebkurā laikā un vietā.

1. UZDEVUMA NOSTĀDNE

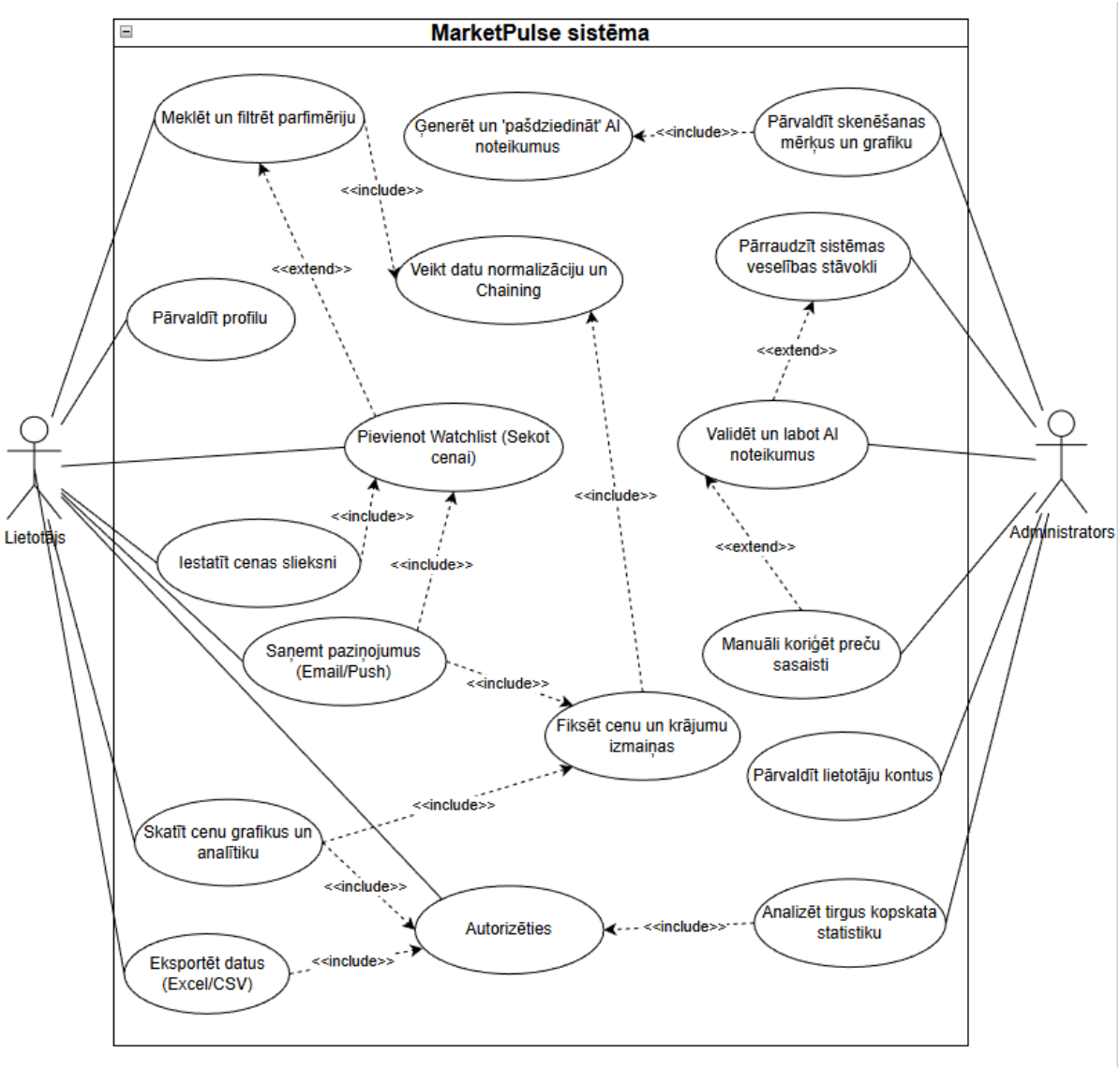
Kvalifikācijas darba uzdevums ir izveidot inteliģentu smaržu cenu uzraudzības un analīzes sistēmu "MarketPulse" Latvijas e-komercijas segmentam (Douglas, Notino, Makeup, 220.lv, 1a.lv). Sistēmas izstrādes nepieciešamību pamato e-komercijas platformu sarežģītā arhitektūra un datu fragmentācija, kas liedz lietotājiem operatīvi salīdzināt cenas dažādos resursos. Galvenais mērķis ir automatizēti vākt, apstrādāt un analizēt datus par vīriešu parfimērijas produktiem, izmantojot mākslīgā intelekta elementus datu precizitātes nodrošināšanai.

Sistēmā ir plānotas šādas galvenās funkcionalitātes, kas sadalītas pēc lietotāju lomām un sistēmas procesiem:

- **sistēmas administratoram būs iespēja:**
    - pārvaldīt un konfigurēt mērķa resursu (veikalu) sarakstus un skenēšanas grafikus;
    - uzraudzīt sistēmas veselības stāvokli un analizēt kļūdu žurnālus;
    - validēt un manuāli labot AI ģenerētos parsēšanas noteikumus;
    - analizēt tirgus kopskata statistiku un lietotāju kontus;
- **sistēmas lietotājam būs iespēja:**
    - reģistrēties un autorizēties sistēmā, pārvaldīt savu profilu;
    - meklēt un filtrēt parfimērijas produktus vienotā katalogā;
    - pievienot preču grupas sekošanas sarakstam (Watchlist) un iestatīt cenas slieksni;
    - saņemt automatizētus paziņojumus (Email/Push) par cenu kritumu;
    - apskatīt interaktīvus cenu izmaiņu grafikus un eksportēt datus Excel/CSV formātos;
- **sistēmai autonomi jānodrošina:**
    - datu ieguve, izmantojot hibrīdo ielādes dzini (curl_cffi [5] un Playwright [11]);
    - mākslīgā intelekta balstīta "pašārstēšanās", automātiski atjaunojot selektorus;
    - inteliģenta identisku produktu saķēdēšana (Chaining) un datu normalizācija;

- fiksēt cenu un krājumu izmaiņas katrā skenēšanas ciklā.

Informācija par sistēmas lietotājiem un to mijiedarbību ar sistēmu ir attēlota lietojumgadījumu diagrammā (skat. 1.att.).



1. att. UML Use Case diagramma sistēmai Market Pulse

2. PRASĪBU SPECIFIKĀCIJA

2.1. Ievades un izvades informācija

2.1.1. Ieejas informācijas apraksts

Sistēmā "MarketPulse" tiek nodrošināta visaptveroša ieejas informācijas apstrāde, kas nepieciešama lietotāja personalizācijai, autonomai datu ieguvei un tirgus analītikai. Informācija tiek ievadīta, izmantojot tastatūru, vai izvēlēta no sistēmas piedāvātajām interaktīvajām izvēlnēm. Visa ieejas informācija ir sadalīta piecās loģiskajās datu kopās.

1. Informācija lietotāja reģistrācijai un piekļuves kontrolei: Šī datu kopa ir kritiska drošai autorizācijai un JWT [14] sesiju pārvaldībai.

- e-pasta adrese - lietotāja unikālais identifikators un saziņas kanāls - burtu, ciparu un speciālo simbolu teksts ar izmēru līdz 255 rakstzīmēm (piem., janis.berzins@piemērs.lv);
- lietotāja pilns vārds - dati personalizētai saskarnei un paziņojumiem - burtu teksts ar izmēru līdz 100 rakstzīmēm (piem., Jānis Bērziņš);
- parole - slepena rakstzīmju virkne piekļuves aizsardzībai - slēpts burtu un ciparu teksts ar izmēru no 8 līdz 64 rakstzīmēm;
- paroles atkārtojums - kontroles lauks reģistrācijas kļūdu novēršanai - slēpts teksts, kuram pilnībā jāsakrīt ar lauku "parole".

2. Informācija sākotnējai iestatīšanai: Informācija, ko lietotājs ievada pēc pirmās autorizācijas, lai sistēma pielāgotu Dashboard saturu.

- prioritārie veikali - e-komercijas resursi, kuru dati lietotājam ir visaktuālākie - izvēle no saraksta, atzīmējot vairākas vērtības (piem., Notino, Douglas, 1a.lv);
- interesējošie zīmoli - saraksts ar parfimērijas ražotājiem, par kuru cenu svārstībām lietotājs vēlas saņemt informāciju prioritāri - izvēle no dinamiska saraksta (piem., Dior, Chanel, Versace);
- onboarding pabeigšanas pazīme - tehniska informācija par procesa statusu - loģiskā vērtība (Boolean: true/false).

3. Informācija preču meklēšanai un analītiskajai filtrēšanai: Dati, ko lietotājs ievada, lai iegūtu specifiskus tirgus pārskatus vai atrastu konkrētas preces.

- meklēšanas frāze - atslēgvārds parfimērijas meklēšanai apvienotajā katalogā - teksta virkne līdz 100 rakstzīmēm (piem., "Sauvage Elixir");
- minimālā un maksimālā cena - filtrēšanas diapazons preču sarakstam - pozitīvi skaitļi ar divām zīmēm aiz komata (piem., no 45.00 līdz 120.00);
- analīzes periods - laika posms, par kuru lietotājs vēlas apskatīt cenu vēstures grafiku - izvēle no saraksta (piem., 1 nedēļa, 1 mēnesis, 3 mēneši, 1 gads);
- kārtošanas kritērijs - parametrs rezultātu attēlošanai - izvēle no saraksta (piem., "Lētākais pirmais", "Lielākā atlaide", "Jaunumi").

4. Informācija paziņojumu un sekošanas konfigurācijai: Dati, kas nepieciešami automatizētai lietotāja informēšanai par cenu kritumu.

- mērķa cena - cenu slieksnis, pie kura lietotājs vēlas saņemt paziņojumu - skaitliska vērtība (piem., 65.50);
- **Telegram Chat ID** - unikāls ciparu identifikators sistēmas sasaistei ar Telegram botu - ciparu virkne līdz 20 zīmēm (piem., 548291044);
- paziņojumu kanālu izvēle - informācija par vēlamajiem saziņas veidiem - atzīmes rūtiņas (izvēle: E-pasts, Telegram);
- paziņojumu slieksnis (procentos) - vērtība, kas definē "ievērojamu" cenu kritumu vispārējā tirgus analīzē - ciparu vērtība (piem., 5.00%).

5. Administratora konfigurācijas un sistēmas vadības informācija: Tehniskā informācija sistēmas darbības uzturēšanai un AI moduļa vadībai.

- domēna konfigurācijas saite - bāzes URL jaunu veikalu pievienošanai - teksta virkne līdz 500 rakstzīmēm (piem., https://www.makeup.lv/category/95/);
- CSS selektoru manuāla korekcija - tehniskie parametri datu ieguves noteikumu precizēšanai - teksta virkne līdz 255 rakstzīmēm (piem., ".product-price-value");
- AI reģenerācijas trigeris - komanda jaunu noteikumu ģenerēšanai vietnes struktūras izmaiņu gadījumā - pogas nospiešana (Event trigeris);
- lietotāja statuss - konta aktivitātes pārvaldība - izvēle no saraksta (Aktīvs, Deaktivizēts).

2.1.2. Izvades informācijas apraksts

Sistēmas "MarketPulse" darbības rezultātā ieejas dati tiek apstrādāti, normalizēti un transformēti lietotājam noderīgā informācijā. Izvades informācija tiek attēlota sistēmas tīmekļa saskarnē, nosūtīta caur ārējiem saziņas kanāliem vai ģenerēta failu formātos lejupielādei. Informācija ir sagrupēta četrās galvenajās vienībās.

1. Analītiskais kopsavilkuma panelis uz ekrāna: Šī vienība sniedz lietotājam tūlītēju pārskatu par tirgus aktuālo stāvokli un sistēmas darbības rezultātiem.

- kopējais monitorējamo preču skaits - informācija par datubāzē esošo unikālo produktu vienību daudzumu;
- cenu krituma statistika - saraksts ar precēm, kurām pēdējo 24 stundu laikā fiksēts lielākais cenas samazinājums procentuālā un naudas izteiksmē;
- veikalu veselības rādītāji - vizuāla indikācija par pēdējās skenēšanas sekmīgumu katrā no pieciem e-komercijas resursiem;
- sistēmas žurnālu izraksts - administratoram pieejama informācija par AI modeļa veiktajām darbībām un parsēšanas noteikumu atjauninājumiem.

2. Cenu svārstību interaktīvie grafiki uz ekrāna: Vizuāla datu interpretācija, kas ļauj analizēt cenu dinamiku ilgtermiņā un salīdzināt veikalu cenu politiku.

- līniju diagramma - attēlo konkrētas preces (piemēram, "Dior Sauvage 100ml") cenu izmaiņas laikā, kur X ass ir datums un Y ass ir cena eiro;
- veikalu salīdzinājuma leģenda - katrs veikals tiek attēlots ar savu unikālo krāsu (piemēram, Douglas - zaļš, Notino - rozā), ļaujot vienā grafikā redzēt vairāku konkurentu cenas;
- labākās cenas marķieris - sistēma automātiski aprēķina un grafikā izceļ vēsturiski zemāko fiksēto cenu konkrētajam produktam.

3. Sistēmas paziņojumi: Reāllaika informācija, kas tiek nosūtīta lietotājam, kad iestājas konkrēts notikums tirgū.

- Telegram paziņojums - ziņojums, kas satur preces nosaukumu, saiti uz veikalu, veco cenu, jauno cenu un aprēķināto ietaupījuma apjomu;
- E-pasta paziņojums - strukturēts HTML sūtījums ar vizuālu preces attēlu un detalizētu informāciju par cenas kritumu zem lietotāja iestatītā sliekšņa;

- tīmekļa saskarnes brīdinājumi - uznirstošie logi par veiksmīgu iestatījumu saglabāšanu vai kļūdām skenēšanas procesā.

4. Pielāgotas tirgus analītikas atskaites failos (.xlsx un .pdf formātos): Lietotāja konfigurēta izvades informācija manuālai datu apstrādei vai arhivēšanai. Sistēma ļauj lietotājam pirms atskaites ģenerēšanas izvēlēties specifiskus filtrus.

- atskaites periods - lietotāja izvēlēts laika posms (piemēram, no 01.01.2026. līdz 01.04.2026.), par kuru tiek iekļauti dati;
- atlasītie zīmoli un preces - informācija tiek ģenerēta tikai par konkrētiem ražotājiem vai produktu grupām pēc lietotāja izvēles;
- Excel (.xlsx) faila struktūra - satur tabulu ar šādām kolonnām: Zīmols, Nosaukums, Tilpums, Veikals, Fiksēšanas datums, Aktuālā cena, Iepriekšējā cena, Izmaiņa (EUR/%);
- PDF (.pdf) faila struktūra - noformēts dokuments ar uzņēmuma MarketPulse logotipu, atskaites kopsavilkumu, cenu dinamikas grafikiem un vizuālu labāko piedāvājumu sarakstu.

2.2. Funkcionālās prasības

1. Lietotāja autorizācija sistēmā

Šī funkcija nodrošina drošu piekļuvi sistēmas personalizētajam panelim, veicot lietotāja e-pasta adreses un šifrētās paroles validāciju. Sekmīgas autorizācijas gadījumā tiek izveidota unikāla **JWT** sesija, kas ļauj lietotājam droši izmantot sistēmas resursus bez atkārtotas pieteikšanās, savukārt kļūdu gadījumā sistēma izvada atbilstošu brīdinājuma paziņojumu.

Identifikators	PS 2.2.1
Ievads	
Funkcija paredzēta, lai lietotājs varētu pieslēgties sistēmai, un varētu izmantot dotās sistēmas funkcijas.	
Ievaddati	
1. E-pasts - obligāts teksta lauks. 2. Parole - obligāts lauks, slēpts ievades laikā.	
Apstrāde	
1. Ja e-pasts un parole ir apstiprināta, lietotājs piekļūst sistēmai. 2. Ja ir notikusi kļūda ievaddatos, tiek paziņota kļūda.	
Izvaddati	
1. Tiek dota piekļuve sistēmas panelim. 2. Ja notikusi kļūda ievaddatos, tiek paziņota kļūda.	
Kļūdas paziņojumi	
1. Ievadīts nepareizs e-pasts vai parole.	
Izsaucamās prasības	
Var veikt jebkurš reģistrēts sistēmas lietotājs, kuram ir piekļuve internetam.	

2. Preču meklēšana un pievienošana sekošanas saraksta

Funkcija piedāvā lietotājam iespēju veikt meklēšanu apvienotajā parfimērijas katalogā, izmantojot brīvteksta atslēgvārdus, un pievienot izvēlētās preces personīgajam "Favorites" sarakstam. Process ietver arī specifiska cenu sliekšņa iestatīšanu, kas kalpo par pamatu automatizētu paziņojumu saņemšanai brīdī, kad tirgus cena nokrītas līdz norādītajam līmenim.

Identifikators	PS 2.2.2
Ievads	
Funkcija ļauj lietotājam atrast interesējošo parfimēriju un iestatīt vēlamo cenu tās uzraudzībai.	
Ievaddati	
1. Meklēšanas frāze - teksta lauks. 2. Vēlamā cena - skaitliska vērtība.	
Apstrāde	
1. Sistēma meklē atbilstības datubāzē. 2. Sistēma saglabā lietotāja norādīto cenas slieksni.	
Izvaddati	
1. Atlasīto preču saraksts. 2. Apstiprinājums par pievienošanu sarakstam.	
Kļūdas paziņojumi	
1. Prece ar šādu nosaukumu nav atrasta.	
Izsaucamās prasības	
Var veikt autorizēts sistēmas lietotājs.	

3. Preču filtrēšana un cenu vēstures analīze

Šis modulis nodrošina padziļinātu tirgus datu analīzi, ļaujot filtrēt informāciju pa pieciem galvenajiem veikaliem. Sistēma automātiski ģenerē interaktīvus cenu svārstību grafikus un aprēķina statistikas rādītājus, piemēram, vēsturiski zemāko un vidējo tirgus cenu, palīdzot lietotājam pieņemt pamatotu lēmumu par pirkumu.

Identifikators	PS 2.2.3
Ievads	
Funkcija ļauj lietotājam apskatīt konkrētu preču cenu vēsturi, filtrējot datus pēc iepriekš definētiem veikaliem (Douglas, Notino, Makeup, 220.lv, 1a.lv).	
Ievaddati	
1. Veikala izvēle - izvēles saraksts. 2. Konkrētas preces izvēle no ģenerētā kataloga.	
Apstrāde	
1. Sistēma izgūst izvēlētās preces cenu vēsturi datubāzē norādītajam veikalam. 2. Sistēma ģenerē vizuālu cenu svārstību grafiku.	
Izvaddati	
1. Interaktīvs grafiks ar cenu izmaiņām. 2. Detalizēta statistika (vidējā cena, zemākā cena).	
Kļūdas paziņojumi	
1. Izvēlētajam veikalam nav vēsturisko datu par šo preci.	
Izsaucamās prasības	
Var veikt jebkurš reģistrēts sistēmas lietotājs.	

4. Autonoma datu ieguve un AI "pašārstēšanās"

Šī ir viena no unikālākajām sistēmas funkcijām, kas nodrošina pilnībā autonomu datu vākšanu, izmantojot asinhronus pieprasījumus un pārlūkprogrammas emulāciju. Ja mērķa vietnes struktūra mainās, sistēma aktivizē **Google Gemini AI** moduli, kas analizē HTML kodu un automātiski atjauno parsēšanas noteikumus (CSS selektorus), nodrošinot nepārtrauktu sistēmas darbību bez programmētāja iejaukšanās.

Identifikators	PS 2.2.4
Ievads	
Funkcija nodrošina nepārtrauktu datu vākšanu no e-komercijas resursiem un automātisku datu ieguves noteikumu atjaunošanu, izmantojot mākslīgo intelektu.	
Ievaddati	
1. Mērķa veikalu domēnu saraksts. 2. Google Gemini AI API piekļuves atslēga. 3. Skenēšanas intensitātes iestatījumi.	
Apstrāde	
1. Sistēma veic asinhronus pieprasījumus uz veikalu tīmekļa vietnēm. 2. Ja dati netiek atrasti vai struktūra ir mainījusies, sistēma nodod vietnes HTML kodu AI modulim. 3. AI modelis identificē jaunos CSS selektorus un atjaunina ierakstus datubāzē.	
Izvaddati	
1. Aktualizēts preču katalogs. 2. Sistēmas veselības rādītājs (Health Score) administrācijas panelī.	
Kļūdas paziņojumi	
1. Mērķa resurss ir bloķējis piekļuvi (403 Forbidden). 2. AI modelis nespēj identificēt preces elementus.	
Izsaucamās prasības	
Funkcija tiek izpildīta autonomi pēc APScheduler grafika vai pēc administratora pieprasījuma.	

5. Personalizētu tirgus analītikas atskaišu eksportēšana

Funkcija ļauj lietotājiem transformēt datubāzē uzkrāto informāciju strukturētos dokumentos tālākai manuālai analīzei. Atkarībā no izvēlētajiem filtriem un laika perioda, sistēma ģenerē **Excel** failus ar detalizētu preču sarakstu vai noformētus **PDF** ziņojumus, kuros iekļauti vizualizēti cenu dinamikas grafiki un tirgus kopsavilkumi.

Identifikators	PS 2.2.5
Ievads	
Funkcija ļauj lietotājam ģenerēt un lejupielādēt strukturētus datus par cenu svārstībām manuālai analīzei vai atskaitēm.	
Ievaddati	
1. Analīzes laika periods. 2. Atlasītās preču grupas vai zīmoli. 3. Izvades faila formāts (Excel vai PDF).	
Apstrāde	
1. Sistēma veic datu atlasi no tabulām product un pricehistory atbilstoši filtriem. 2. Tiek veikta datu transformācija uz .xlsx vai .pdf struktūru, izmantojot sistēmas veidnes.	
Izvaddati	
1. Lejupielādējams fails ar tirgus kopsavilkumu un cenu dinamiku.	
Kļūdas paziņojumi	
2. Atlasītajā periodā nav pieejami dati par izvēlētajām precēm.	
Izsaucamās prasības	
Var veikt autorizēts sistēmas lietotājs.	

6. Administratora sistēmas uzraudzība un vadība

Šī funkcija piešķir administratoram pilnas tiesības pārvaldīt sistēmas resursus, tostarp iespēju manuāli aktivizēt skenēšanas procesus un uzraudzīt to izpildi reāllaikā. Modulis ietver arī piekļuvi sistēmas žurnāliem (Logs).

Identifikators	PS 2.2.6
Ievads	
Funkcija nodrošina administratoram pilnu kontroli pār datu ieguves procesiem un sistēmas resursiem.	
Ievaddati	
1. Komanda "Run Scraper". 2. Veikala izvēle. 3. Piekļuve URL /admin.	
Apstrāde	
1. Sistēma pārbauda lietotāja tiesības (is_admin). 2. Palaiž asinhrono procesu un izvada reāllaika žurnālus.	
Izvaddati	
1. Aktīvs skenēšanas statuss ("RUNNING"). 2. Sistēmas logu plūsma.	
Kļūdas paziņojumi	
1. Piekļuve liegta (403 Forbidden).	
Izsaucamās prasības	
Var veikt tikai lietotājs ar administratora tiesībām.	

Lai nodrošinātu pilnīgu sistēmas darbību, ir realizētas šādas papildu funkcijas, kas aprakstītas saskaņā ar standartu:

7. Lietotāja profila un onbordinga pārvaldība:

    7.1. Jānodrošina jauna lietotāja reģistrācija, pārbaudot e-pasta unikālumu datubāzē;

    7.2. Pēc pirmās autorizācijas sistēmai jāpiedāvā onbordings prioritāro zīmolu un veikalu izvēlei;

    7.3. Lietotājam jābūt iespējai nomainīt paroli un saziņas valodu iestatījumos.

8. Telegram paziņojumu kanāla konfigurēšana:

    8.1. Sistēmai jāģenerē unikāls sasaistes kods (Token) lietotāja konta pievienošanai Telegram botam;

    8.2. Jānodrošina iespēja ieslēgt vai izslēgt paziņojumus par katru sekojamo preci atsevišķi;

    8.3. Jāļauj lietotājam iestatīt "klusuma periodu", kurā paziņojumi netiek sūtīti.

9. Preču identitātes sasaistes loģika:

    9.1. Sistēmai automātiski jāatpazīst identiskas preces dažādos veikalos, balstoties uz normalizētu nosaukumu un tilpumu;

    9.2. Jauna prece ir jāpiesaista esošai productgroup, ja sakritības koeficients pārsniedz 95%;

    9.3. Administratoram jābūt iespējai manuāli atvienot nepareizi sasaistītas preces.

10. Administratora sistēmas uzraudzība:

    10.1. Jānodrošina reāllaika piekļuve skenēšanas žurnāliem kļūdu diagnosticēšanai;

    10.2. Administratoram jābūt iespējai manuāli palaist skenēšanas ciklu konkrētam veikalam;

    10.3. Sistēmai jāattēlo statistika par kopējo preču skaitu, aktīvajiem lietotājiem un AI modeļa veiktspēju.

2.3. Nefunkcionālās prasības

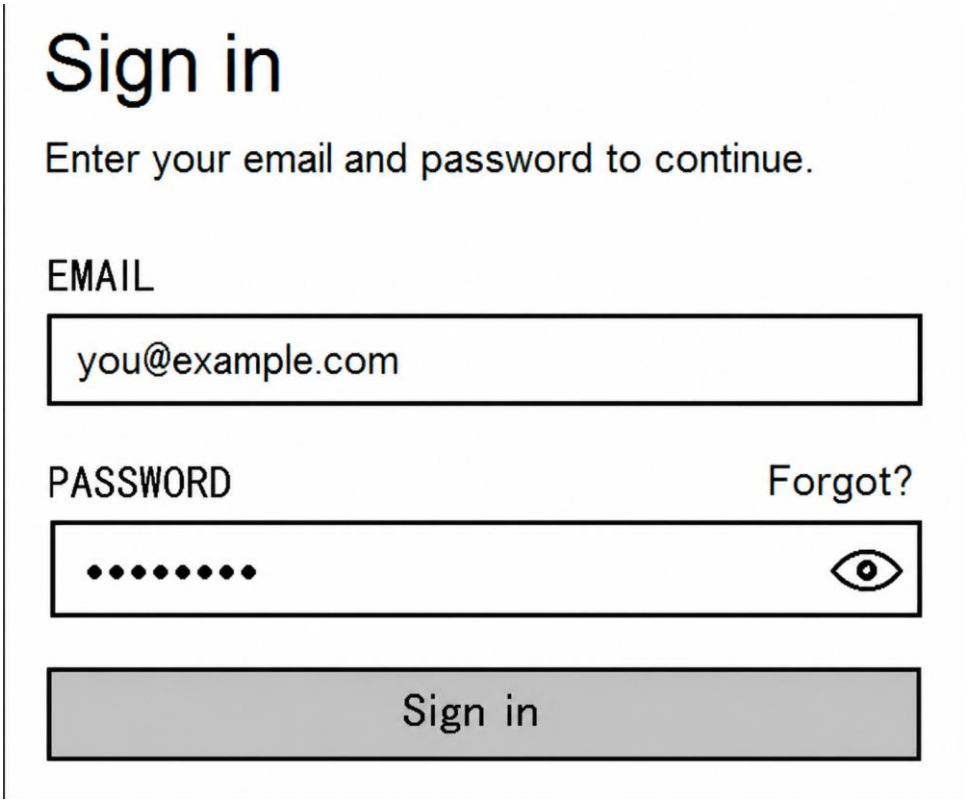
Sistēmas "MarketPulse" nefunkcionālās prasības nosaka tehniskos ierobežojumus un kvalitātes kritērijus, kas nodrošina programmatūras stabilu darbību, augstu drošības līmeni un lietotājam draudzīgu vidi. Prasības ir iedalītas četros galvenajos blokos.

1. Veiktspējas un pieejamības prasības:
    - Sistēmai jānodrošina asinhrona datu ieguve no mērķa resursiem (Notino, Douglas u.c.), lai pilns tirgus skenēšanas cikls, darbojoties paralēliem procesiem, nepārsniegtu **60 minūtes**;
    - Tīmekļa saskarnei jāielādē analītiskie cenu grafiki un preču saraksti ne ilgāk kā 2 sekunžu laikā, izmantojot Redis [16] kešatmiņas mehānismus;
    - Sistēmas backend daļai jābūt izvietotai uz DigitalOcean mākoņservera, nodrošinot 99.9% sistēmas pieejamību 24/7 režīmā;
    - AI "pašārstēšanās" modulim jāspēj noģenerēt jauni CSS selektori un atjaunot noteikumi datubāzē ne ilgāk kā 30 sekunžu laikā pēc kļūdas fiksēšanas.
2. Lietojamības un saskarnes prasības:
    - Sistēmas saskarnei jābūt izstrādātai latviešu valodā;
    - Jānodrošina responsīvs dizains, kas automātiski pielāgojas dažādiem ekrāna izmēriem (datori, planšetdatori, viedtālruņi);
    - Sistēmai jābūt saderīgai ar populārākajām pārlūkprogrammām: Google Chrome, Mozilla Firefox un Safari;
    - Vizuālajā noformējumā jāizmanto tumšā B2B krāsu palete (dziļi zilie un tirkīza toņi), lai izceltu analītiskos datus un grafikus.
3. Drošības un uzticamības prasības:
    - Piekļuve lietotāja datiem un sekošanas sarakstiem ir atļauta tikai autorizētiem lietotājiem, izmantojot drošu JWT (JSON Web Token) mehānismu;
    - Lietotāju paroles datubāzē jāsaglabā tikai šifrētā veidā (izmantojot bcrypt [10] algoritmu);
    - Datu integritāte jānodrošina, izmantojot PostgreSQL [12] ārējo atslēgu ierobežojumus, nepieļaujot "bāreņu" ierakstu veidošanos cenu vēsturē;

- Sistēmai jāveic automātiska datu validācija, reģistrējot visas kļūdas sistēmas žurnālos administratora uzraudzībai.

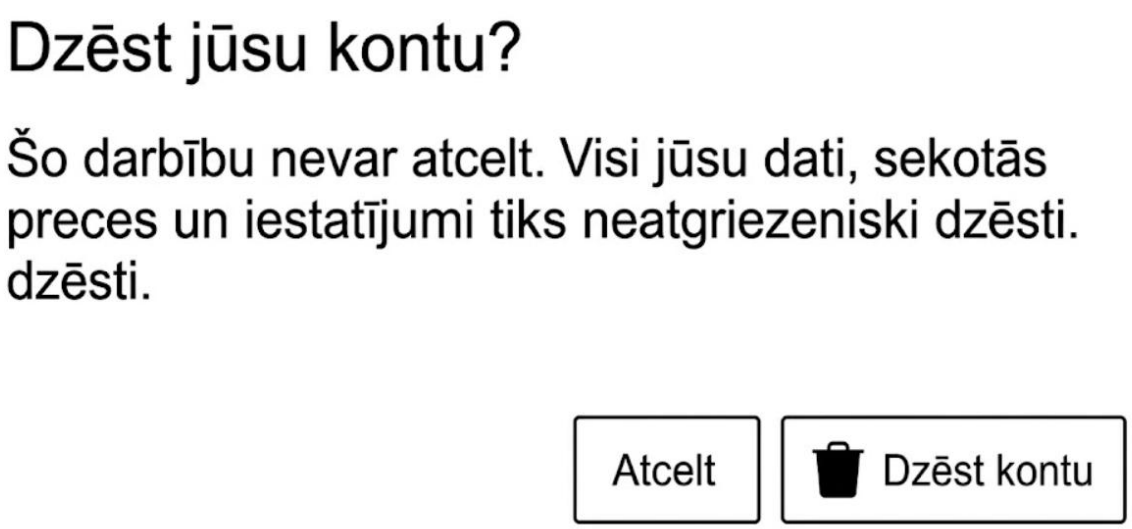
Lai vizualizētu plānoto lietotāja mijiedarbību ar sistēmu, ir izstrādātas **piecas galvenās ekrānformu skices**.

4. Lietotāja pieslēgšanās formai jāizskatās sekojoši (skat. 2.att.).



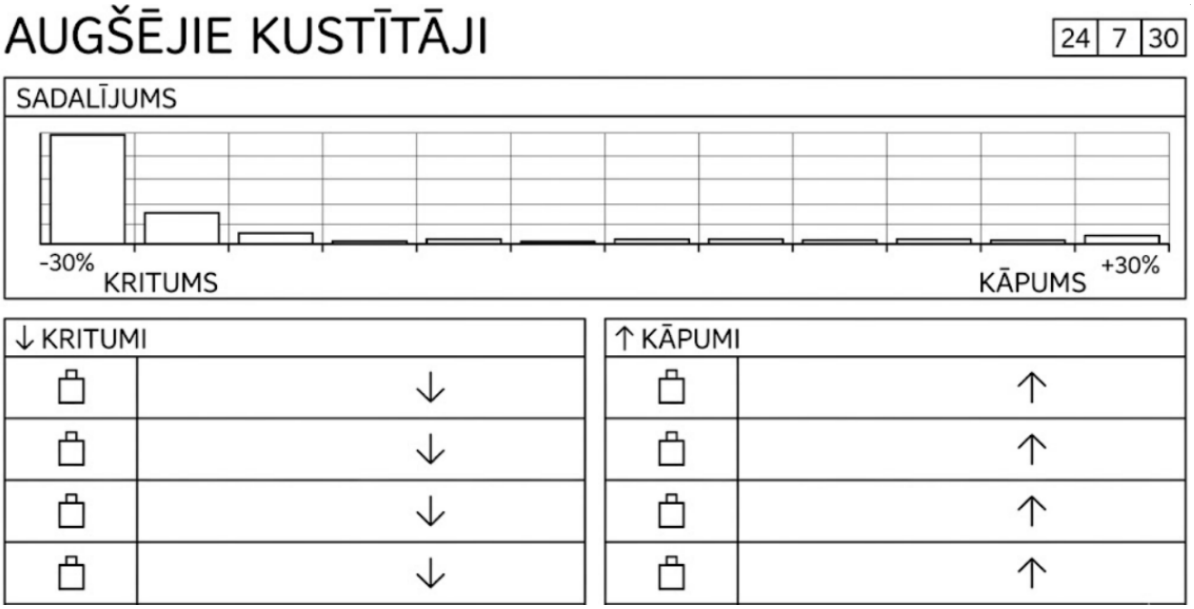
2. att. Lietotāja autorizācijas formas skice

5. Lietotāja profila dzēšanas sīktēlim jāizskatās sekojoši (skat. 3.att.).



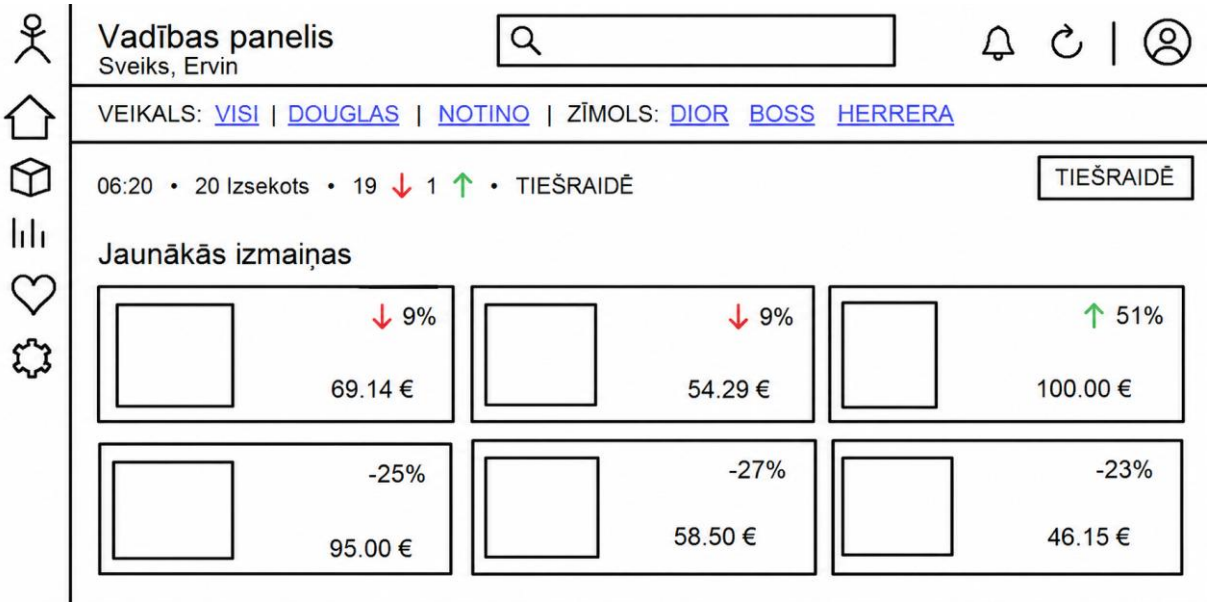
3. att. Lietotāja profila rediģēšanas formas skice

6. Analītikas tīmekļa lapai jāizskatās sekojoši (skat. 4.att.).



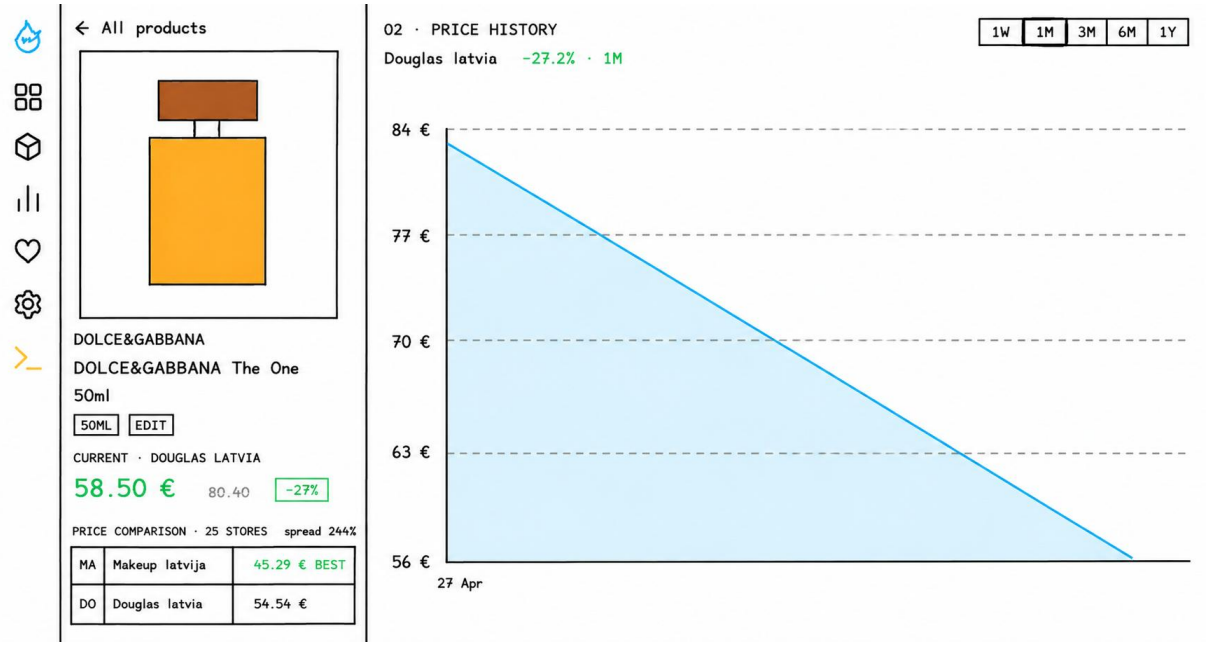
4. att. Tirgus aktuālo izmaiņu analītiskā moduļa skice

7. Preču katalogam jāizskatās sekojoši (skat. 5.att.).



5. att. Apvienotā preču kataloga un datu bāzes pārskata skice

8. Produkta kartei jāizskatās sekojoši (skat. 6.att.).



6. att. Produkta karte iekšā

3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS

"MarketPulse" ir sarežģīta datu ieguves un analītikas tīmekļa platforma, kas paredzēta liela apjoma informācijas apstrādei, izmantojot mākslīgā intelekta elementus. Sistēma sastāv no divām galvenajām daļām: autonomā datu ieguves dziņa (Backend / Discovery Engine) un datu attēlošanas saskarnes (Frontend). Ņemot vērā e-komercijas vietņu (Notino, Douglas u.c.) sarežģīto arhitektūru (React, dinamiski XHR pieprasījumi) un aizsardzību pret robotiem, tika izvēlēti modernākie rīki un bibliotēkas.

Sistēmas izstrādē tika izmantotas šādas galvenās tehnoloģijas un rīki:

- **Python 3.12** [13]**:** izvēlēta kā galvenā programmēšanas valoda tās jaudīgās datu apstrādes ekosistēmas un asinhronās programmēšanas atbalsta (asyncio) dēļ, kas ir kritiski svarīgi ātrai un paralēlai resursu skenēšanai;
- **Google Gemini AI (google-genai bibliotēka):** izmantots *gemini-3-flash-preview* modelis sistēmas "pašārstēšanās" (Self-Healing) mehānisma nodrošināšanai. Tas tika izvēlēts tā izcilā ātruma un precizitātes dēļ, ģenerējot CSS selektorus un JSON datu struktūras no bojāta HTML koda;
- **Playwright**: jaudīgs pārlūkprogrammas automatizācijas rīks. Tiek izmantots kā "Hibrīdā dziņa" smagā daļa, lai renderētu JavaScript balstītas lapas un simulētu fiziskus lietotāja klikšķus (piemēram, "Deep Dive" variantu atvēršanai Notino vietnē);
- **curl_cffi**: moderns HTTP klients, kas spēj atdarināt reālu pārlūkprogrammu TLS un JA3 nospiedumus (Fingerprinting). Tiek izmantots ātrai un "neredzamai" datu ieguvei no aizsargātiem domēniem, kur pilna Playwright emulācija būtu pārāk lēna;
- **FastAPI** [7] tika izvēlēts kā augstas veiktspējas ietvars sistēmas REST API realizēšanai, nodrošinot pilnu asinhronitātes atbalstu paralēlai datu apstrādei un automātisku Swagger dokumentācijas ģenerēšanu. Šis rīks garantē drošu datu validāciju, izmantojot Pydantic modeļus, kas būtiski paātrina izstrādes procesu un uzlabo visas sistēmas stabilitāti.
- **PostgreSQL un SQLModel** [17]**:** relāciju datubāžu pārvaldības sistēma un ORM (Object-Relational Mapping) bibliotēka. Nodrošina drošu un strukturētu milzīga apjoma datu (cenu vēstures punktu, preču variantu) glabāšanu un ātru sasaisti;
- **Redis**: izmantots kā augstas veiktspējas datu struktūru krātuve operatīvajā atmiņā. Nodrošina ātru datu kešatmiņu un asinhrono uzdevumu rindu pārvaldību, kas ievērojami paātrina datu apstrādi un samazina slodzi uz galveno datubāzi skenēšanas laikā;

- **Docker** [6]**:** izmantots sistēmas un tās atkarību konteinerizācijai. Nodrošina ātru datubāzes (PostgreSQL, Redis) un citu izolētu komponentu palaišanu, garantējot sistēmas vienmērīgu darbību gan izstrādes, gan produkcijas serveros bez konfigurācijas konfliktiem;
- **Visual Studio Code:** izvēlēta kā galvenā integrētā izstrādes vide (IDE), jo tā ir ātra, atbalsta plašu Python paplašinājumu klāstu un nodrošina ērtu atkļūdošanas (debugging) procesu sistēmas izstrādes laikā;
- **DigitalOcean:** mākoņpakalpojumu platforma (Cloud Platform), kas izmantota sistēmas izvietošanai (hostingam). Projekta darbināšanai izvēlēts *DigitalOcean Droplet* (virtuālais privātais serveris), kas nodrošina nepārtrauktu datubāzes un aizmugursistēmas (Backend) darbību un pieejamību caur globālo tīmekli;
- **Git un GitHub** [8]**:** izmantoti versiju kontrolei un drošai koda uzglabāšanai mākoņpakalpojumā, nodrošinot iespēju izsekot izmaiņām un atgriezties pie iepriekšējām sistēmas versijām.

4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA

4.1. Sistēmas struktūras modelis

4.1.1. Sistēmas arhitektūra

Sistēma "**MarketPulse**" ir veidota no četrām galvenajām apakšsistēmām (skat. 1.pielikums. Funkcionālās dekompozīcijas diagramma), kas nodrošina pilnu datu dzīves ciklu - no ieguves līdz padziļinātai analītikai. Katra apakšsistēma sastāv no specializētiem moduļiem.

DATU IEGUVES APAKŠSISTĒMA nodrošina informācijas vākšanu no e-komercijas vietnēm, izmantojot dažādas ielādes metodes. Tā ietver 9 moduļus:

- **mērķa resursu skenēšanas moduli**, kas atbild par sākotnējo veikalu katalogu apstaigāšanu un saišu apkopošanu;
- **hibrīdās ielādes moduli (Playwright/curl)**, kas dinamiski pārslēdzas starp ātru pieprasījumu veikšanu un pilnu pārlūka emulāciju JavaScript renderēšanai;
- **preču parsēšanas moduli (Deep Dive)**, kas iegūst detalizētu informāciju par visiem preces variantiem un tilpumiem;
- **mērķa veikalu konfigurēšanas moduli**, kas ļauj administratoram definēt jaunus domēnus un to bāzes adreses;
- **skenēšanas grafiku vadības moduli**, kas kontrolē automatizētos datu vākšanas ciklus, izmantojot asinhrono plānotāju;
- **kataloga dziņa moduli**, kas nodrošina navigāciju pa vairākām kataloga lapām (paginmāciju);
- **asinhronās ielādes moduli (curl-cffi)**, kas nodrošina "neredzamu" datu ieguvi no vietnēm ar augstu aizsardzību;
- **produktu saišu izgūšanas moduli**, kas identificē un saglabā unikālas preču adreses turpmākai apstrādei;
- **datu kvalitātes validācijas moduli**, kas veic sākotnējo pārbaudi, lai nepieļautu nepilnīgu datu nonākšanu sistēmā.

AI UN ANALĪTIKAS APAKŠSISTĒMA atbild par inteliģentu datu apstrādi, sasaisti un tirgus tendenču noteikšanu. Tā ietver 9 moduļus:

- **parsēšanas noteikumu ģenerēšanas moduli**, kas izmanto Google Gemini AI, lai radītu jaunus selektorus un nodrošinātu sistēmas "pašārstēšanos";
- **preču sasaistes moduli**, kas apvieno identiskas preces no dažādiem veikaliem vienotā produktu grupā;
- **cenu analīzes un fiksēšanas moduli**, kas fiksē katru izmaiņu un saglabā to vēstures tabulā;
- **CSS selektoru analīzes moduli**, kas pārbauda esošo noteikumu darbību pirms katra skenēšanas cikla;
- **AI noteikumu ģenerēšanas moduli**, kas sagatavo HTML fragmentus nodošanai mākslīgajam intelektam;
- **noteikumu manuālas korekcijas moduli**, kas ļauj administratoram pārbaudīt un labot AI veiktās darbības;
- **nosaukumu normalizācijas moduli**, kas attīra datus no liekiem vārdiem, izveidojot standartizētu nosaukumu formātu;
- **krājumu statusa uzskaites moduli**, kas seko līdzi preču pieejamībai katrā no monitorētajiem veikaliem;
- **tirgus tendenču aprēķināšanas moduli**, kas ģenerē kopējos statistikas rādītājus par cenu svārstībām tirgū.

LIETOTĀJU PĀRVALDĪBAS APAKŠSISTĒMA nodrošina drošu piekļuvi un personalizētu lietotāja pieredzi. Tā ietver 10 moduļus:

- **lietotāju autorizācijas moduli**, kas pārvalda piekļuvi sistēmas pamatfunkcijām;
- **sekošanas sarakstu rediģēšanas moduli**, kur lietotāji pārvalda sev interesējošās preces;
- **paziņojumu izsūtīšanas moduli**, kas informē lietotājus par cenas kritumu zem iestatītā sliekšņa;
- **autorizācijas moduli (JWT)**, kas nodrošina drošas sesijas, izmantojot JSON Web Token tehnoloģiju;
- **profila datu vadības moduli**, kas ļauj lietotājam atjaunot savu informāciju un mainīt paroli;

- **onboarding iestatījumu moduli**, kas palīdz jaunajiem lietotājiem izvēlēties prioritāros zīmolus un veikalus;
- **sekošanas saraksta izveides moduli**, kas piesaista izvēlētās preces lietotāja profilam;
- **cenu sliekšņu definēšanas moduli**, kur lietotājs norāda mērķa cenu paziņojumu saņemšanai;
- **Telegram sasaistes vadības moduli**, kas ģenerē kodus lietotāja konta pievienošanai ziņapmaiņas botam;
- **konta deaktivizēšanas moduli**, kas nodrošina lietotāja datu un iestatījumu dzēšanu pēc pieprasījuma.

VIZUALIZĀCIJAS UN ATSKAIŠU APAKŠSISTĒMA nodrošina apstrādāto datu attēlošanu lietotājam saprotamā un analītiskā veidā. Tā ietver 14 moduļus:

- **galvenā paneļa moduli**, kas vizualizē aktuālākos tirgus datus un izmaiņas;
- **interaktīvo cenu grafiku moduli**, kas attēlo vēsturiskās cenu svārstības konkrētai precei, izmantojot Recharts [15];
- **tirgus cenu tendenču analīzes moduli**, kas parāda vidējās cenas izmaiņas visā parfimērijas segmentā;
- **cenu krituma histogrammu moduli**, kas vizuāli sadala atlaides pēc to lieluma;
- **volatilitātes diagrammu moduli**, kas ļauj identificēt preces ar vislielākajām cenu svārstībām;
- **zīmolu un veikalu matricas moduli**, kas parāda preču izplatību dažādos e-komercijas resursos;
- **kataloga meklēšanas moduli**, kas nodrošina ātru piekļuvi visām datubāzē esošajām precēm;
- **datu filtrēšanas saskarnes moduli**, kas ļauj atlasīt datus pēc veikaliem, zīmoliem un tilpuma;
- **datu eksporta Excel failā moduli**, kas ģenerē strukturētas tabulas padziļinātai analīzei;
- **atskaišu ģenerēšanas PDF formātā moduli**, kas izveido noformētus tirgus pārskatus ar grafikiem;

- **Telegram paziņojumu servisa moduli**, kas sūta tūlītējus brīdinājumus uz lietotāja viedtālruni;
- **e-pasta brīdinājumu izsūtīšanas moduli**, kas nosūta HTML formāta paziņojumus par cenu kritumu;
- **sistēmas veselības indikācijas moduli**, kas vizualizē skenēšanas procesu sekmību administratoram;
- **skenēšanas žurnālu plūsmas moduli**, kas nodrošina reāllaika pārskatu par AI un sistēmas veiktajām darbībām.

4.1.2. Sistēmas ER modelis

Datu bāzes struktūra nodrošina mērogojamību pārejai no CLI rīka uz pilnvērtīgu tīmekļa platformu. Sistēmas konceptuālais datu modelis (skat. 2.pielikums. Sistēmas ER-diagramma) demonstrē galvenās entītijas un to savstarpējo saistību.

Sistēmas galvenās entītijas:

- **LIETOTĀJS:** glabā informāciju par reģistrētajiem lietotājiem (e-pasts, parole, loma);
- **SEKOŠANAS_SARAKSTS:** ļauj lietotājiem iestatīt vēlamās cenas sliekšņus konkrētām produktu grupām;
- **PAZIŅOJUMS:** fiksē izsūtītos brīdinājumus par cenu kritumiem vai izmaiņām;
- **PRODUKTU_GRUPA:** sistēmas centrālais ieraksts, kas apvieno identiskas preces no dažādiem veikaliem (Chaining loģika);
- **VEIKALS:** e-komercijas platformas (piemēram, Douglas, Notino), kurās tiek veikts monitorings;
- **PRECE:** konkrēta veikala piedāvājums, kas piesaistīts attiecīgajai produktu grupai;
- **PRECES_VARIANTS:** konkrēts preces tilpums un koncentrācija, kas iegūts padziļinātas skenēšanas (Deep Dive) rezultātā;
- **CENU_VĒSTURE:** laikā rindoti datu punkti katram preces variantam, kas fiksē cenas un pieejamības izmaiņas;
- **PARSĒŠANAS_NOTEIKUMS:** AI ģenerētas datu ieguves instrukcijas konkrētam veikalam.

ER-modelis demonstrē entītiju savstarpējo saistību:

- starp entītijām **LIETOTĀJS** un **SEKOŠANAS_SARAKSTS** ir attiecība viens pret daudziem (1..N), jo viens lietotājs var izveidot vairākus sekošanas sarakstus;
- starp entītijām **LIETOTĀJS** un **PAZIŅOJUMS** ir attiecība viens pret daudziem (1..N), jo viens lietotājs var saņemt vairākus paziņojumus;
- starp entītijām **SEKOŠANAS_SARAKSTS** un **PRODUKTU_GRUPA** ir attiecība daudzi pret vienu (N..1), jo vairāki saraksti var novērot vienu centralizētu produktu grupu;
- starp entītijām **PRODUKTU_GRUPA** un **PRECE** ir attiecība viens pret daudziem (1..N), jo viena produktu grupa apvieno vairākas identiskas preces no dažādiem veikaliem;
- starp entītijām **VEIKALS** un **PRECE** ir attiecība viens pret daudziem (1..N), jo viens veikals piedāvā daudzas preces;
- starp entītijām **PRECE** un **PRECES_VARIANTS** ir attiecība viens pret daudziem (1..N), jo vienai precei var būt vairāki varianti (piemēram, 50ml un 100ml);
- starp entītijām **PRECES_VARIANTS** un **CENU_VĒSTURE** ir attiecība viens pret daudziem (1..N), jo katram variantam laika gaitā tiek reģistrēti vairāki cenu vēstures ieraksti;
- starp entītijām **VEIKALS** un **PARSĒŠANAS_NOTEIKUMS** ir attiecība viens pret daudziem (1..N), jo vienam veikalam var tikt izmantoti vairāki parsēšanas noteikumi.

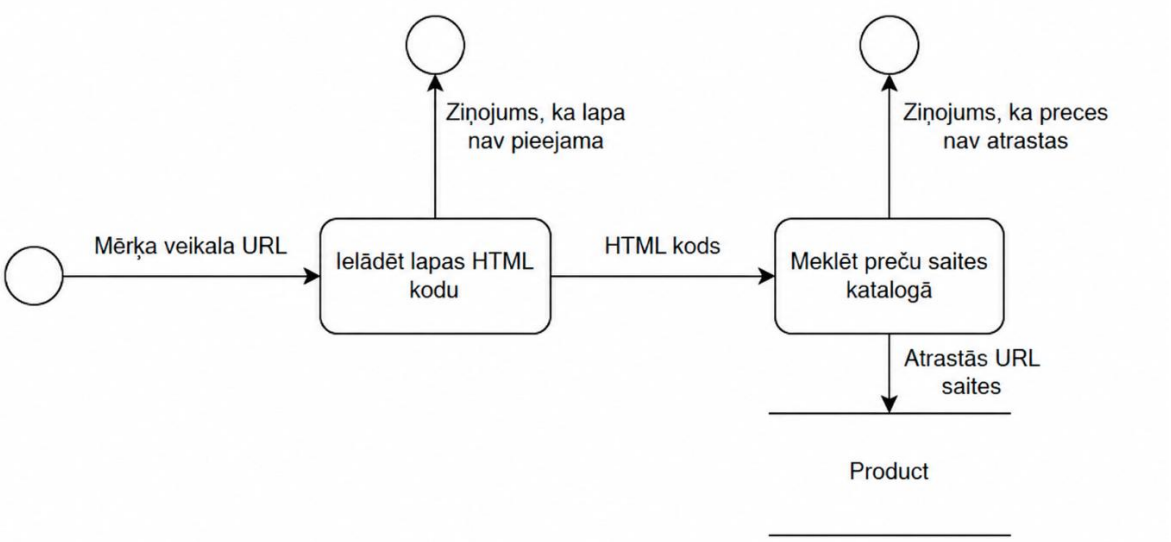
4.2. Funkcionālais sistēmas modelis

4.2.1. Datu plūsmu modelis

Šajā sadaļā ir aprakstīti septiņi būtiskākie MarketPulse sistēmas datu apstrādes un transformācijas procesi, kas parāda programmēšanas pieejas oriģinalitāti, tostarp mākslīgā intelekta (AI) un hibrīdās ielādes izmantošanu.

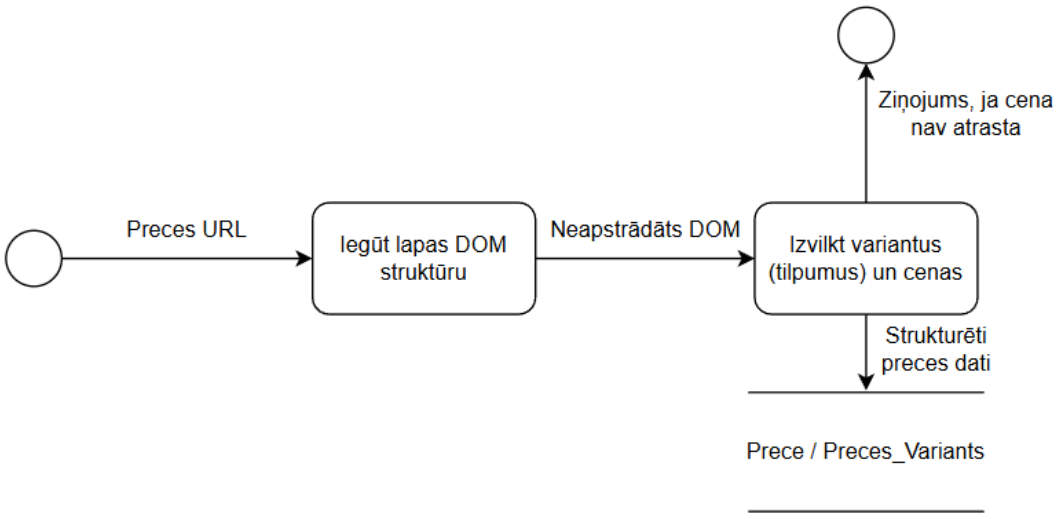
1. Mērķa resursu skenēšanas modulis (Catalog Crawler) (skat. 7. att.) Modulis nodrošina sākotnējo e-komercijas veikalu katalogu apstaigāšanu. Sistēma saņem mērķa resursa URL no konfigurācijas faila. Izmantojot asinhrono pieprasījumu dzinēju, modulis ielādē kataloga lapas HTML kodu. Pēc tam kods tiek analizēts, lai identificētu preču kartītes un izvilktu atsevišķu preču saites (URL). Ja katalogā ir vairākas lapas, modulis ģenerē nākamās

lapas pieprasījumu. Iegūtās preču saites tiek saglabātas sistēmas atmiņā tālākai padziļinātai apstrādei.



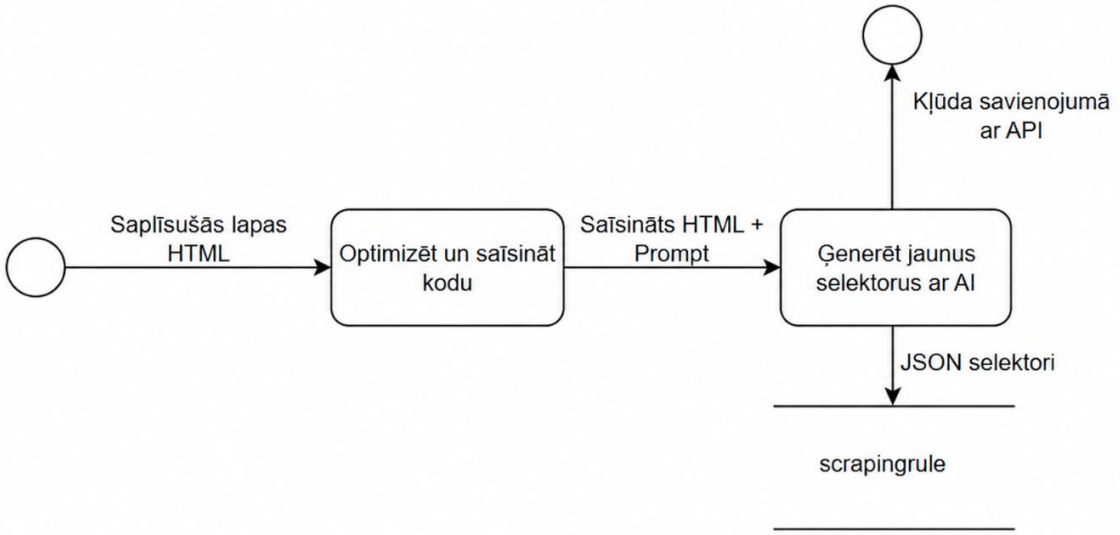
7.att. Mērķa resursu skenēšanas moduļa datu plūsmas diagramma

2. Preču parsēšanas modulis (Deep Dive apstrāde) (skat. 8. att.) Sistēma saņem konkrētas preces URL adresi no skenēšanas moduļa un izmanto hibrīdo ielādes dzini (curl_cffi vai Playwright), lai iegūtu lapas HTML struktūru. Pēc tam dati tiek padoti apstrādei, kur sistēma meklē "Deep Dive" trigerus. Process sadala HTML kodu, lai identificētu visus preces variantus (piem., 50ml, 100ml) un to specifiskās cenas no luigisbox/Tealium analītikas vai DOM elementiem, nofiltrējot lieko informāciju. Attīrītie dati tiek strukturēti un nodoti saglabāšanai datubāzē tabulā **product**.



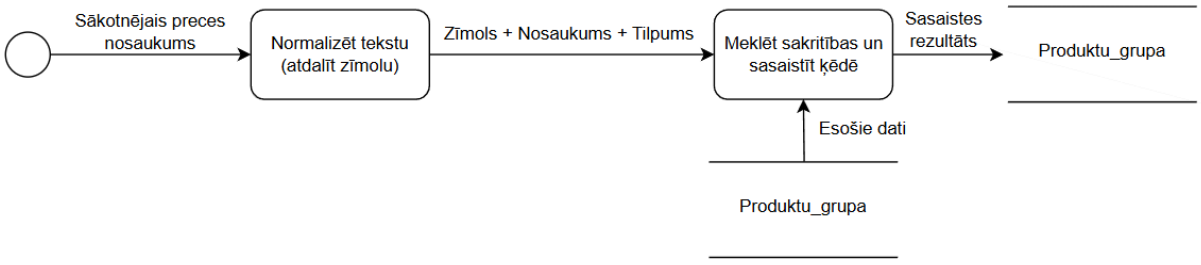
8.att. Preču parsēšanas moduļa datu plūsmas diagramma

3. Parsēšanas noteikumu ģenerēšanas modulis (Self-Healing AI) (skat. 9. att.) Ja sistēma skenēšanas laikā atklāj, ka vietnes dizains ir mainījies un vecie selektori vairs nestrādā, tiek aktivizēts "Self-Healing" process. Modulis ņem iegūto HTML kodu un, izmantojot teksta saīsināšanas algoritmu, sagatavo optimizētu datu paketi. Šī pakete tiek nosūtīta caur API uz Google Gemini AI modeli. Mākslīgais intelekts analizē kodu un ģenerē jaunus CSS selektorus JSON formātā. Pēc tam modulis validē saņemto JSON objektu un atjaunina ierakstu datubāzē.



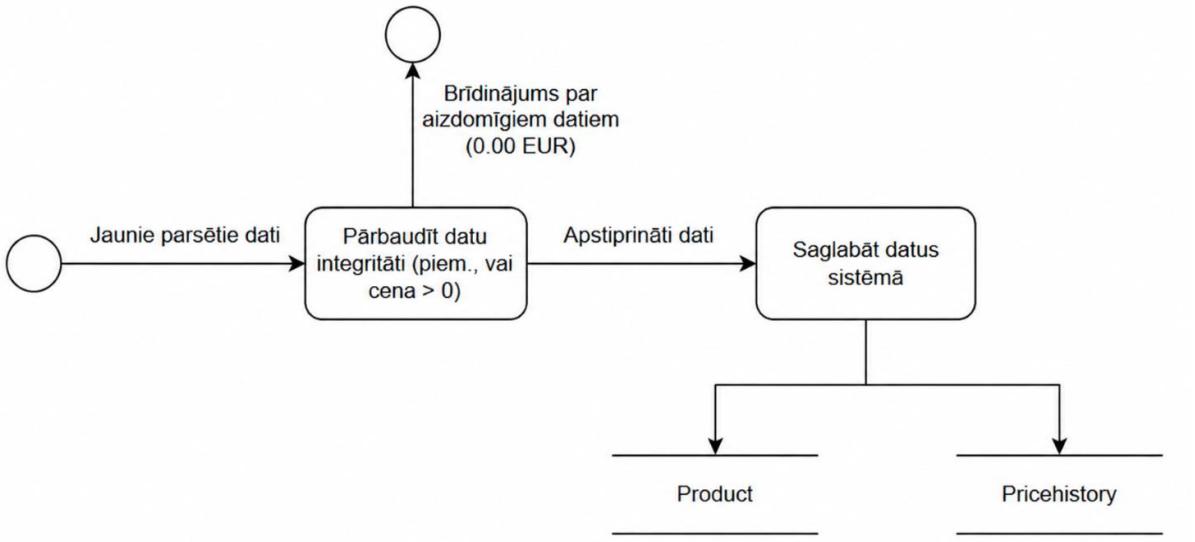
9.att. AI noteikumu ģenerēšanas moduļa datu plūsmas diagramma

4. Preču sasaistes modulis (Chaining loģika) (skat. 10. att.) Šis modulis atbild par datu normalizāciju un identisku preču apvienošanu. Jauniegūtās preces nosaukums tiek nodots normalizatoram, kas no teksta izveido standartizētu virkni (atdalot zīmolu un koncentrāciju). Normalizētie dati tiek salīdzināti ar jau eksistējošām entītijām datubāzē. Ja tiek atrasta sakritība, jaunā prece tiek piesaistīta esošajai ķēdei (Produktu grupai). Ja sakritību nav, sistēma ģenerē jaunu produktu grupu tabulā **productgroup**.



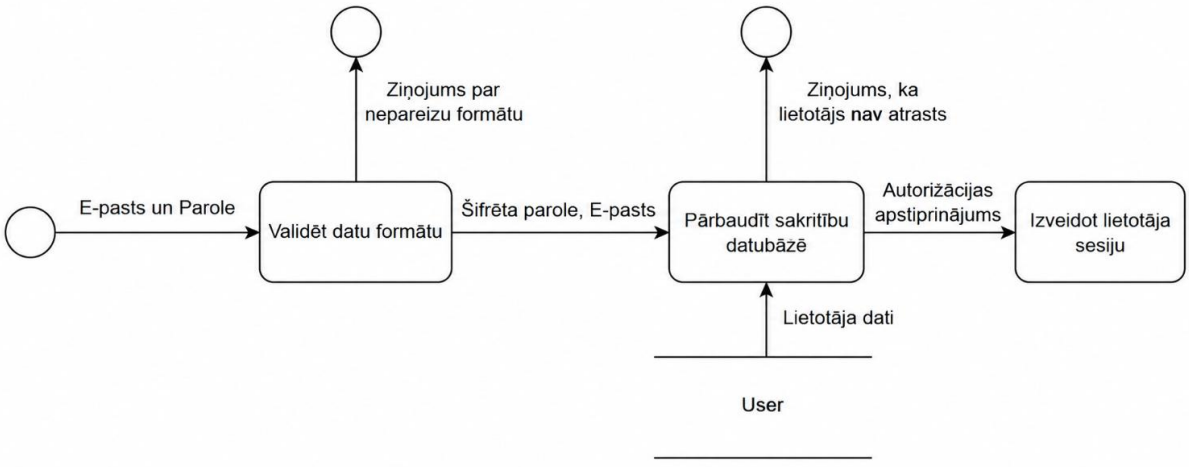
10.att. Preču sasaistes moduļa datu plūsmas diagramma

5. Datu kvalitātes validācijas modulis (skat. 11. att.) Pirms datu saglabāšanas galvenajā datubāzē, šis modulis veic iegūtās informācijas pārbaudi. Sistēma saņem strukturētos preču datus un pārbauda to integritāti (vai cena nav 0.00 EUR, vai ir norādīts tilpums un zīmols). Ja dati atbilst kvalitātes standartiem, tie tiek nodoti tālāk. Ja tiek konstatētas anomālijas, modulis reģistrē brīdinājumu sistēmas žurnālā (Logs) un nepieciešamības gadījumā ierosina "Self-Healing" procedūru.



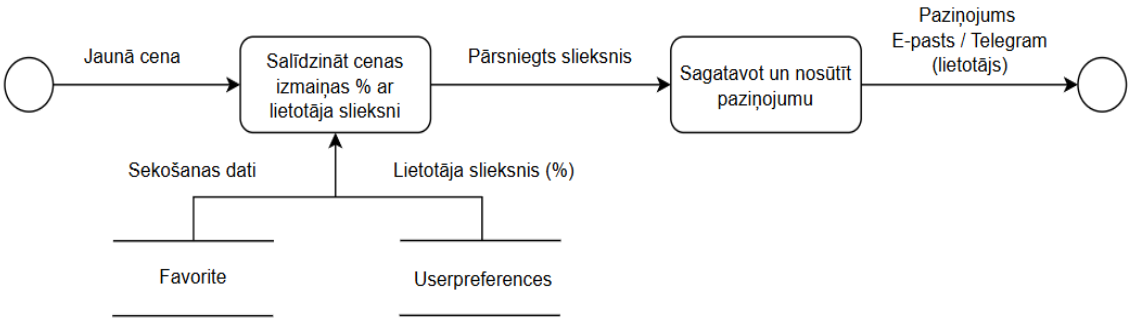
11.att. Datu kvalitātes validācijas moduļa datu plūsmas diagramma

6. Lietotāja autorizācijas modulis (skat. 12. att.) Modulis nodrošina drošu lietotāju piekļuvi sistēmas tīmekļa saskarnei. Lietotājs ievada savu e-pastu un paroli. Modulis pārbauda ievadīto datu formātu. Pēc tam parolē tiek pielietots jaukšanas (hashing) algoritms un rezultāts tiek salīdzināts ar datubāzē "Lietotājs" saglabāto vērtību. Veiksmīgas sakritības gadījumā modulis ģenerē autorizācijas sesiju un piešķir lietotājam atbilstošas piekļuves tiesības.



12.att. Lietotāja autorizācijas moduļa datu plūsmas diagramma

7. Sekošanas sarakstu un paziņojumu modulis (skat. 13. att.) Modulis nodrošina lietotāju informēšanu par cenu izmaiņām izsekotajām precēm. Kad sistēma fiksē jaunu cenas izmaiņu produktam, modulis pārbauda, vai šim produktam ir aktīvi sekošanas ieraksti tabulā favorite. Ja tādi ir, modulis aprēķina cenas izmaiņas procentuālo apjomu un salīdzina to ar lietotāja iestatīto slieksni. Ja izmaiņa pārsniedz slieksni, modulis sagatavo brīdinājuma paziņojumu un nosūta to lietotājam pa izvēlētajiem kanāliem - Telegram un/vai e-pastu (caur Resend HTTP API). Mērķa tabulas: favorite un userpreferences.



13.att. Paziņojumu moduļa datu plūsmas diagramma

5. DATU STRUKTŪRU APRAKSTS

Datubāzes izveidošanai ir nepieciešama datubāzes fiziskā struktūra. Tā tika iegūta normalizējot ER diagrammu, kuras rezultātā no entītijām izveidoja relācijas. Sistēmas "MarketPulse" datu slānis ir realizēts **PostgreSQL** vidē, izmantojot **SQLModel** bibliotēku datu manipulācijām no lietojumprogrammas puses (skat. 3.pielikums. Datubāzes tabulu shēma).

Kopumā datubāzes loģisko arhitektūru veido **8 galvenās tabulas**, kas ir savstarpēji saistītas, lai nodrošinātu pilnu tirgus monitoringa un datu analīzes ciklu. Šīs struktūras ietver visaptverošu informāciju par sistēmas lietotājiem un to individuālajām preferencēm paziņojumu saņemšanai, autonomās datu ieguves tehniskajiem noteikumiem un AI "pašārstēšanās" procesā izmantotajiem selektoru paraugiem, kā arī sistēmas analītisko kodolu - informāciju par apvienotajām preču grupām, konkrētiem veikalu piedāvājumiem un to vēsturiskajām cenu un krājumu statusa izmaiņām, kas nepieciešamas interaktīvās analītikas un vizuālo grafiku ģenerēšanai.

Sistēmas datu struktūra ietver šādas tabulas:

- **Tabula "productgroup"** - apvieno identiskas preces no dažādiem veikaliem vienotā analītiskā grupā;
- **Tabula "product"** - satur datus par konkrētiem preču piedāvājumiem no e-komercijas resursiem;
- **Tabula "pricehistory"** - glabā vēsturiskos datus par cenu un krājumu statusa izmaiņām;
- **Tabula "user"** - satur informāciju par reģistrētajiem sistēmas lietotājiem;
- **Tabula "userpreferences"** - glabā lietotāju personalizētos iestatījumus un Telegram paziņojumu datus;
- **Tabula "favorite"** - nodrošina lietotāja sekošanas saraksta (Watchlist) funkcionalitāti;
- **Tabula "scrapingrule"** - satur tehniskos noteikumus datu ieguvei un to precizitātes rādītājus;
- **Tabula "sharedpattern"** - kalpo kā sistēmas zināšanu bāze biežāk izmantotajiem CSS selektoriem.

5.1.tabula

Tabulas **"productgroup"** struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id (PK)	INTEGER	-	Unikāls grupas identifikators
2.	name	VARCHAR	255	Normalizēts preces grupas nosaukums
3.	category	VARCHAR	100	Preču kategorija (piem., “Smaržas”)
4.	created_at	TIMESTAMP	-	Ieraksta izveides datums un laiks

Tabula **"productgroup"** ir saistīta ar tabulu **"product"** ar saiti **viens-pret-daudziem (1..N)**, jo viena produktu grupa var apvienot piedāvājumus no vairākiem veikaliem.

5.2.tabula

Tabulas **"product"** struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id (PK)	INTEGER	-	Unikāls preces identifikators
2.	url	VARCHAR	500	Tiešā saite uz preces lapu veikalā
3.	title	VARCHAR	255	Oriģinālais preces nosaukums no vietnes
4.	brand	VARCHAR	100	Normalizēts zīmola nosaukums
5.	volume	VARCHAR	20	Normalizēts tilpums (piem., “100 ml”)
6.	concentration	VARCHAR	20	Koncentrācijas tips (EDP, EDT u.c.)
7.	last_price	DECIMAL	12,2	Aktuālā fiksētā cena eiro
8.	old_price	DECIMAL	12,2	Iepriekšējā cena (atlaižu analīzei)
9.	search_term	VARCHAR	50	Veikala nosaukums (Notino, Douglas u.c.)
10.	group_id (FK)	INTEGER	-	Sasaiste ar tabulu “productgroup”

Tabula **"product"** satur saiti ar tabulu **"pricehistory"** ar kardinalitāti **viens-pret-daudziem (1..N)**, nodrošinot iespēju katram piedāvājumam uzkrāt vairākus cenu punktus laika gaitā.

5.3.tabula

Tabulas **"pricehistory"** struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id (PK)	INTEGER	-	Vēstures ieraksta identifikators
2.	price	DECIMAL	12,2	Cena fiksēšanas brīdī
3.	in_stock	BOOLEAN	-	Pieejamība noliktavā fiksēšanas brīdī
4.	parsed_at	TIMESTAMP	-	Datu iegūšanas precīzs laiks
5.	change_type	VARCHAR	50	Izmaiņu veids (price_down, initial)
6.	product_id (FK)	INTEGER	-	Sasaiste ar tabulu “product”

Glabā vēsturiskos datus par cenu un krājumu izmaiņām laika rindas veidā, nodrošinot nepieciešamo informāciju analītiskajiem grafikiem.

5.4.tabula

Tabulas **"user"** struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id (PK)	INTEGER	-	Lietotāja unikāls identifikators
2.	email	VARCHAR	255	Lietotāja e-pasta adrese (unikāla)
3.	hashed_password	VARCHAR	255	Ar bcrypt algoritmu šifrēta parole
4.	is_admin	BOOLEAN	-	Administratora tiesību pazīme
5.	created_at	TIMESTAMP	-	Reģistrācijas datums un laiks

Tabula **"user"** ir savienota ar tabulu **"userpreferences"** ar saiti **viens-pret-viens (1..1)**, nodrošinot, ka katram lietotājam ir tikai viena iestatījumu kopa.

5.5.tabula

Tabulas **"userpreferences"** struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id (PK)	INTEGER	-	Iestatījumu ieraksta identifikators
2.	user_id (FK)	INTEGER	-	Sasaiste ar tabulu “user”
3.	monitored_stores	JSON	-	Lietotāja izvēlēto veikalu saraksts
4.	telegram_chat_id	VARCHAR	64	Identifikators sasaistei ar Telegram botu
5.	threshold	NUMERIC	5,2	Paziņojumu slieksnis (procentos)

Satur lietotāju personalizētos iestatījumus, izvēlēto veikalu sarakstus un tehniskos datus sasaistei ar Telegram paziņojumu botu.

5.6.tabula

Tabulas **"favorite"** struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	user_id (FK)	INTEGER	-	Atsauce uz lietotāju
2.	product_id (FK)	INTEGER	-	Atsauce uz konkrētu preci
3.	alert_enabled	BOOLEAN	-	Vai paziņojumi precei ir aktīvi
4.	price_when_added	DECIMAL	12,2	Cena preces pievienošanas brīdī

Tabula **"favorite"** kalpo kā starptabula, lai realizētu **daudzi-pret-daudziem (M..N)** relāciju starp lietotājiem un precēm.

5.7.tabula

Tabulas **"scrapingrule"** struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id (PK)	INTEGER	-	Noteikuma identifikators
2.	domain	VARCHAR	255	Veikala domēna nosaukums
3.	container_css	VARCHAR	255	CSS selektors preces konteineram
4.	price_css	VARCHAR	255	CSS selektors cenas izgūšanai
5.	health_score	FLOAT	-	Noteikuma precizitātes rādītājs
6.	last_success	TIMESTAMP	-	Pēdējās sekmīgās skenēšanas laiks

Ietver tehniskos noteikumus un CSS selektorus datu ieguvei no katra domēna, kā arī uzrauga noteikumu precizitātes rādītājus.

5.8.tabula

Tabulas **"sharedpattern"** struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id (PK)	INTEGER	-	Patterna unikāls identifikators
2.	pattern_type	VARCHAR	50	Elementa tips (piem., “price”)
3.	css_selector	VARCHAR	255	CSS selektora virkne
4.	success_count	INTEGER	-	Sekmīgo pielietošanas reižu skaits

Ietver tehniskos noteikumus un CSS selektorus datu ieguvei no katra domēna, kā arī uzrauga noteikumu precizitātes rādītājus

6. LIETOTĀJA CEĻVEDIS

Sistēmas lietotāja ceļvedis ir izstrādāts, lai nodrošinātu ātru un efektīvu MarketPulse platformas uzstādīšanu un lietošanu. Rokasgrāmata ietver tehniskās prasības, instalācijas instrukcijas un detalizētu lietotāja saskarnes aprakstu.

6.1. Sistēmas prasības aparatūrai un programmatūrai

Lai nodrošinātu MarketPulse sistēmas stabilu darbību, lietotāja ierīcei jāatbilst šādām minimālajām prasībām:

Aparatūra:

- **Procesors:** 64-bitu arhitektūra, vismaz 2 kodoli;
- **Operatīvā atmiņa:** vismaz 4 GB RAM (ieteicams 8 GB liela apjoma datu apstrādei);
- **Brīvā diska vieta:** vismaz 5 GB (Docker attēliem un datubāzes uzkrājumiem).

Programmatūra un pārlūkprogrammas:

- **Operētājsistēma:**
    - Windows 10+;
    - macOS 10.15+;
    - vai moderna Linux versija (Ubuntu 20.04+).
- **Tīmekļa pārlūks:**
    - Google Chrome (v89+);
    - Mozilla Firefox (v86+);
    - Microsoft Edge (v89+);
    - Safari (v14+).
- **Obligāts nosacījums:** Pārlūkprogrammā jābūt iespējotam **JavaScript** atbalstam.

6.2. Sistēmas instalācija un palaišana

Sistēmas uzstādīšana tiek veikta, izmantojot konteinerizācijas tehnoloģiju, kas garantē identisku darbības vidi uz jebkuras platformas.

Sagatavošanās darbi: Pārliecinieties, ka ir instalēts:

- **Git**, **Docker Desktop**;
- **Python 3.12;**
- **Node.js 20+**.

1. **Projekta klonēšana no GitHub:**

1.1. Izmantojot terminmāli, lejupielādējiet repozitoriju (skat. 14. att. Git Clone);

```
git clone https://github.com/pikcrvt-students/26_d43-ErvinsUbinins-Marketinga-automizacija-komerckompanijai.git
```

14.att. Git Clone

1.2. Ieejiet projekta mapē (skat. 15.att. Projekta mape).

```
cd Market_pulse
```

15.att. Projekta mape

2. **Konfigurācijas faila iestatīšana:**

2.1. Izveidojiet .env failu [19], kopējot esošo šablonu (skat. 16. att. Configuration);

```
copy .env.example .env
```

16.att. Configuration

(Linux/macOS) (skat. 17. att. Configuration for Linux/macOS);

```
cp .env.example .env
```

17.att. Configuration for Linux/macOS

2.2. Atveriet .env failu un aizpildiet nepieciešamos laukus (skat. 18. att. Atslēgas).

```
2   GOOGLE_API_KEY=
3
4   JWT_SECRET=
```

18.att. Atslēgas

- GOOGLE_API_KEY - Google Gemini API atslēga (iegūstama: https://aistudio.google.com)
- JWT_SECRET - drošības atslēga (ģenerēt ar komandu) (skat. 19. att. Atslēgas ģenerēšana):

```
python -c "import secrets; print(secrets.token_hex(32))"
```

19.att. Atslēgas ģenerēšana

- ADMIN_EMAIL - e-pasta adrese, kas automātiski saņems administratora tiesības pēc reģistrācijas (skat. 20. att. Admin nosaukums):

```
ADMIN_EMAIL=ervinubinin1@gmail.com
```

20.att. Admin nosaukums

3. **Docker konteineru palaišana:**

3.1. Pārliecinieties, ka Docker Desktop ir palaists, pēc tam izpildiet komandu (skat. 21. att.):

```
docker-compose up -d
```

21.att. Docker Desktop

3.3. Pārbaudiet konteineru stāvokli (skat. 22. att. Docker pārbaude):

```
docker-compose ps
```

22.att. Docker pārbaude

Visiem konteineriem jāparādās statuss **"Up"**.

4. **Playwright pārlūka instalēšana** *(tikai pirmajā reizē)*:

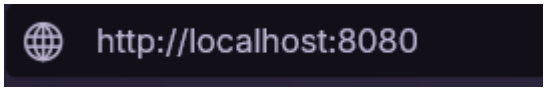
4.1. Izpildiet komandu konteinera iekšienē (skat. 23. att. Playwright pārlūka instalēšana).

```
docker-compose exec app playwright install chromium
```

23.att. Playwright pārlūka instalēšana

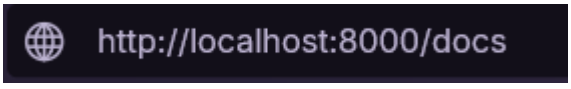
5. **Piekļuve sistēmai:**

5.1. Atveriet pārlūkprogrammu un ievadiet adresi (skat. 24. att. Adrese);



24.att. Adrese

5.2. Aizmugursistēmas API dokumentācija pieejama (skat. 25. att. Dokumentācijas adrese).



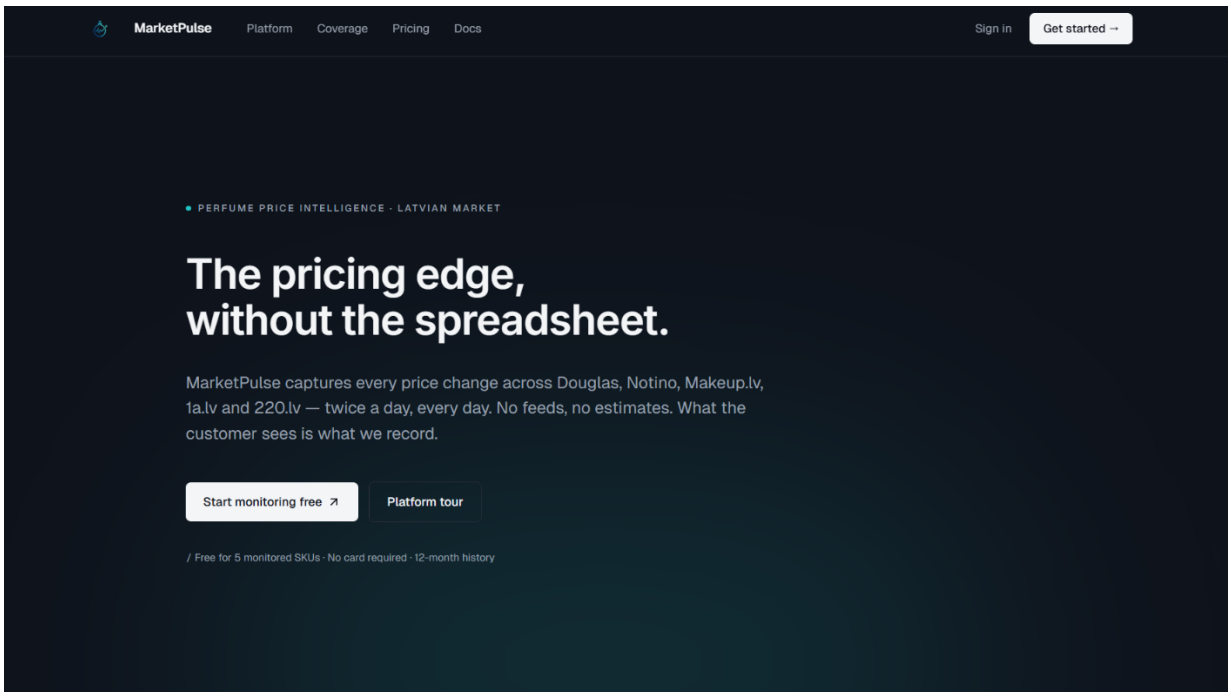
25.att. Dokumentācijas adrese

6.3. Programmas apraksts

Sistēmas lietotāja saskarne, ar kuru var iepazīties vietnē **https://marketpulse1.me**, ir veidota, ievērojot responsīvā dizaina principus un izmantojot tumšo B2B krāsu paleti, lai akcentētu analītiskos datus un grafikus.

Mājaslapa.

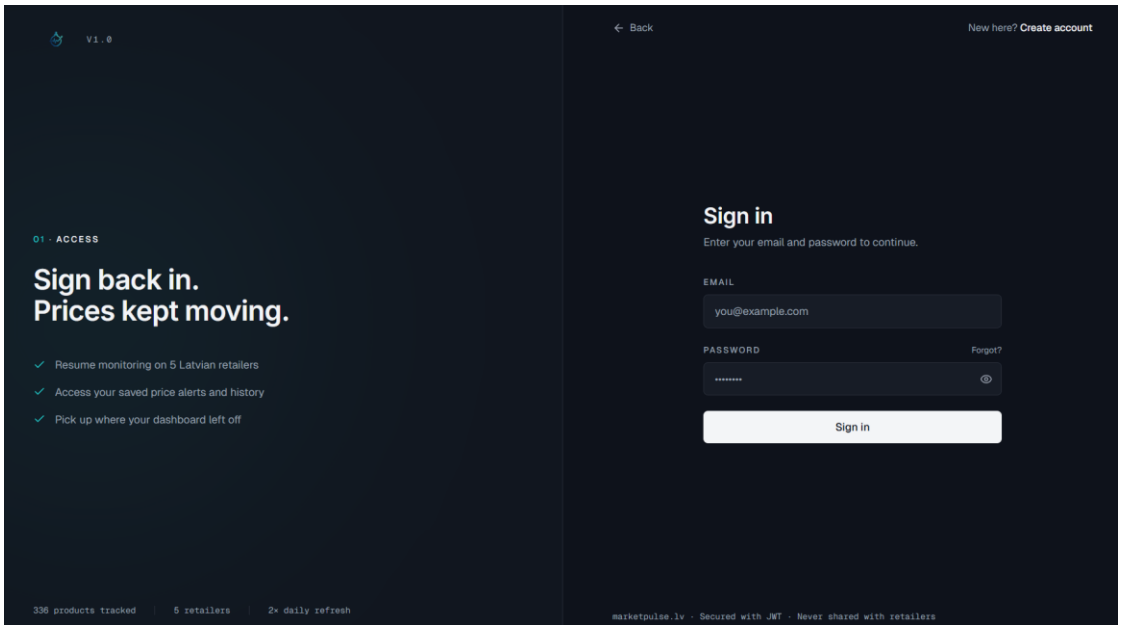
Ieejot vietnē, jūs sagaidīs MarketPulse mājaslapa. Šeit jūs varēsiet izpētīt vietni, uzzināt par tās funkcijām un uzzināt, kas tā ir. Lai turpinātu reģistrāciju, noklikšķiniet uz pogas “Sākt”. Lai pieteiktos, atlasiet “Pierakstīties” (skat. 26. att. Mājaslapa).



26.att. Mājaslapa

Pierakstīšana.

Pēc noklikšķināšanas uz pogas “Pierakstīties” lietotājam tiek parādīts logs, kurā tiek lūgts ievadīt savu e-pasta adresi un paroli (skat. 27. att. Pierakstīšana).

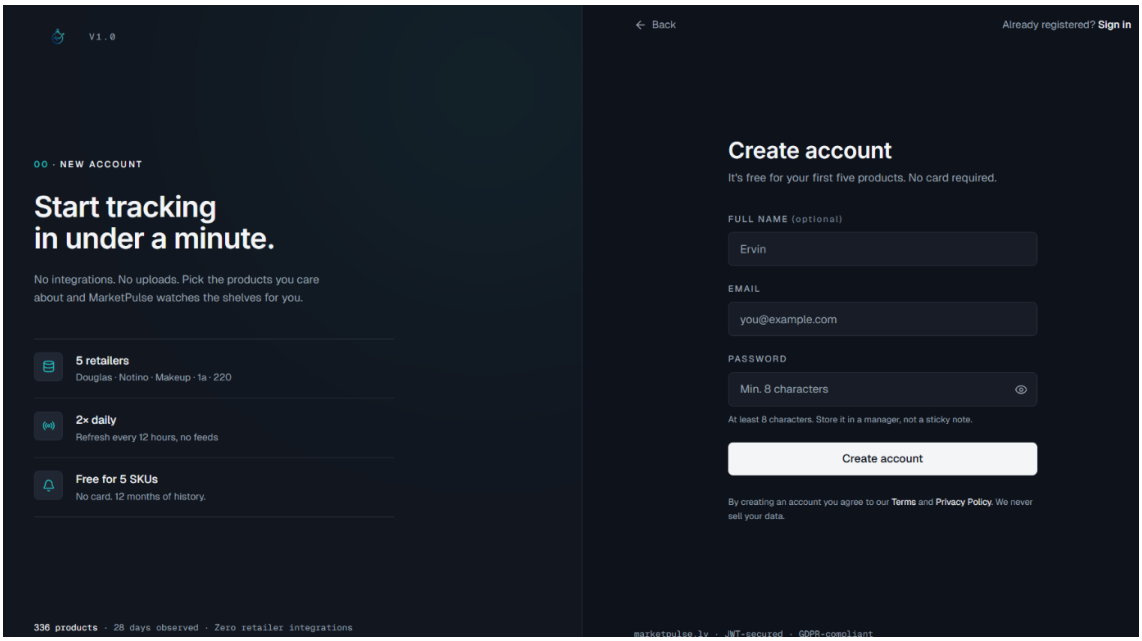


27.att. Pierakstīšana

Pēc informācijas ievadīšanas noklikšķiniet uz pogas “Pierakstīties”. Ja visa informācija ir ievadīta pareizi, jūs tiksiet novirzīts uz vietnes galveno lapu.

Registrācija.

Ja esat jauns lietotājs, jums būs jāaizpilda sadaļa “Reģistrācija”. Lai reģistrētos, aizpildiet šādus laukus: “vards”, e-mail, “parole” (skat. 28. att. Registrācija).

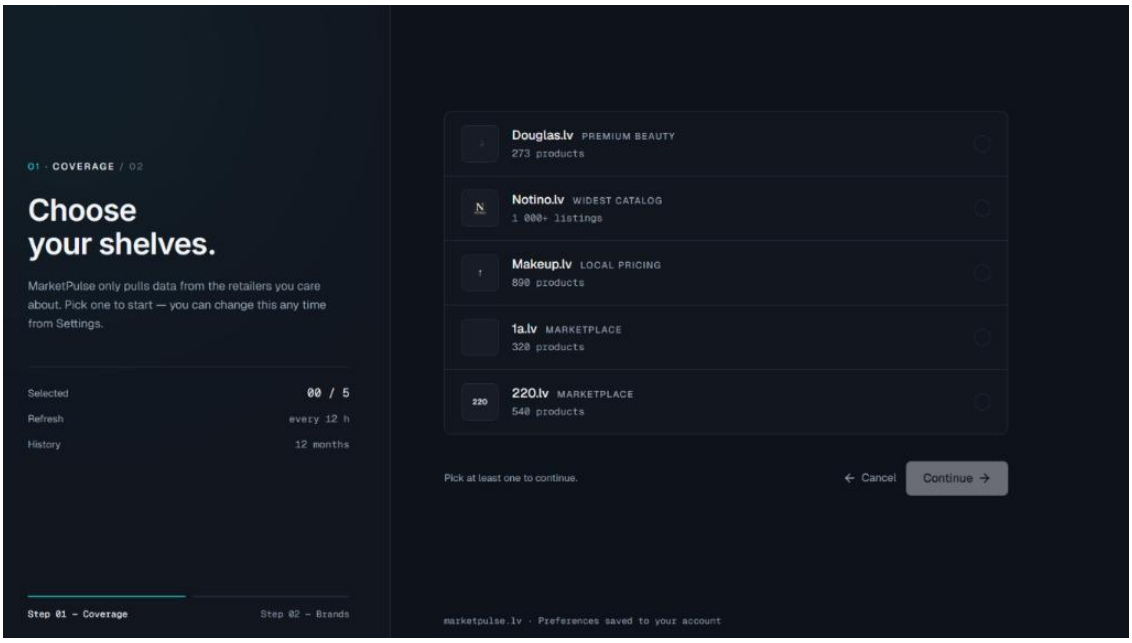


28.att. Registrācija

Parolei jābūt vismaz 8 rakstzīmes garai, pretējā gadījumā jūs nevarēsiet turpināt.

Pārraugāmo vietņu izvēle.

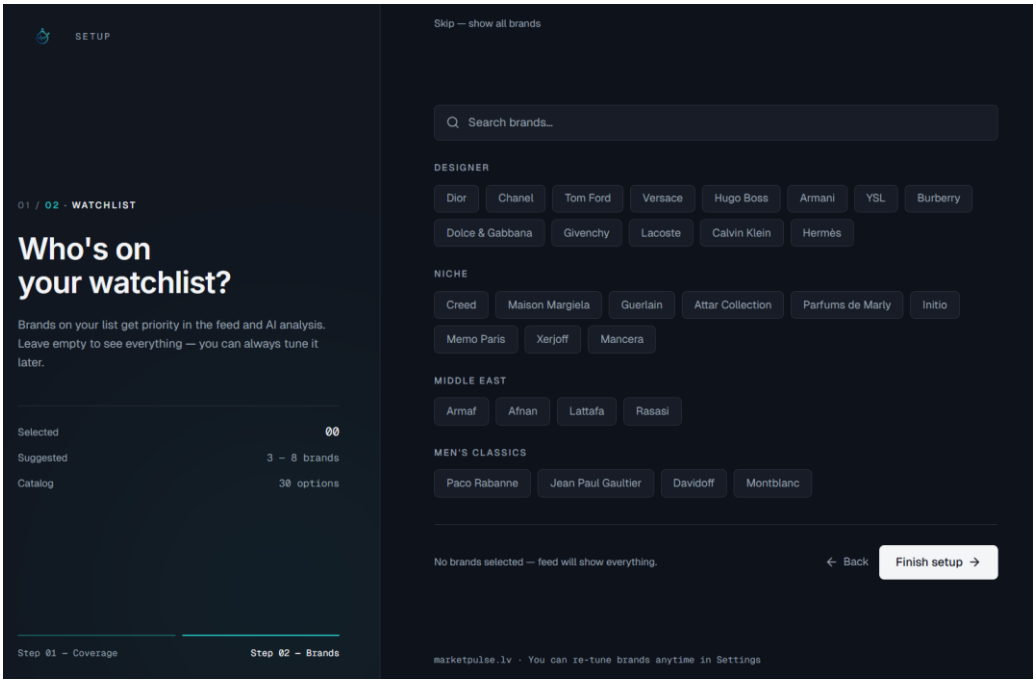
Šeit lietotājam tiek lūgts izvēlēties tīmekļa vietnes, kurām viņš vēlas sekot, vai tās, kas viņu visvairāk interesē (skat. 29. att. Vietņu izvēle). Lietotājs var izvēlēties no tādām vietnēm kā: Douglas.lv, Notino.lv, Makeup.lv, 1a.lv, 220.lv (šo sarakstu vēlāk var mainīt iestatījumos).



29.att. Vietņu izvēle

Zīmolu izvēle izsekošanai.

Tagad seko reģistrācijas pēdējais solis: jāizvēlas, kurus zīmolus lietotājs sekos. Tas ir nepieciešams, lai pašreizējā produktu plūsma būtu pielāgota lietotāja izvēlētajām interesēm (skat. 30. att.).



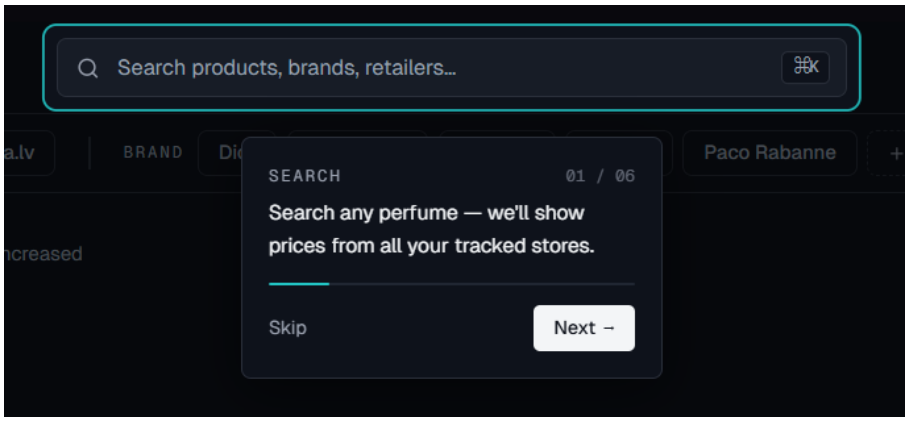
30.att. Zīmolu izvēle

Tāpat, ja lietotājs nevēlas izvēlēties nevienu zīmolu un vienkārši turpināt, viņš var noklikšķināt uz pogas “Izlaist - rādīt visus zīmolus”, pēc tam viņš nekavējoties turpinās.

Pamācība.

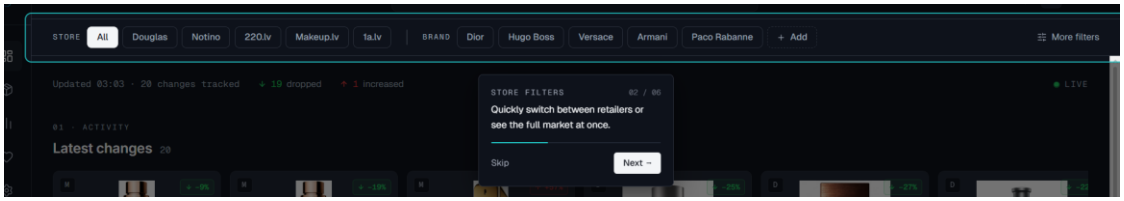
Tūlīt pēc reģistrācijas lietotāju sagaida pamācība, kurā parādīta vissvarīgākā informācija un tīmekļa vietnes, kas nepieciešamas, lai sāktu lietot Fenqciogal. Pamācība sastāv no sešiem soļiem, un beigās lietotājam ir jānoklikšķina uz pogas, lai apstiprinātu, ka visi soļi ir veiksmīgi pabeigti (skat. 31., 32., 33., 34., 4.pielikums., 35., 36. att.).

Šeit lietotājs var atrast sev nepieciešamās smaržas.



31.att. Pamācība Nr. 1.

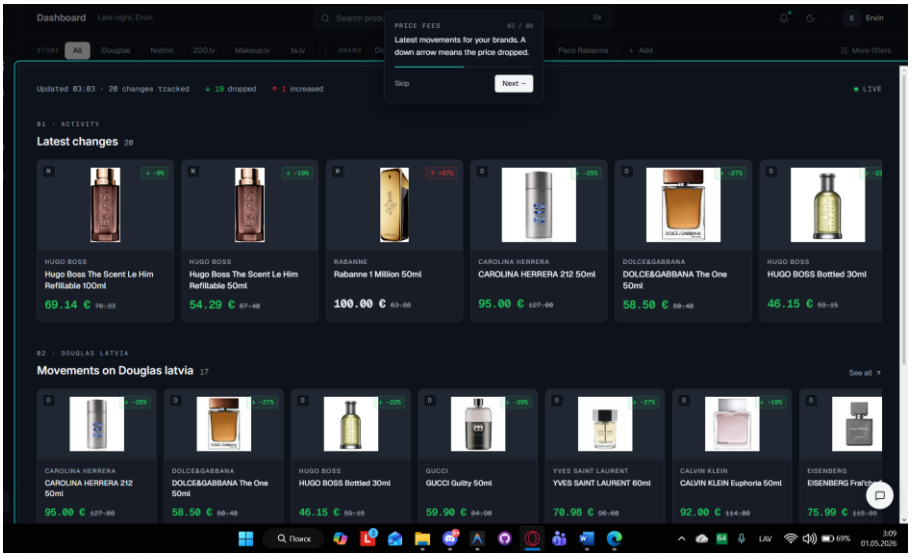
Tiešsaistes veikala un smaržu zīmola izvēle. Katra izmaiņa ietekmēs pašreizējo produktu plūsmu.



32.att. Pamācība Nr. 2.

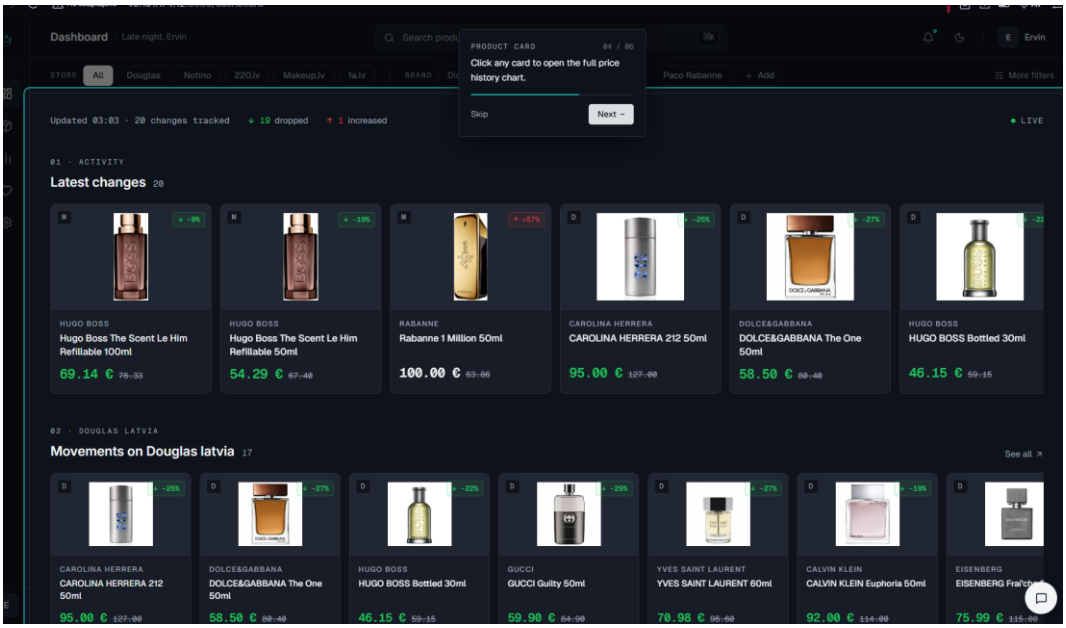
Pašreizējo produktu galvenais katalogs. Šeit tiek parādīti jaunākie produkti, kā arī visas izmaiņas.

Pēc noklusējuma tiek parādītas visas piecas vietnes (Notino.lv, Makeup.lv, Douglas.lv, 1a.lv, 220.lv).



33.att. Pamācība Nr. 3.

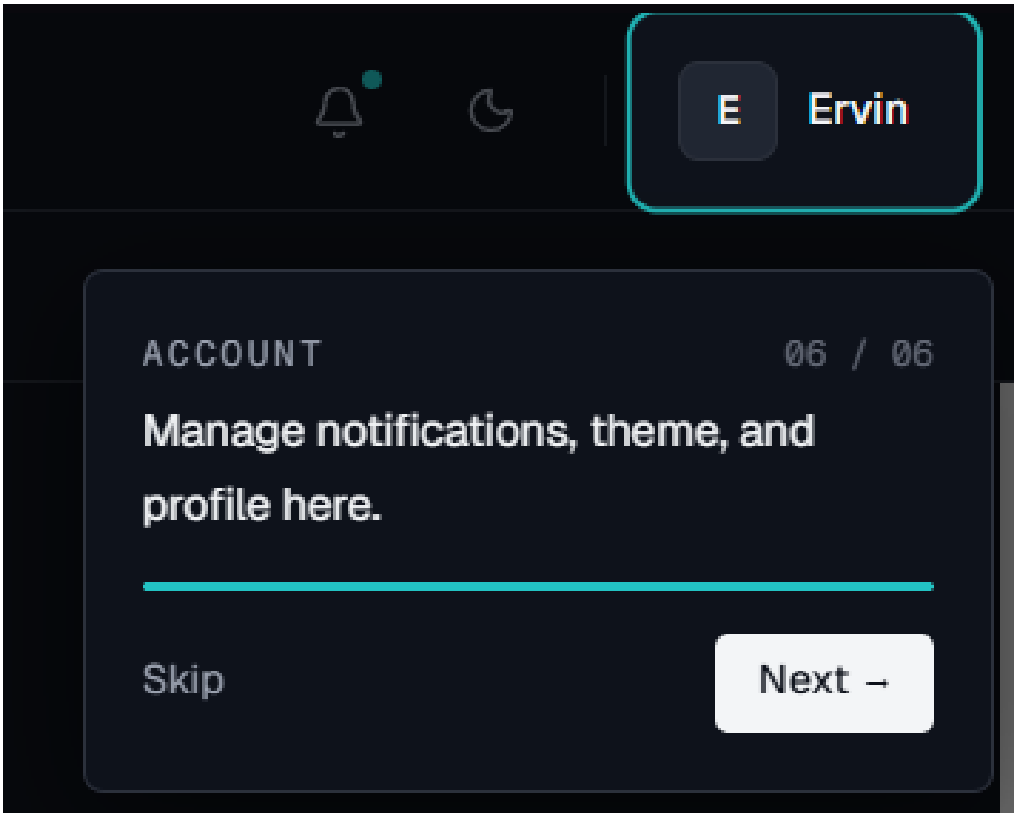
Noklikšķinot uz jebkuras kartītes sarakstā, lietotājs tiek novirzīts uz pašu karti, kur viņš var redzēt smaržu statistiku un salīdzināt to cenu ar konkurentiem.



34.att. Pamācība Nr. 4.

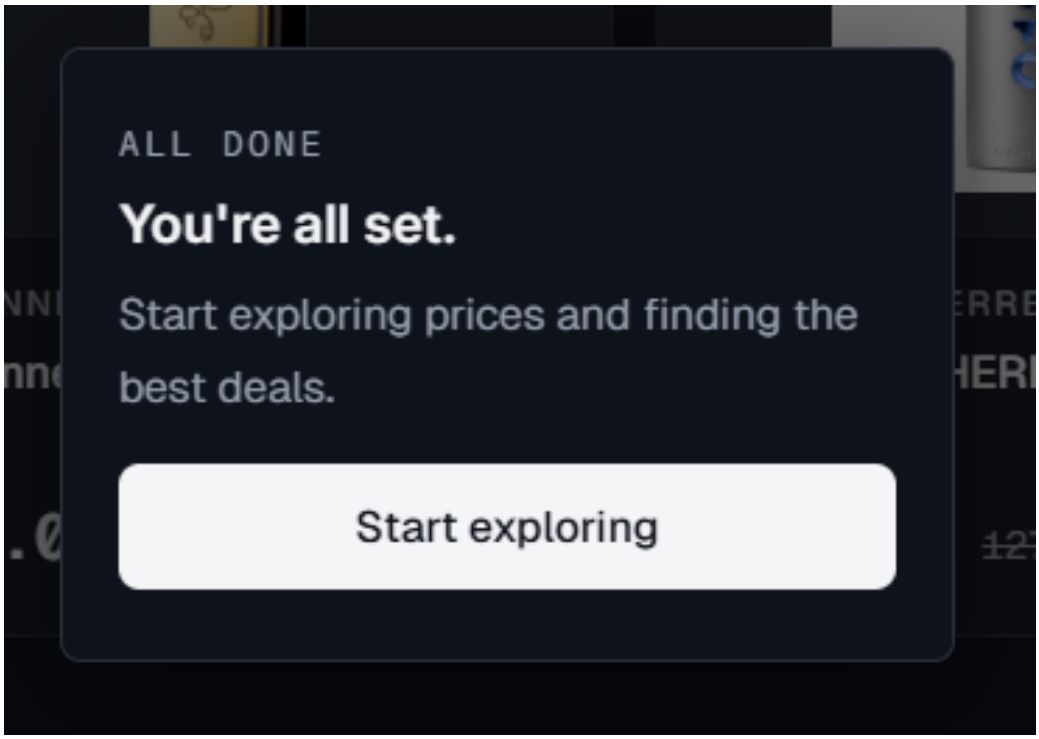
Mākslīgā intelekta asistents, ar kuru var uzdot jebkuru jautājumu par smaržām. Asistents ir tieši savienots ar datubāzi, tāpēc tam vienmēr ir jaunākie un precīzākie dati (skat. 4.pielikums. Pamācība Nr. 5).

Noklikšķinot uz profila ikonas, varat piekļūt profila iestatījumiem.



35.att. Pamācība Nr. 6.

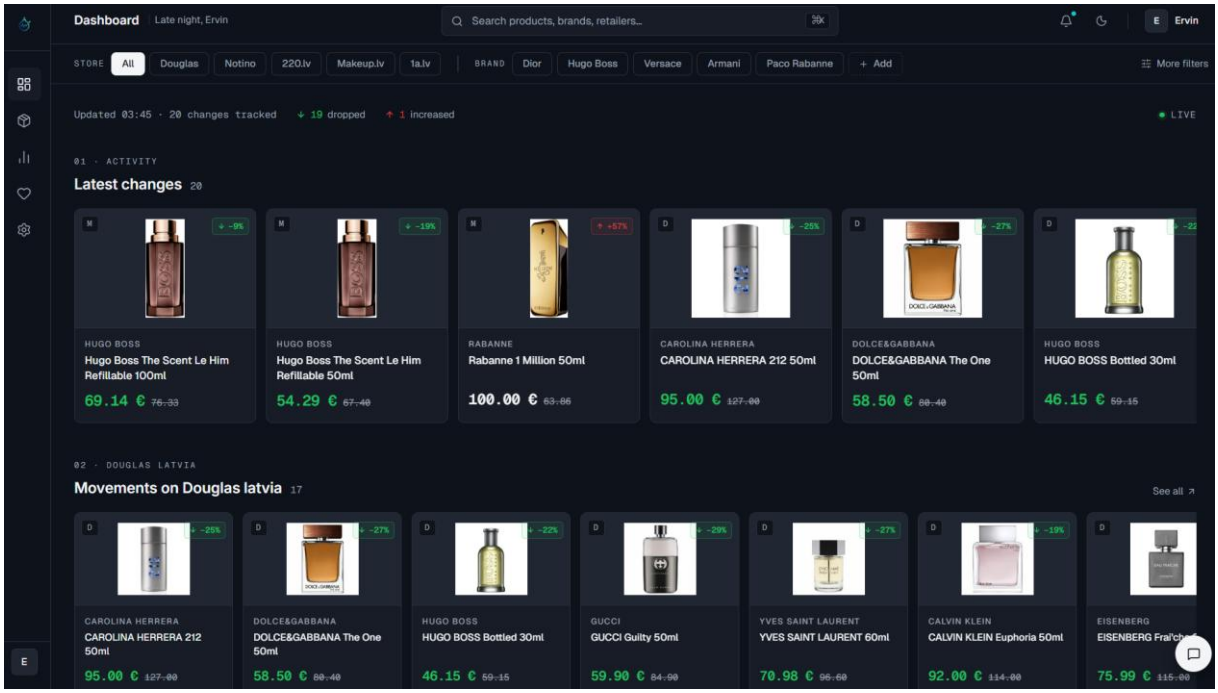
Pamācība ir beigusies, varat pāriet uz galveno ekrānu.



36.att. Pamācība Nr. 7.

Galvena tīmēkļa lapa

Šeit ir viss nepieciešamais: zīmola un tiešsaistes veikala iestatījumi, atjauninātas produktu kartītes, navigācijas josla, meklēšanas josla un piekļuve mākslīgā intelekta asistentam (skat. 37. att.).



37. att. Galvena tīmēkļa lapa

Produkta karte iekšpusē.

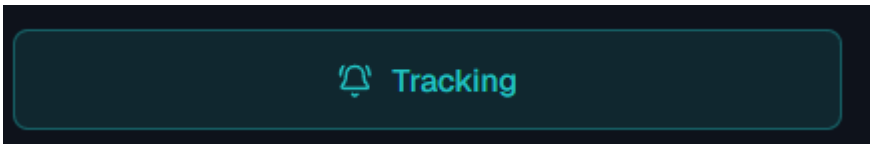
Kartes iekšpusē var redzēt smaržu tuvplāna fotoattēlu, kā arī cenu tabulu izvēlētajam laika periodam. Zemāk ir salīdzināšanas josla, kurā varat atrast tā paša produkta cenu citos tiešsaistes veikalos. Pēc noklusējuma tiek izvēlēts produkts ar zemāko tirgus cenu (skat. 38. att. Produkta karte iekšpusē).



38.att. Produkta karte iekšpusē

Saglabāšanas un paziņojumu sistēma.

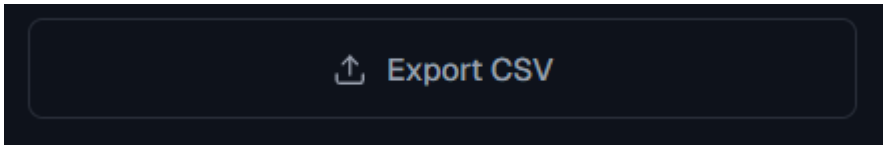
Noklikšķinot uz šīs pogas, lietotājs sāk izsekot preci, kas viņu interesē. Šī prece tagad ir pievienota lietotāja kontam “Izlase”. Šī funkcija ļauj lietotājiem nekavējoties saņemt paziņojumus, ja mainās šīs preces cena (skat. 39. att. Saglabāšanas un paziņojumu sistēma).



39.att. Saglabāšanas un paziņojumu sistēma

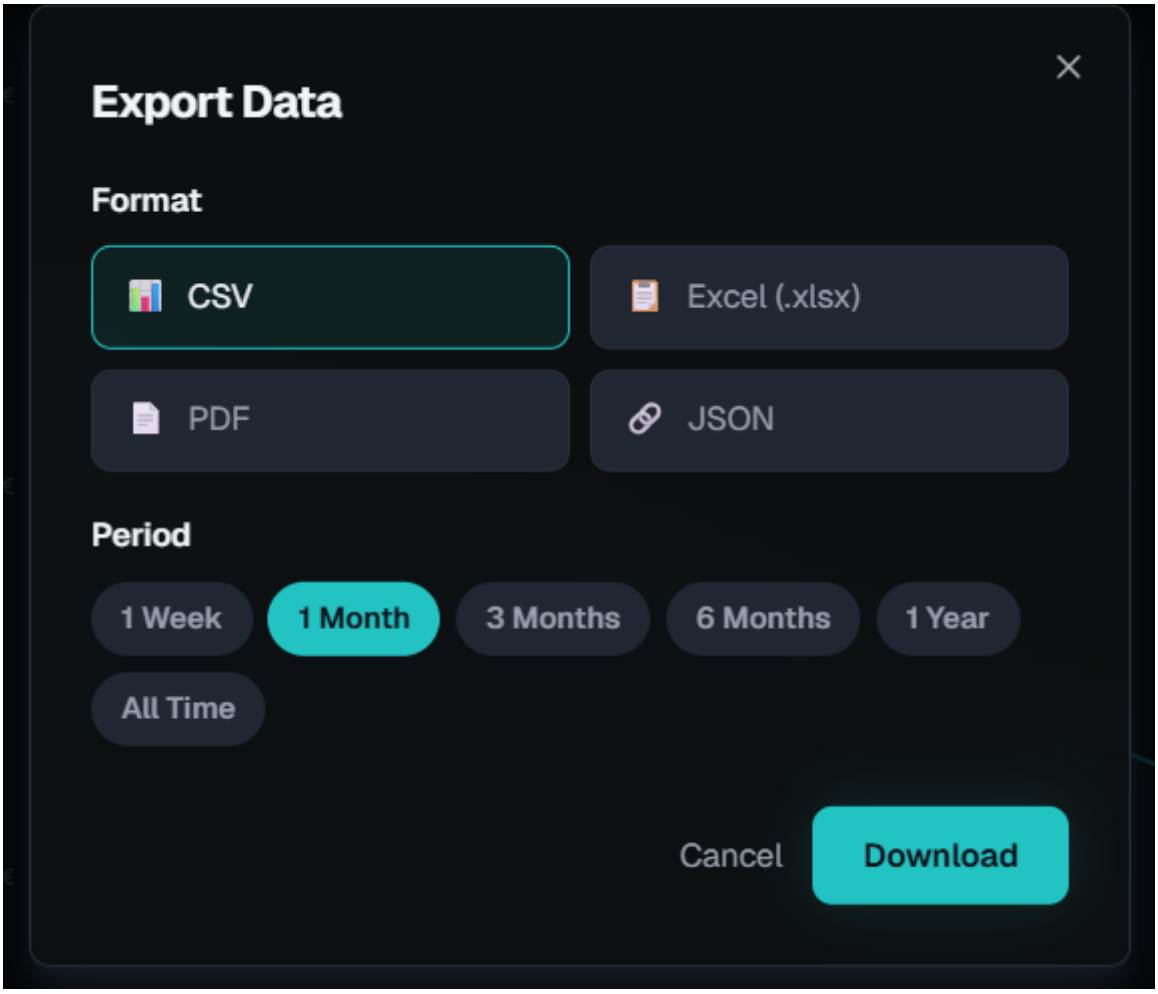
Datu eksportēšana.

Varat eksportēt produktu datus par izvēlētu laika periodu. Varat izvēlēties CSV, Excel, PDF vai JSON formātu (skat. 40. att. Eksportēšanas poga)



40.att. Eksportēšanas poga

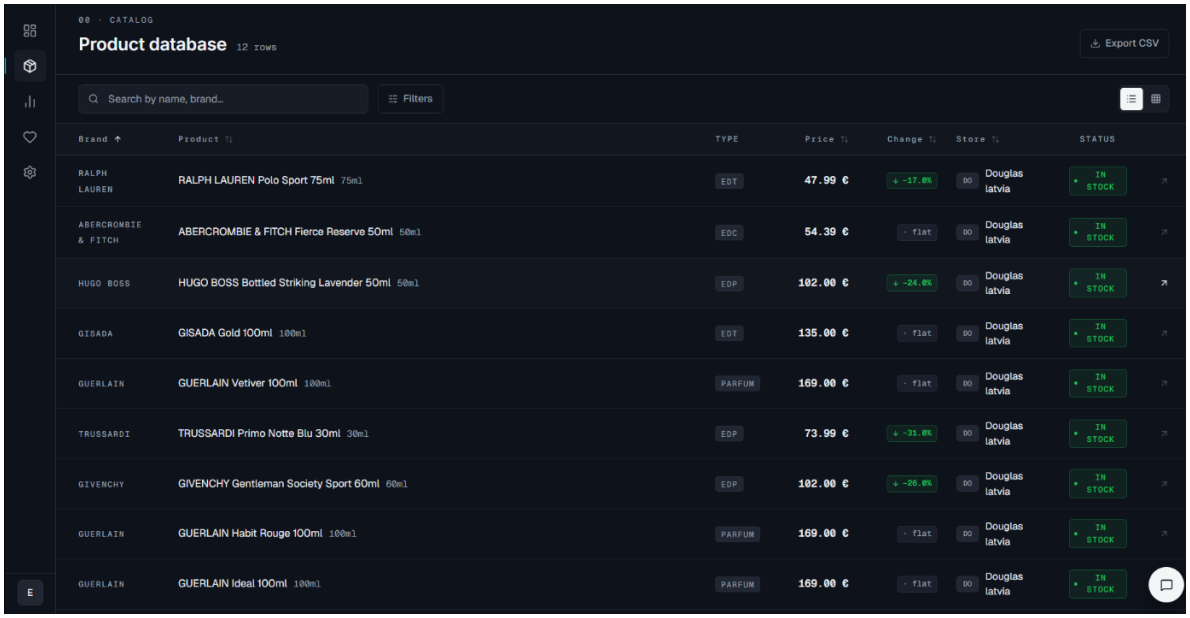
Iespējas, kur var eksportēt datus no datubāzes (skat. 41. att. Datu eksportēšana).



41.att. Datu eksportēšana

Produkta datubāze.

Visas datubāzes saraksts, kurā ir profesionālāks filtrēšanas rīks, kas ļauj izveidot visu veidu produktu kombinācijas eksportam (skat. 42. att. Produkta datubāze).



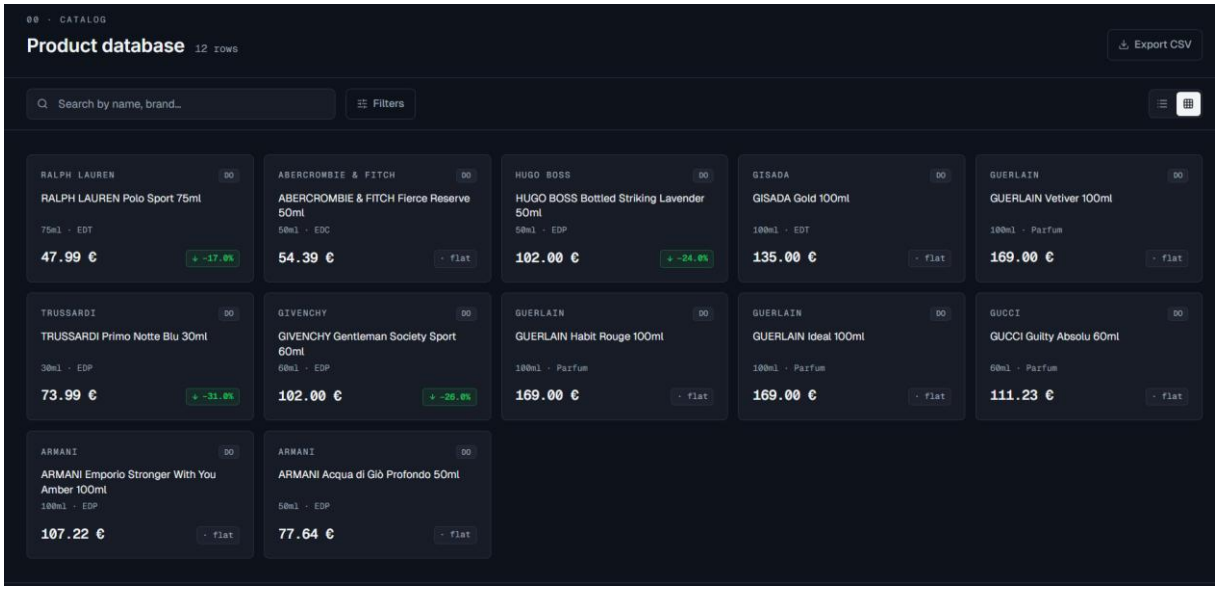
42.att. Produkta datubāze

Datubāzes datu filtrēšana.

Rīks, kas ļauj profesionāli filtrēt datubāzes datus (skat. 5.pielikums. Filtrēšana).

Produkta datubāzes otra versija.

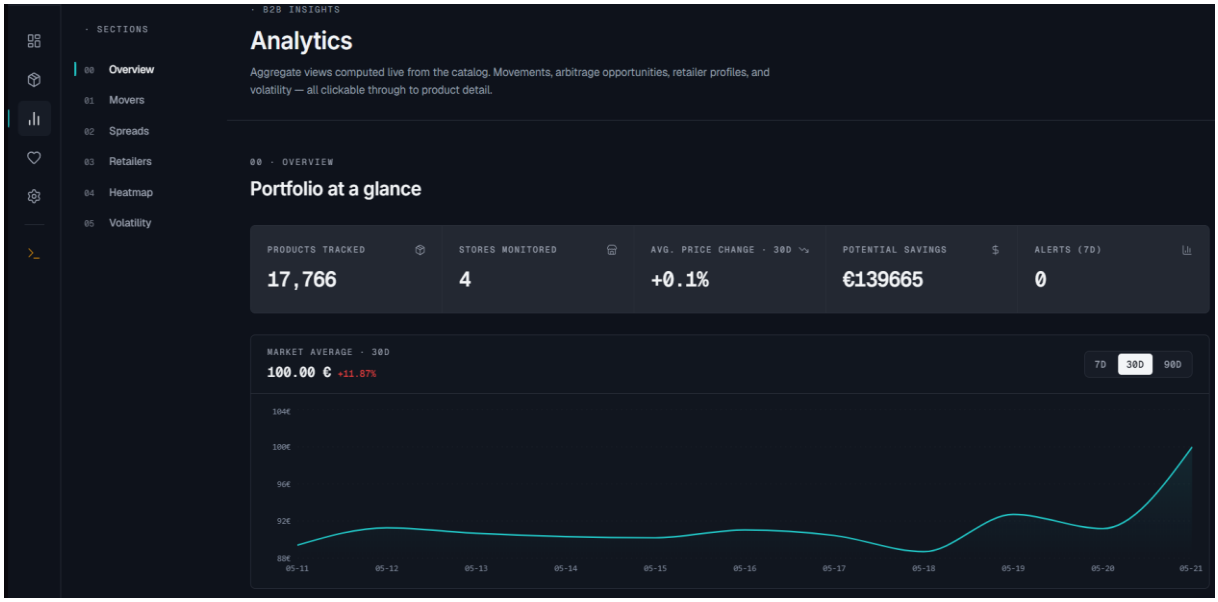
Labajā stūrī varat mainīt tabulu izskatu uz kārtīm (skat. 43. att. Produkta datubāzes 2. versija).



43.att. Produkta datubāzes 2. versija

Analītika.

Apkopotie skatījumi, kas aprēķināti tiešraidē no kataloga. Kustības, arbitrāžas iespējas, mazumtirgotāju profili un svārstīgums - tas viss noklikšķinot atveras, lai atvērtu produkta detalizēto informāciju (skat. 44. att. Analītika).



44.att. Analītika

Analītikas sastāvdaļas.

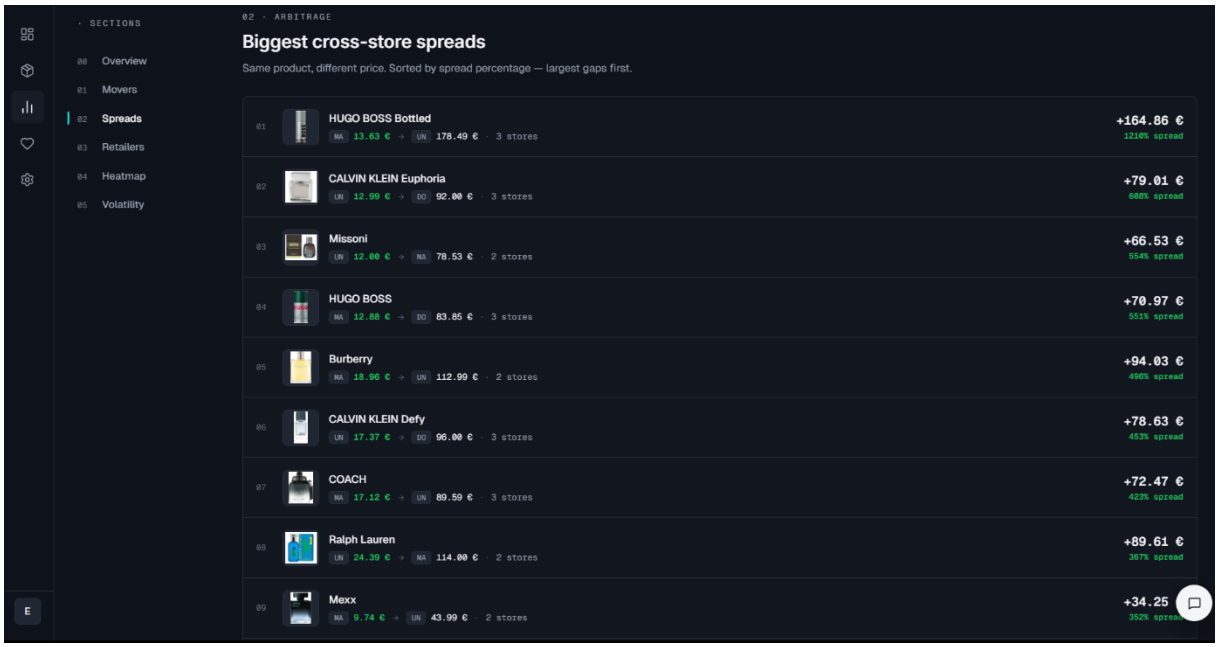
Šajā lielajā sadaļā tiks parādītas 5 analītikas daļas, un katra no tām ir neatņemama profesionāla analītiskā rīka sastāvdaļa (skat. 45., 46., 47., 48., 49. att.).

Lielākie ieguvēji un zaudētāji visā katalogā atlasītajā laika periodā (skat. 45. att. Analītikas sastāvdaļa Nr. 1).



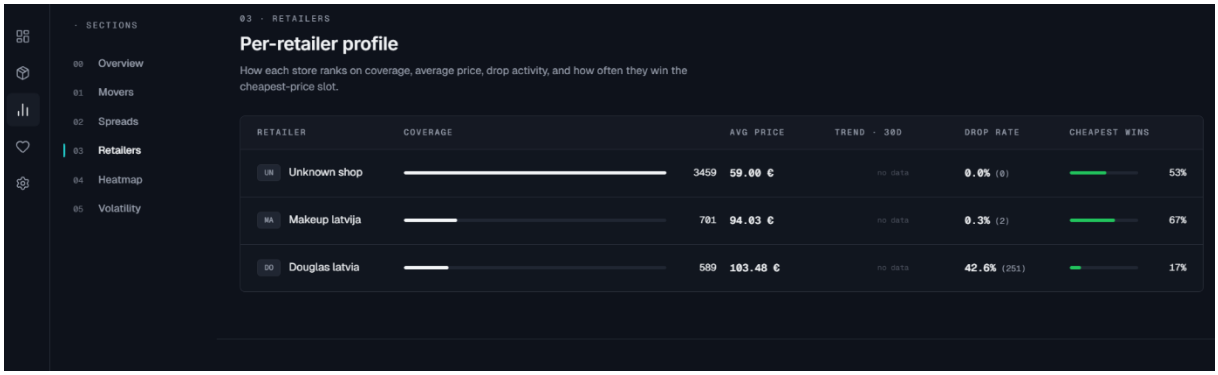
45.att. Analītikas sastāvdaļa Nr. 1

Lielākās starpveikalu cenas starpības. Tas pats produkts, cita cena. Kārtots pēc starpības procentuālās daļas - vispirms lielākās atšķirības (skat. 46. att. Analītikas sastāvdāļa Nr. 2).



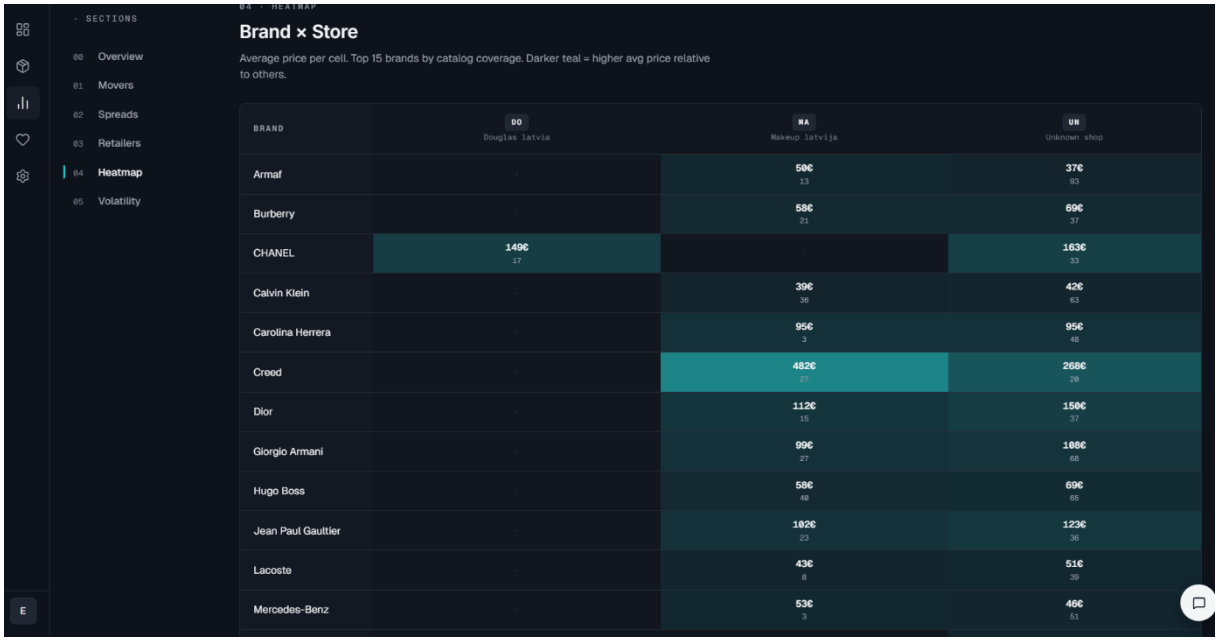
46.att. Analītikas sastāvdāļa Nr. 2

Mazumtirgotāja profils. Kā katrs veikals ierindojas pēc pārklājuma, vidējās cenas, pārdošanas aktivitātes un cik bieži tas iegūst lētākās cenas pozīciju (skat. 47. att. Analītikas sastāvdāļa Nr. 3).



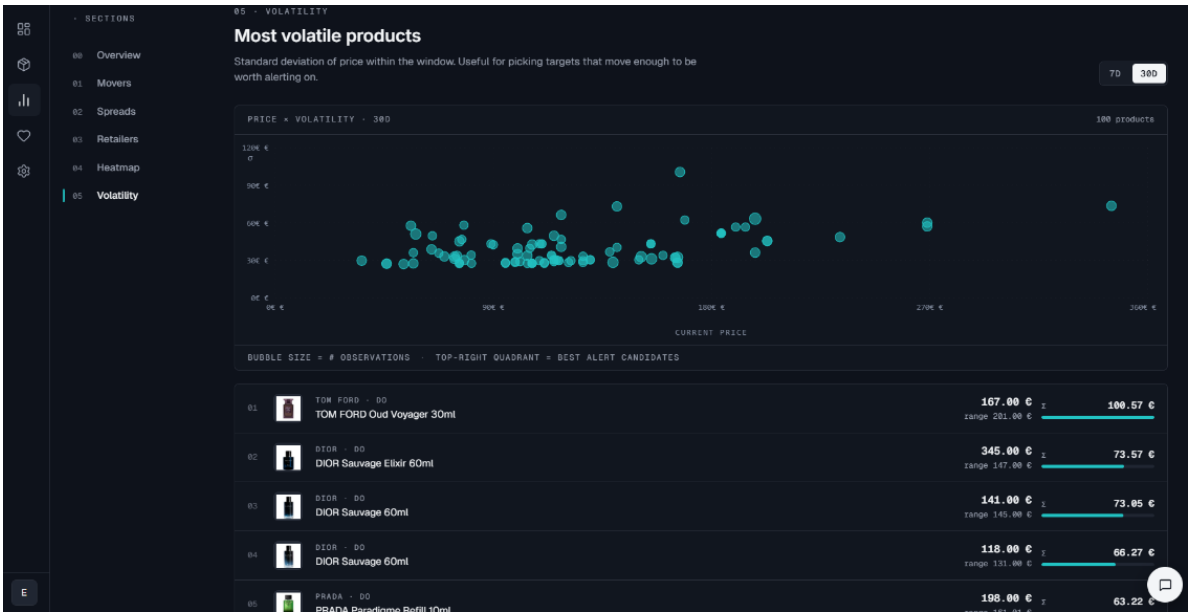
47.att. Analītikas sastāvdāļa Nr. 3

Zīmols x Veikals. Vidējā cena par šūnu. 15 populārākie zīmoli pēc kataloga pārklājuma. Tumšāka zilganzaļa = augstāka vidējā cena salīdzinājumā ar citiem (skat. 48. att. Analītikas sastāvdāļa Nr. 4).



48.att. Analītikas sastāvdāļa Nr. 4

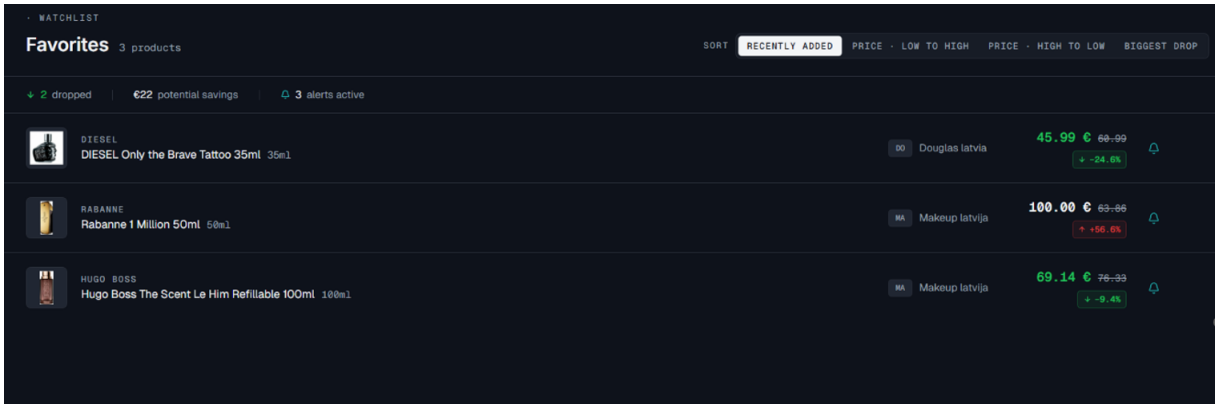
Visnestabilākie produkti. Cenas standartnovirze laika periodā. Noderīgi, lai izvēlētos mērķus, kas mainās pietiekami, lai būtu vērts par tiem brīdināt (skat. 49. att. Analītikas sastāvdāļa Nr. 5).



49.att. Analītikas sastāvdāļa Nr. 5

Izvēlētie produkti (Favorites).

Šeit ir uzskaitītas visas preces, kurām lietotājs ir noklikšķinājis uz pogas “Sekot cenai”. Tagad visas izmaiņas šajās precēs tiks reģistrētas, un lietotājs nekavējoties tiks informēts, ja notiks kādas izmaiņas (skat. 50. att. Izvēlētie produkti).



50.att. Izvēlētie produkti

Darbības ar izvēlētājiem produktiem.

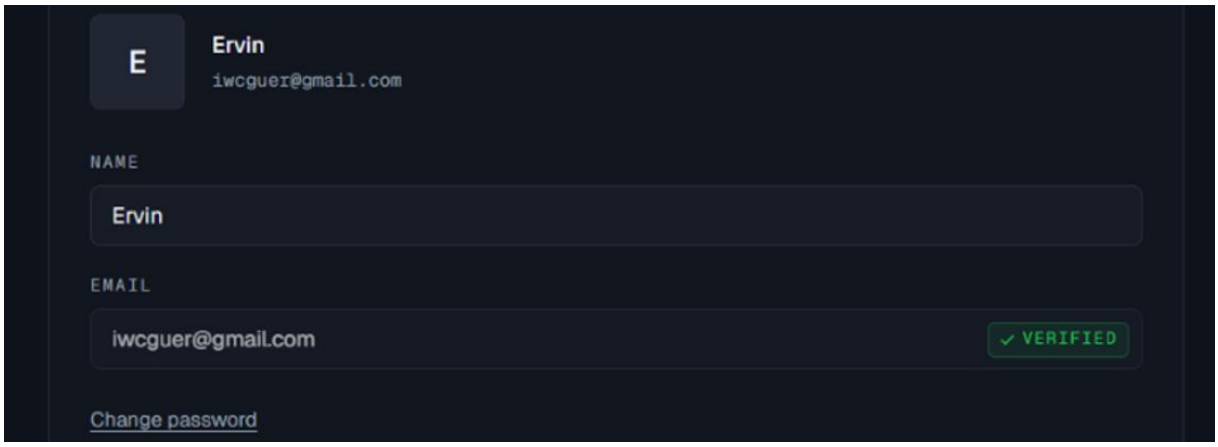
Tāpat, virzot kursoru virs produkta kartītes, lietotājs var noņemt paziņojumus no produkta vai pilnībā izdzēst produktu (skat. 51. att. Darbības ar izvēlētājiem produktiem).



51. att. Darbības ar izvēlētājiem produktiem

Lietotāja iestatījumi.

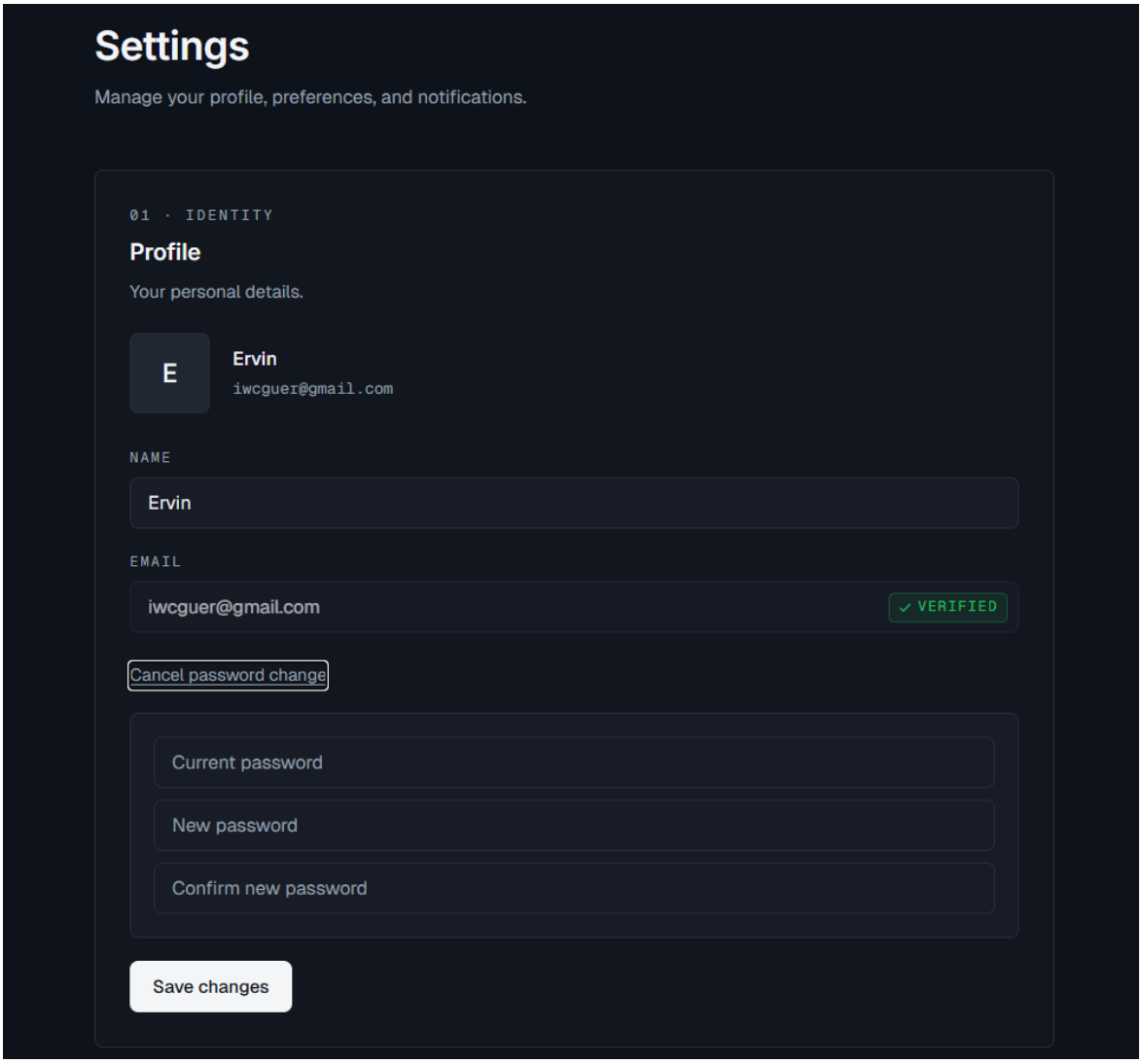
Šeit lietotājs var mainīt savu vārdu, uzvārdu un paroli (skat. 52. att. Lietotāja iestatījumi).



52.att. Lietotāja iestatījumi

Paroles maiņa.

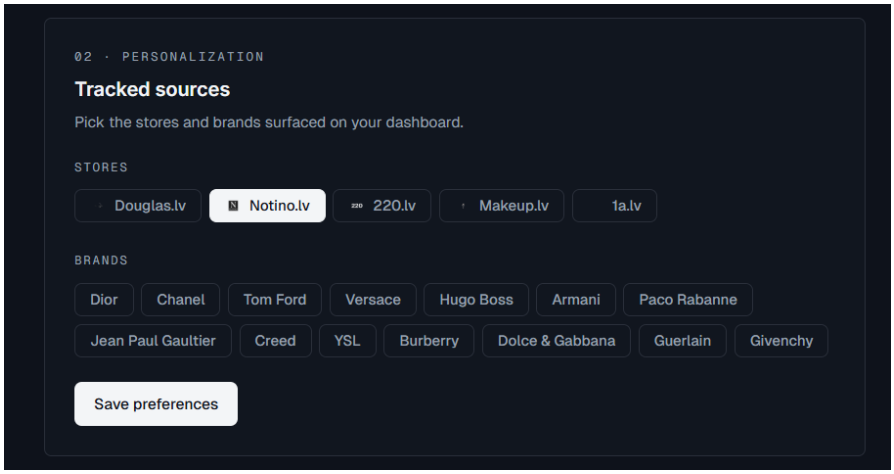
Lai mainītu paroli, noklikšķiniet uz pogas "Mainīt paroli", kas atrodas zem e-pasta adreses lauka. Pēc noklikšķināšanas parādīsies trīs jauni ievades lauki. Lai mainītu paroli, ievadiet pašreizējo paroli, jauno paroli un vēlreiz jauno paroli, lai apstiprinātu. Pēc visas nepieciešamās informācijas ievadīšanas noklikšķiniet uz pogas "Saglabāt izmaiņas", lai saglabātu izmaiņas. Jūsu parole tagad tiks veiksmīgi nomainīta (skat. 53. att. Paroles maiņa).



53.att. Paroles maiņa

Izsekojamo veikalu un produktu iestatīšana.

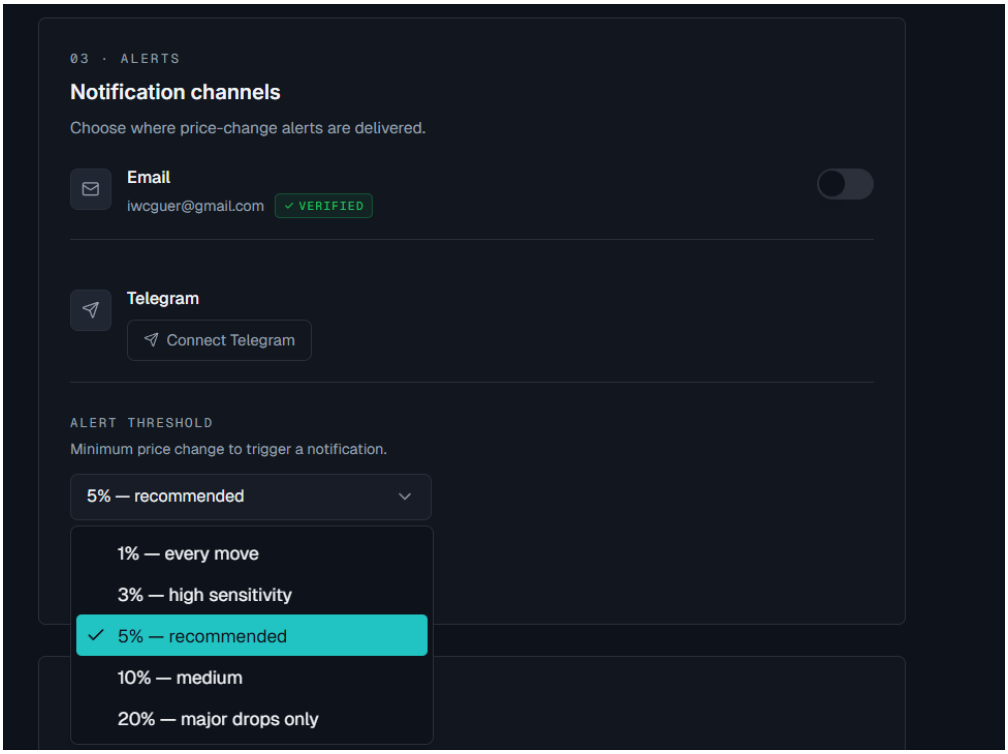
Šeit jūs varat mainīt informāciju, ko izvēlējāties reģistrācijas laikā. Šī ir vienīgā vieta, kur tiks saglabātas jūsu izvēles; vienkārši izvēloties tiešsaistes veikalus un zīmolus galvenajā lapā, tie netiks saglabāti pēc vietnes atstāšanas (skat. 54. att. Izsekoto avotu iestatījumi).



54.att. Izsekoto avotu iestatījumi

Paziņojumi.

Ļoti svarīga iestatījumu sadaļa ļauj konfigurēt e-pasta un Telegram paziņojumus. Lai saņemtu e-pasta paziņojumus, vienkārši iespējojiet pogu pa labi no sava e-pasta adreses. Lai izveidotu savienojumu ar Telegram, noklikšķiniet uz pogas "Savienot Telegram". Jūsu pārlūkprogramma liks jums atvērt Telegram. Jums ir jāpiekrīt, un Telegram robots tiks palaists un izveidos savienojumu ar jūsu profilu (skat. 6.pielikums. Telegram savienojums).

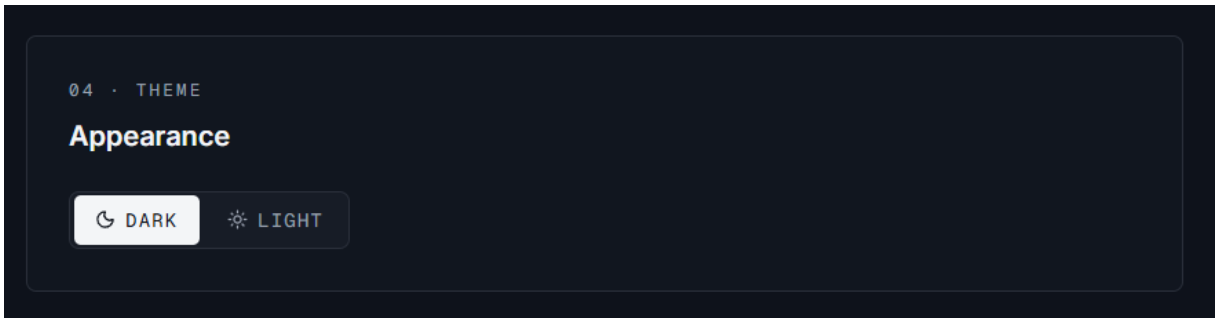


55.att. Paziņojumi

Paziņojuma čats ar Telegram botu (skat. 6.pielikums. Telegram savienojums).

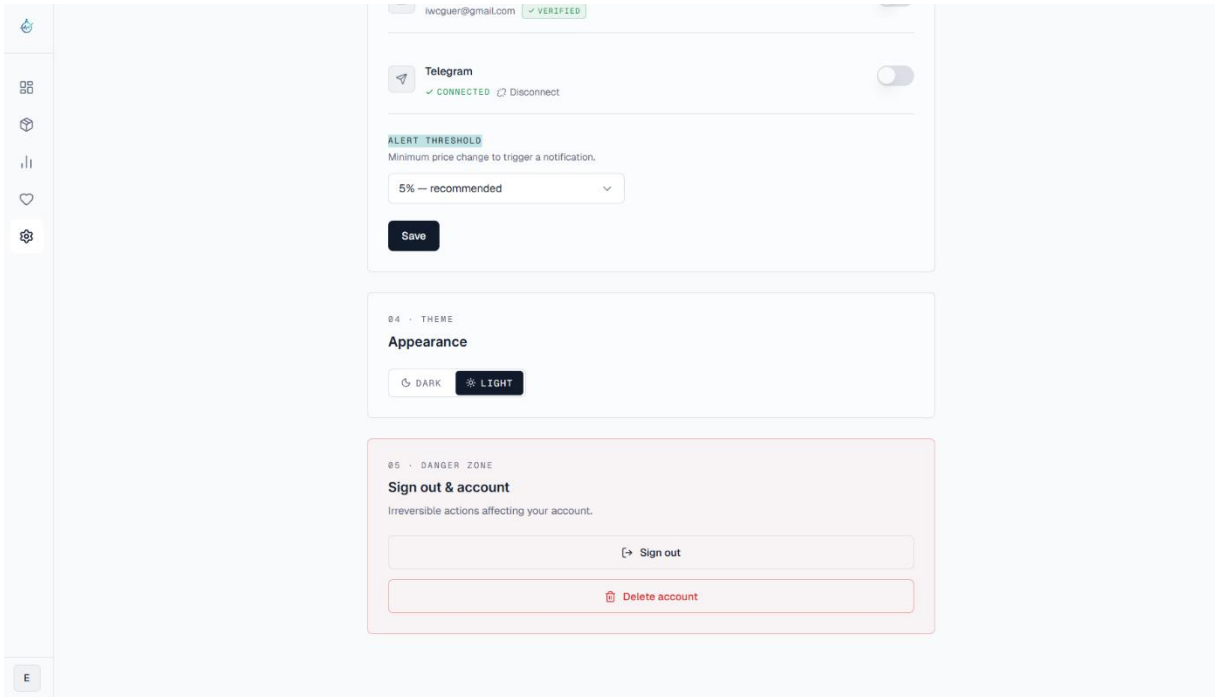
Tumšas/gaišas tēmas iestatīšana.

Iestatījumos varat mainīt noklusējuma tumšo tēmu uz baltu tēmu (skat. 56. att. Tumšas/gaišas tēmas iestatīšana).



56.att. Tumšas/gaišas tēmas iestatīšana

Tā izskatās gaišā tēma (skat. 57. att. Gaišā tēma).



57.att. Gaišā tēma

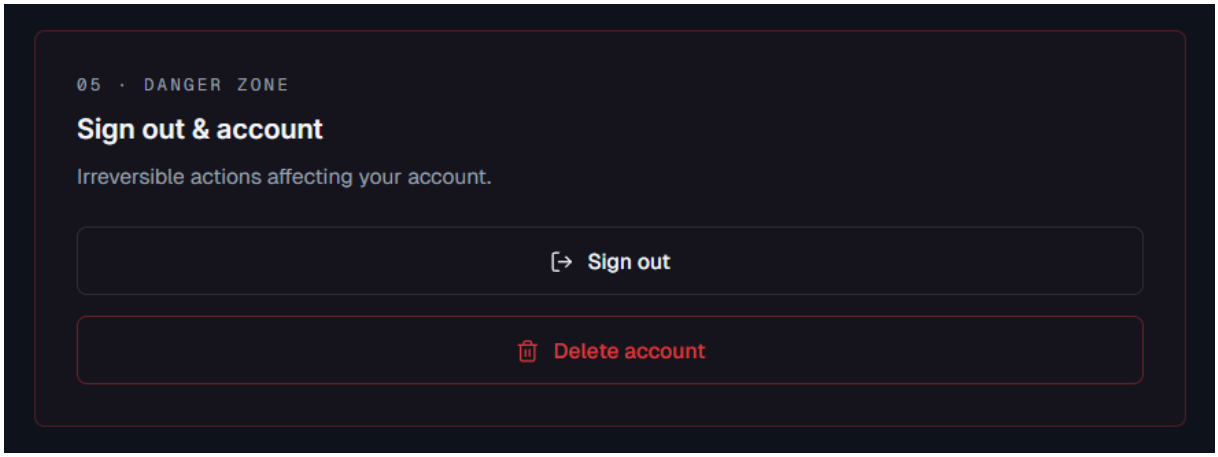
Varat arī mainīt tēmu galvenajā ekrānā blakus lietotāja pogai (skat. 58. att. Tēmas poga).



58.att. Tēmas poga

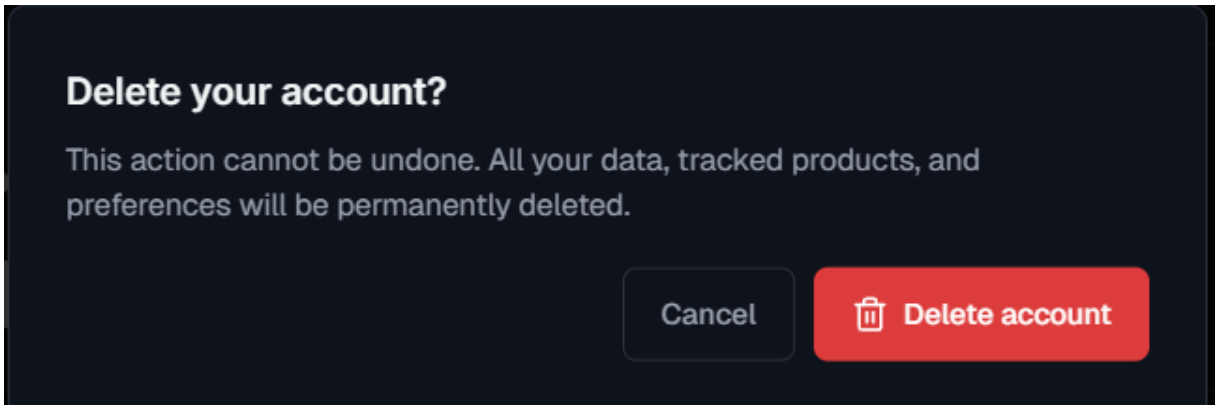
Izrakstīties un dzēst konta iestatījumus.

Iestatījumu pēdējā sadaļa. Šeit lietotājs var vai nu atteikties no sava konta, vai arī neatgriezeniski to izdzēst (skat. 59. att. Izrakstīties un dzēst konta pogas).



59.att. Izrakstīties un dzēst konta pogas

Noklikšķinot uz pogas "Dzēst kontu", tiks parādīts vēl viens brīdinājuma logs. Tas ir paredzēts, lai novērstu nejaušu noklikšķināšanu uz pogas "Dzēst kontu" (skat. 60. att. Brīdinājuma ziņojums).



60.att. Brīdinājuma ziņojums

Eksportēšana Excel failā.

Tā izskatīsies datu izvade Excel dokumentā (61. att. Excel fails).

Parfīms	Zīmols	Nosaukums	Tilpums	Cena (€)	Cenas izmaiņas (%)	Parfīmu interneta veikals
Acqua di Gio Profondo	Giorgio Armani	Acqua di Gio Profondo Eau de Parfum	75 ml	89,95	-12,4%	Notino.lv
Sauvage	Dior	Sauvage Eau de Parfum	100 ml	129,00	-8,1%	Douglas.lv
Bleu de Chanel	Chanel	Bleu de Chanel Parfum	100 ml	149,99	-5,3%	Makeup.lv
1 Million	Paco Rabanne	1 Million Eau de Toilette	100 ml	79,90	-10,2%	1a.lv
Le Male	Jean Paul Gaultier	Le Male Eau de Toilette	125 ml	74,95	-7,6%	220.lv
Invictus	Paco Rabanne	Invictus Eau de Toilette	100 ml	75,50	-9,8%	Notino.lv
Eros	Versace	Eros Eau de Toilette	100 ml	69,95	-11,3%	Douglas.lv
Dylan Blue	Versace	Dylan Blue Eau de Toilette	100 ml	64,90	-8,7%	Makeup.lv
Y	Yves Saint Laurent	Y Eau de Parfum	100 ml	109,00	-6,6%	1a.lv
La Nuit de L'Homme	Yves Saint Laurent	La Nuit de L'Homme Eau de Toilette	100 ml	94,90	-5,7%	220.lv
Stronger With You	Emporio Armani	Stronger With You Eau de Toilette	100 ml	89,00	-7,1%	Notino.lv
Code	Armani	Armani Code Eau de Toilette	75 ml	79,95	-9,2%	Douglas.lv
Allure Homme Sport	Chanel	Allure Homme Sport Eau de Toilette	100 ml	129,99	-6,0%	Makeup.lv
Terre d'Hermès	Hermès	Terre d'Hermès Eau de Toilette	100 ml	99,95	-4,4%	1a.lv
Light Blue	Dolce & Gabbana	Light Blue Eau de Toilette	100 ml	69,00	-10,5%	220.lv
The One	Dolce & Gabbana	The One Eau de Parfum	100 ml	84,90	-6,8%	Notino.lv
Black XS	Paco Rabanne	Black XS Eau de Toilette	100 ml	59,95	-12,0%	Douglas.lv
Boss Bottled	Hugo Boss	Boss Bottled Eau de Toilette	100 ml	74,95	-8,2%	Makeup.lv
Fahrenheit	Dior	Fahrenheit Eau de Toilette	100 ml	89,99	-7,7%	1a.lv
L'Homme	Yves Saint Laurent	L'Homme Eau de Toilette	100 ml	84,50	-6,1%	220.lv
Adventure	Davidoff	Cool Water Adventure Eau de Toilette	125 ml	54,95	-9,5%	Notino.lv
Cool Water	Davidoff	Cool Water Eau de Toilette	125 ml	49,90	-11,2%	Douglas.lv
Hugo	Hugo Boss	Hugo Man Eau de Toilette	125 ml	59,95	-7,0%	Makeup.lv
Pour Homme	Dolce & Gabbana	Dolce & Gabbana Pour Homme Eau de Toilette	125 ml	69,90	-6,3%	1a.lv
Club de Nuit Intense Man	Armaf	Club de Nuit Intense Man Eau de Toilette	105 ml	39,90	-13,2%	220.lv
Aventus	Creed	Aventus Eau de Parfum	100 ml	279,00	-4,9%	Notino.lv
Nautica Voyage	Nautica	Nautica Voyage Eau de Toilette	100 ml	29,95	-14,1%	Douglas.lv
CK One	Calvin Klein	CK One Eau de Toilette	200 ml	34,90	-9,3%	Makeup.lv
Eternity for Men	Calvin Klein	Eternity for Men Eau de Toilette	100 ml	49,90	-8,4%	1a.lv
Issey Miyake L'Eau d'Issey	Issey Miyake	L'Eau d'Issey Pour Homme Eau de Toilette	125 ml	59,00	-7,6%	220.lv
Missoni Wave	Missoni	Missoni Wave Eau de Toilette	100 ml	39,95	-12,8%	Notino.lv
Bentley for Men	Bentley	Bentley for Men Eau de Toilette	100 ml	44,90	-10,0%	Douglas.lv
Montblanc Legend	Montblanc	Legend Eau de Toilette	100 ml	54,95	-6,9%	Makeup.lv
Joop! Homme	Joop!	Joop! Homme Eau de Toilette	125 ml	42,50	-11,5%	1a.lv
Oscar for Men	Oscar de la Renta	Oscar for Men Eau de Toilette	100 ml	29,90	-13,6%	220.lv
Burberry Hero	Burberry	Burberry Hero Eau de Toilette	100 ml	74,90	-5,8%	Notino.lv
Givenchy Gentleman	Givenchy	Gentleman Eau de Parfum	100 ml	89,90	-6,2%	Douglas.lv
Chrome	Azzaro	Chrome Eau de Toilette	100 ml	49,95	-10,3%	Makeup.lv
Bvlgari Man	Bvlgari	Bvlgari Man Eau de Toilette	100 ml	79,95	-7,4%	1a.lv
Guess Seductive Homme	Guess	Guess Seductive Homme Eau de Toilette	100 ml	34,90	-12,2%	220.lv
Tommy	Tommy Hilfiger	Tommy Eau de Toilette	100 ml	34,95	-9,7%	Notino.lv
Lacoste Essential	Lacoste	Lacoste Essential Eau de Toilette	125 ml	49,90	-8,0%	Douglas.lv

61.att. Excel fails

Eksportēšana PDF failā

Attēlā redzams neapstrādātu cenu datu konvertēšanas process gatavā analītiskā ziņojumā (skat. 7.pielikums., 8.pielikums PDF fails). Vispirms sistēma apkopo un apvieno datus no dažādiem tiešsaistes veikaliem, salīdzina līdzīgus produktus un saglabā cenu vēsturi. Pēc tam tā analizē šos datus: aprēķina vidējos rādītājus, izseko cenu pieaugumus un kritumus, identificē galvenās izmaiņas un ģenerē kopsavilkuma rādītājus, kas tiek parādīti kodolīgā kopsavilkumā. Pēc tam dati tiek vizualizēti kā grafiki, lai ātri parādītu tendences, tirgus sadalījumu un produktus ar lielākajām izmaiņām. Pēc tam tiek salīdzinātas atlasīto produktu cenas, automātiski nosakot, kurš veikals piedāvā vislabāko vērtību. Visbeidzot, sistēma visu to apkopo strukturētā PDF dokumentā ar skaidru loģiku un formatējumu, ļaujot lietotājam ātri izprast pašreizējo tirgus situāciju un pieņemt lēmumus, pašam neanalizējot datus.

7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA

Programmatūras testēšana ir būtisks izstrādes cikla posms, kura mērķis ir pārbaudīt sistēmas atbilstību izvirzītajām funkcionālajām prasībām un nodrošināt kļūdu savlaicīgu identificēšanu. Testēšana tika veikta, izmantojot "melnās kastes" metodi, simulējot reālas lietotāju un administratoru darbības sistēmas tīmekļa saskarnē.

Zemāk ir apkopoti svarīgāko funkciju testēšanas rezultāti.

7.1. tabula

Lietotāja autorizācijas funkcionālā testa pārskats

Šī tabula apraksta drošas pieslēgšanās procesa pārbaudi sistēmai. Testēšana ietver scenārijus, kuros tiek ievadīti pareizi dati (veiksmīga JWT sesijas izveide), nepareiza parole (piekļuves liegšana ar kļūdas paziņojumu) un mēģinājums autorizēties ar tukšu e-pasta lauku, lai pārliecinātos par obligāto lauku validāciju.

Lietotāja loma :	Reģistrēts lietotājs / administrators	
Prasības identifikātors :	PS 2.2.1	
Prasības apraksts		
Jānodrošina, ka lietotājs var droši pieslēgties sistēmai, izmantojot e-pastu un paroli.		
Ievaddati/situācijas apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Tiek ievadīts pareizs e-pasts un pareiza parole.	Lietotājs piekļūst sistēmas panelim, tiek izveidota JWT sesija.	Pareizi
2. Tiek ievadīts pareizs e-pasts, bet nepareiza parole.	Piekļuve tiek liegta, parādās kļūdas paziņojums "Nepareizs e-pasts vai parole".	Pareizi
3. Tiek atstāts tukšs e-pasta lauks un nospiesta poga "Pieslēgties".	Sistēma neļauj nosūtīt formu un pieprasa aizpildīt obligātos laukus.	Pareizi

7.2. tabula

Preču meklēšanas un sekošanas saraksta pārvaldības testa pārskats

Tabulā dokumentēta funkcija, kas ļauj lietotājam atrast interesējošo parfimēriju pēc atslēgvārdiem (piemēram, "Sauvage") un pievienot to savam sekošanas sarakstam.

Lietotāja loma :	Autorizēts lietotājs	
Prasības identifikātors :	PS 2.2.2	
Prasības apraksts		
Lietotājam jāspēj atrast interesējošo preci un pievienot to sekošanas sarakstam cenu monitoringam.		
Ievaddati/situācijas apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Meklēšanas joslā tiek ievadīts frāze "Sauvage" un nospiests meklēt.	Tiek attēlots saraksts ar visām precēm, kas satur šo atslēgvārdu.	Pareizi
2. Lietotājs uzklikšķina uz "Track Price" pogas konkrētai precei.	Prece tiek pievienota "Favorites" sadaļai, parādās apstiprinājuma paziņojums.	Pareizi
3. Meklēšanas joslā tiek ievadīta nejauša simbolu virkne (piem. "asdfgh").	Sistēma izvada ziņojumu: "Prece ar šādu nosaukumu nav atrasta".	Pareizi

7.3. tabula

Preču filtrēšanas un cenu vēstures analīzes testa pārskats

Šī testa mērķis ir verificēt interaktīvo cenu grafiku darbību. Tiek pārbaudīts, vai sistēma spēj pareizi vizualizēt vēsturiskos datus, mērogot grafiku atbilstoši izvēlētajam filtram un kā sistēma informē lietotāju gadījumos, kad precei cenu vēsture vēl nav uzkrāta.

Lietotāja loma :	Autorizēts lietotājs	
Prasības identifikātors :	PS 2.2.3	
Prasības apraksts		
Sistēmai jāvizualizē konkrētas preces cenu vēsture interaktīvā grafikā.		
Ievaddati/situācijas apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Lietotājs atver preces detaļu lapu, kurai datubāzē ir vēsturiskie dati.	Tiek ģenerēts interaktīvs grafiks ar cenu svārstībām laika gaitā.	Pareizi
2. Lietotājs izvēlas filtru "Last 30 days" cenu analīzei.	Grafiks automātiski mērogojas un parāda datus tikai par pēdējo mēnesi.	Pareizi
3. Tiek atvērta prece, kas tikko pievienota un kurai nav cenu vēstures.	Sistēma attēlo paziņojumu: "Cenu vēsture šai precei vēl nav pieejama"	Pareizi

7.4. tabula

Personalizētu tirgus analītikas atskaišu eksportēšana

Tabulā aprakstīta datu eksportēšanas funkcijas pārbaude dažādos formātos. Testi apstiprina sekmīgu **PDF** un **Excel** failu ģenerēšanu ar strukturētu informāciju (zīmols, nosaukums, cena u.c.), kā arī sistēmas spēju atpazīt situācijas, kad atlasītajā periodā nav pieejamu datu eksportam.

Lietotāja loma :	Autorizēts lietotājs	
Prasības identifikātors :	PS 2.2.5	
Prasības apraksts		
Funkcija ļauj lietotājam ģenerēt un lejupielādēt strukturētus datus par cenu svārstībām manuālai analīzei vai atskaitēm.		
Ievaddati/situācijas apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Lietotājs izvēlas laika periodu "Pēdējās 30 dienas", atlasa zīmolu "Dior" un PDF formātu.	Sistēma ģenerē un automātiski lejupielādē PDF failu ar vizualizētiem datiem.	Pareizi
2. Lietotājs izvēlas Excel (.xlsx) izvades formātu visam preču katalogam.	Tiek lejupielādēts strukturēts Excel fails, kas satur kolonnas: Zīmols, Nosaukums, Cena, Veikals.	Pareizi
3. Tiek mēģināts eksportēt datus par periodu, kurā datubāzē nav fiksētu cenu punktu.	Sistēma izvada paziņojumu: "Atlasītajā periodā nav pieejami dati par izvēlētajām precēm".	Pareizi

7.5. tabula

Administratora funkciju un sistēmas uzraudzības testa pārskats

Pēdējā tabula fokusējas uz administratora lomas specifiskajām darbībām. Tiek pārbaudīta skenēšanas procesa manuāla palaišana un reāllaika žurnālu uzraudzība, kā arī sistēmas drošība, nodrošinot, ka lietotājiem bez administratora tiesībām piekļuve vadības panelim tiek liegta ar 403 kļūdu.

Lietotāja loma :	lietotājs ar administratora tiesībām	
Prasības identifikātors :	PS 2.2.6	
Prasības apraksts		
Administratoram jābūt iespējai manuāli palaist skeneri un pārraudzīt tā darba žurnālus.		
Ievaddati/situācijas apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Administrators Admin panelī nospiež pogu "Run Scraper" veikalam Notino.lv.	Sākas skenēšanas process, stāvoklis nomainās uz "RUNNING", parādās reāllaika logi	Pareizi
2. Lietotājs bez administratora tiesībām mēģina piekļūt /admin lapai.	Sistēma atgriež 403 kļūdu vai pāradresē uz galveno lapu.	Pareizi

NOBEIGUMS

Kvalifikācijas darba izstrādes rezultātā ir izveidota inovatīva un autonoma vīriešu parfimērijas tirgus uzraudzības un analīzes sistēma "**MarketPulse**". Darba gaitā tika pilnībā sasniegts izvirzītais mērķis un realizēti visi uzdevuma nostādnē definētie funkcionālie moduļi. Sistēmas pamatā ir unikāls hibrīdais datu ieguves dzinējs, kas apvieno asinhronos HTTP pieprasījumus ar pārlūkprogrammas emulāciju (izmantojot *Playwright*), nodrošinot stabilu datu vākšanu no pieciem lielākajiem Latvijas e-komercijas resursiem: **Notino, Douglas, Makeup, 220.lv** un **1a.lv**. Īpaša uzmanība tika veltīta mākslīgā intelekta integrācijai - izmantojot **Google Gemini AI**, tika ieviests "pašārstēšanās" mehānisms, kas automātiski atjauno parsēšanas noteikumus vietņu struktūras izmaiņu gadījumā, garantējot sistēmas ilgtermiņa ilgtspēju. Izstrādātā sistēma nodrošina pilnu datu apstrādes ciklu: no nosaukumu normalizācijas un inteliģentas preču sasaistes (Ķēdes loģika ar >95% sakritību) līdz interaktīvu cenu vēstures grafiku vizualizācijai un tūlītēju paziņojumu nosūtīšanai caur **Telegram** un e-pastu.

Sistēma pašlaik ir pilnībā darboties spējīga un izvietota **DigitalOcean** mākoņserverī, izmantojot **Docker** konteinerizāciju un automatizētu **CI/CD** darba plūsmu caur GitHub Actions. Tehnoloģiskais steks, kura pamatā ir **Python 3.12, FastAPI, PostgreSQL** un **Redis**, ir pierādījis savu efektivitāti, nodrošinot ātru datu apstrādi un reāllaika analītikas pieejamību. "MarketPulse" var kalpot kā vērtīgs palīgrīks gan patērētājiem izdevīgāko pirkumu atrašanai, gan tirgus analītiķiem konkurentu cenu politikas pētīšanai. Projekta arhitektūra ir veidota pēc universāluma principa, kas ļauj to nākotnē viegli mērogot, pievienojot jaunus interneta veikalus ne tikai Latvijas, bet arī visas Baltijas mērogā. Turpmākā attīstība varētu ietvert mobilās lietotnes izstrādi, **Vision-Language modeļu (VLM)** ieviešanu vizuālai vietņu analīzei un mašīnmācīšanās algoritmu izmantošanu cenu tendenču prognozēšanai, balstoties uz uzkrāto cenu vēstures bāzi.

INFORMĀCIJAS AVOTI

1. [LAT] Fizisko personu datu apstrādes likums: Latvijas Republikas likums - https://likumi.lv/ta/id/300099-fizisko-personu-datu-apstrades-likums - (Resurss apskatīts 15.12.2025.).
2. [LAT] Informācijas tehnoloģiju drošības likums: Latvijas Republikas likums - https://likumi.lv/ta/id/221379-informacijas-tehnologiju-drosibas-likums - (Resurss apskatīts 20.12.2025.).
3. [ANG] APScheduler: Job stores and persistence configuration - https://apscheduler.readthedocs.io/en/3.x/userguide.html#configuring-the-scheduler - (Resurss apskatīts 15.04.2026.).
4. [ANG] Asyncpg API: Connection class and execute method reference - https://magicstack.github.io/asyncpg/current/api/index.html#asyncpg.connection.Connection.execute - (Resurss apskatīts 28.04.2026.).
5. [ANG] curl_cffi: TLS and HTTP/2 Fingerprinting impersonation - https://curl-cffi.readthedocs.io/en/latest/impersonate/fingerprint.html#tls-and-http-2 - (Resurss apskatīts 22.04.2026.).
6. [ANG] Docker: Command-line interface and configuration files - https://docs.docker.com/reference/cli/docker/#configuration-files - (Resurss apskatīts 12.02.2026.).
7. [ANG] FastAPI: OAuth2 with Password hash and JWT tokens - https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/#hash-and-verify-the-passwords - (Resurss apskatīts 20.01.2026.).
8. [ANG] GitHub Actions: Essential features and workflow jobs - https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions#jobs - (Resurss apskatīts 18.04.2026.).
9. [ANG] Google Gemini API: Generating text from text-only inputs - https://ai.google.dev/gemini-api/docs/quickstart?lang=python#generate-text-from-text-inputs - (Resurss apskatīts 05.02.2026.).
10. [ANG] Passlib: Using the Bcrypt password hashing algorithm - https://passlib.readthedocs.io/en/stable/lib/passlib.hash.bcrypt.html#usage - (Resurss apskatīts 14.03.2026.).
11. [ANG] Playwright for Python: Browser contexts and isolation - https://playwright.dev/python/docs/browser-contexts#browser-contexts - (Resurss apskatīts 08.02.2026.).

12. [ANG] PostgreSQL 16: JSON functions and path operators - https://www.postgresql.org/docs/16/functions-json.html#FUNCTIONS-JSON-OP-TABLE - (Resurss apskatīts 20.02.2026.).
13. [ANG] Python 3.12: String methods and text processing - https://docs.python.org/3.12/library/stdtypes.html#text-sequence-type-str - (Resurss apskatīts 12.12.2025.).
14. [ANG] Python-jose: JWT Encoding and decoding examples - https://python-jose.readthedocs.io/en/latest/jwt/index.html#encoding-decoding - (Resurss apskatīts 22.03.2026.).
15. [ANG] Recharts: AreaChart component data structure - https://recharts.org/en-US/api/AreaChart#data - (Resurss apskatīts 02.04.2026.).
16. [ANG] Redis: Command reference for String data types - https://redis.io/docs/latest/develop/data-types/strings/ - (Resurss apskatīts 25.02.2026.).
17. [ANG] SQLModel: Defining Relationship attributes in models - https://sqlmodel.tiangolo.com/tutorial/relationship-attributes/ - (Resurss apskatīts 18.01.2026.).
18. [ANG] Telegram Bot API: Formatting options for message text - https://core.telegram.org/bots/api#formatting-options - (Resurss apskatīts 12.04.2026.).
19. [ANG] Vite: Environment variables and .env files guide - https://vite.dev/guide/env-and-mode.html#env-files - (Resurss apskatīts 19.03.2026.).

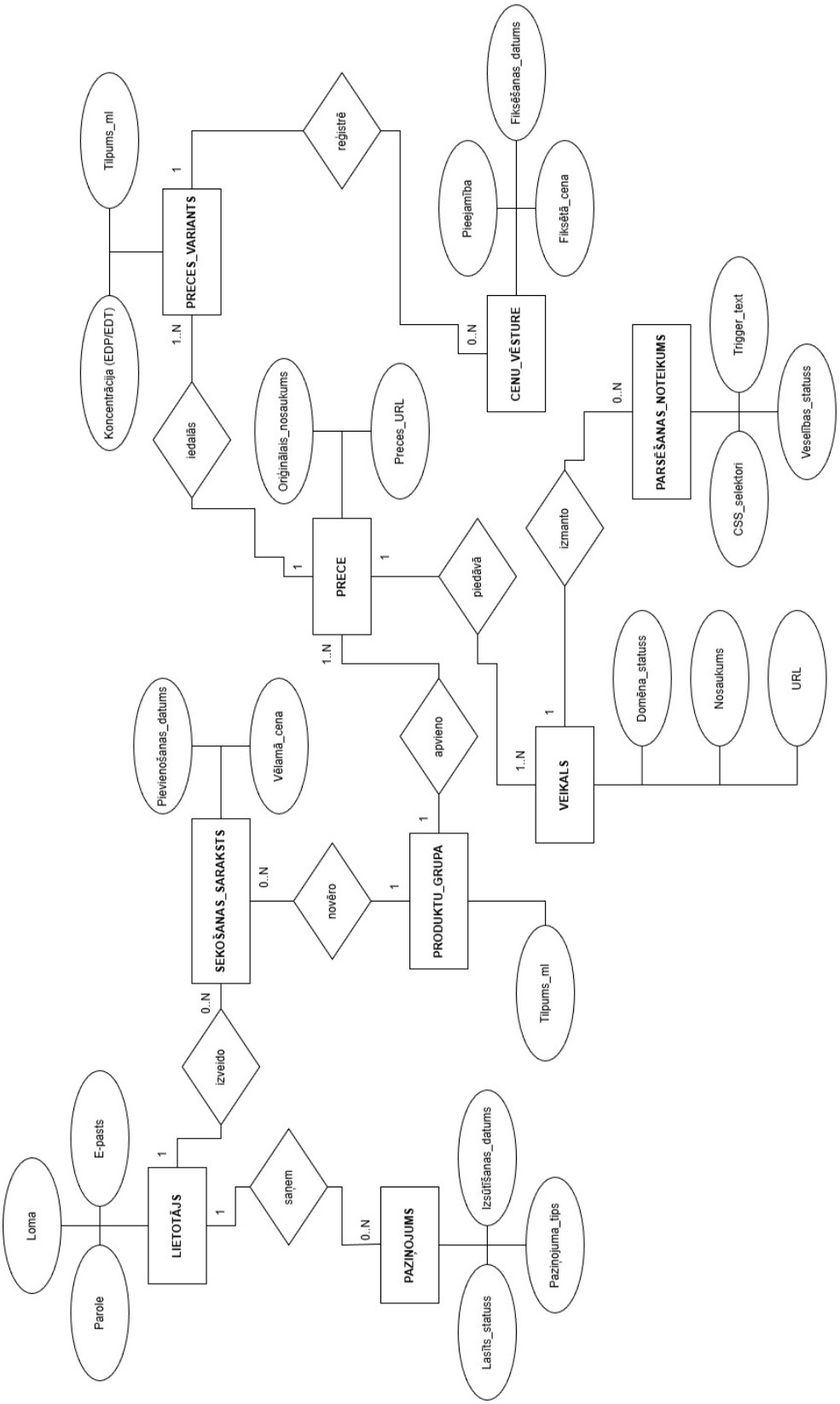
PIELIKUMI

1. pielikums

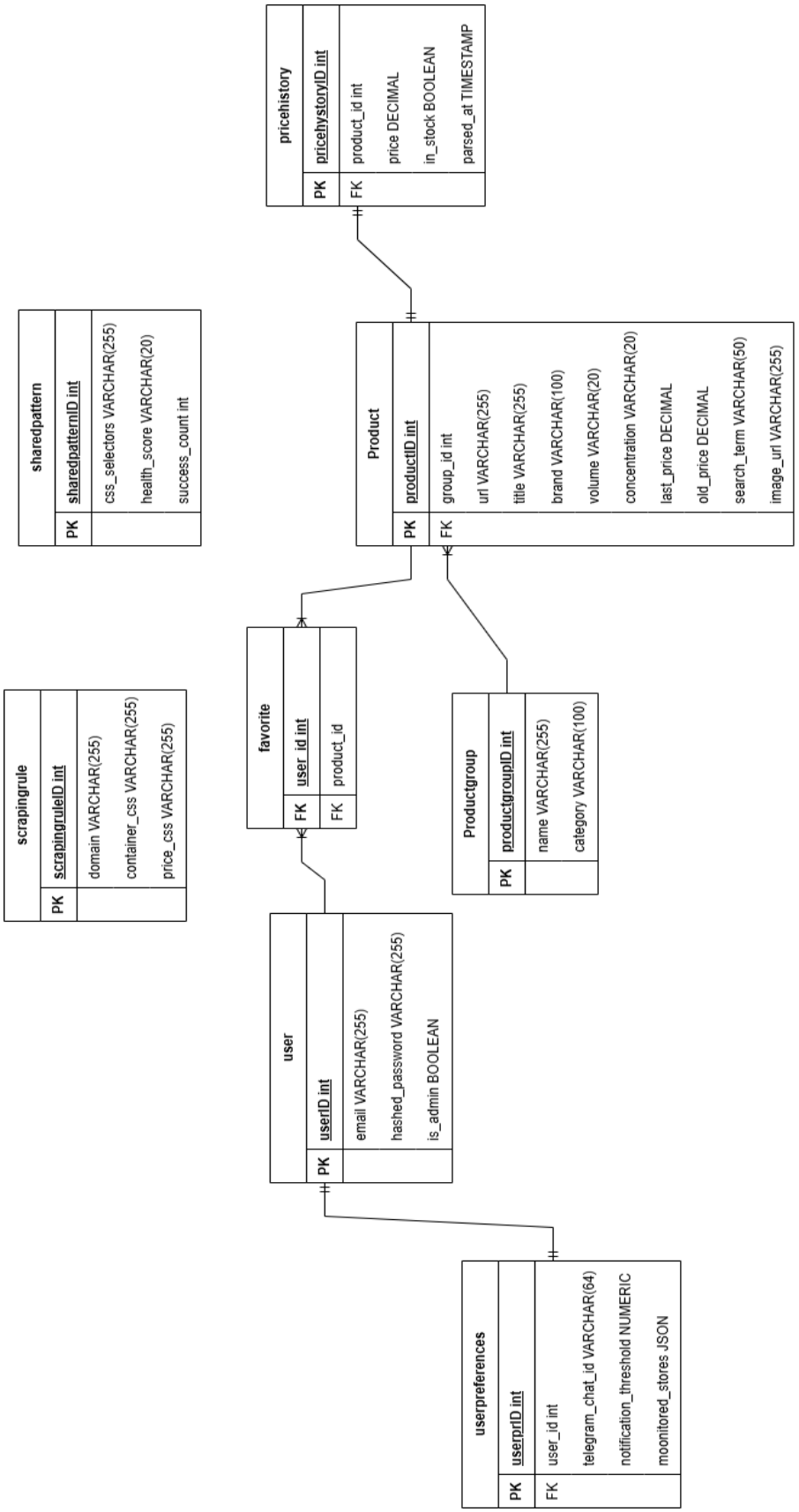
Funkcionālās dekompozīcijas diagramma



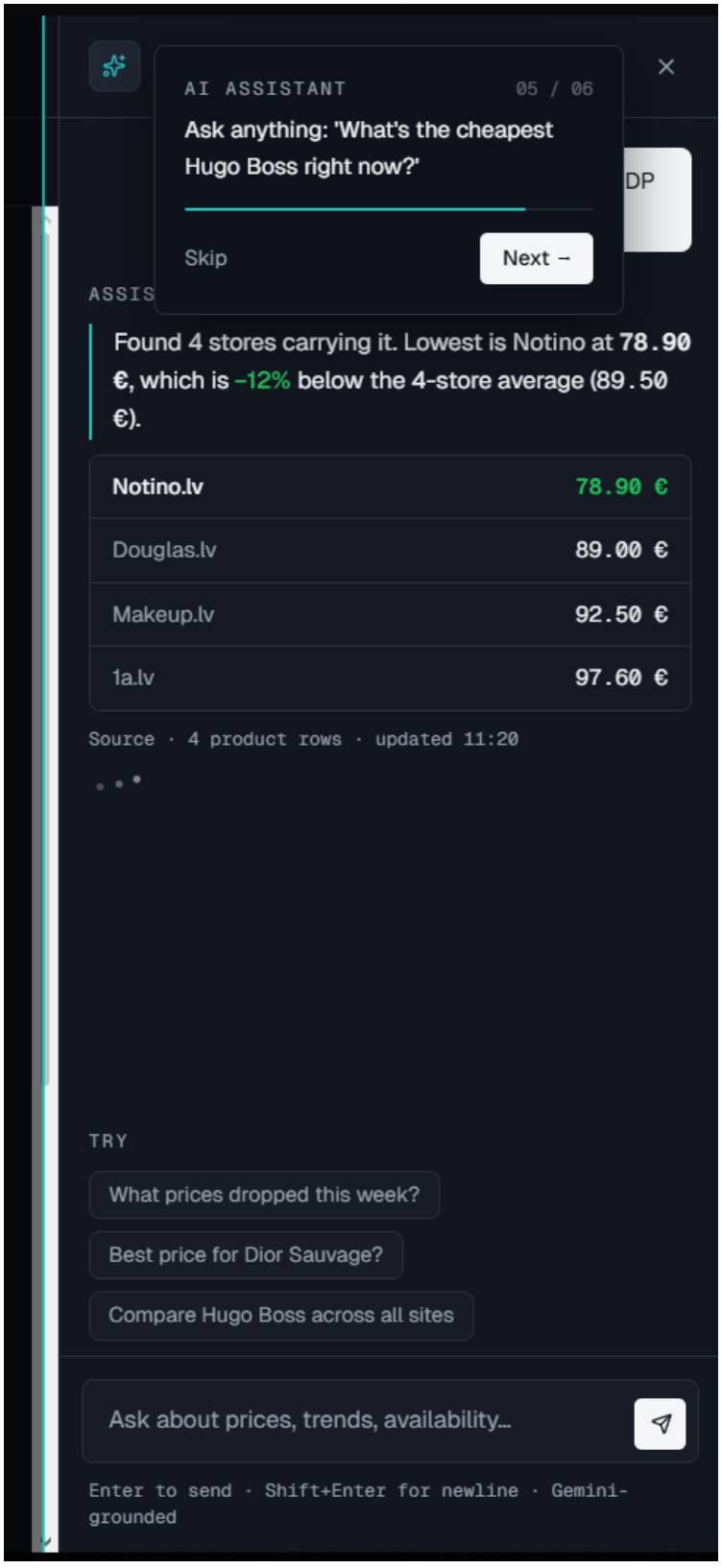
Sistēmas ER-diagramma



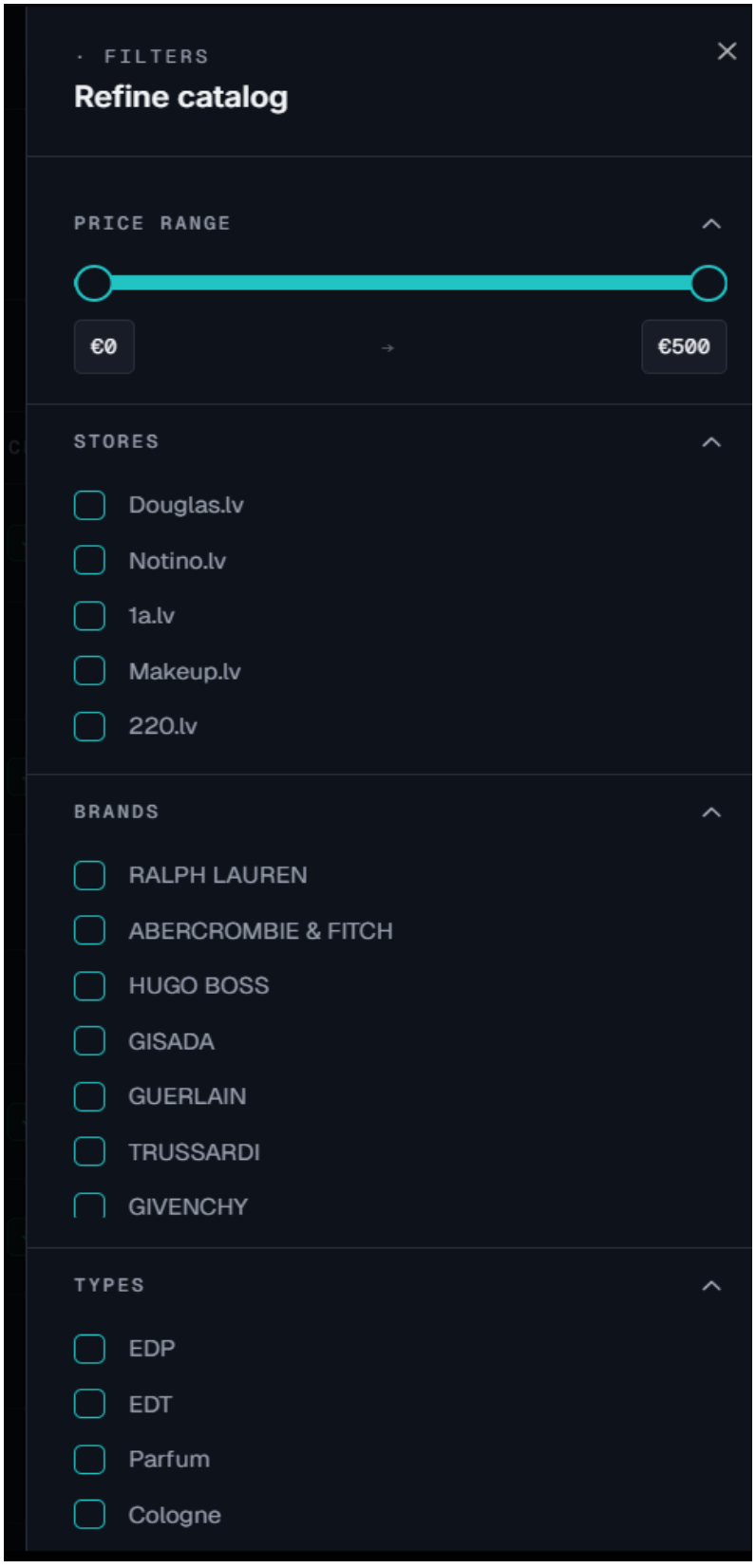
Datubāzes tabulu shēma



Pamācība Nr. 5



Datu atlāse veikaliem, precem un citiem datiem



Telegram savienojums



PDF fails (Kataloga eksports)

MarketPulse · Catalog Export

Generated 2026-05-15 10:24 UTC · **211 rows** · brands=Hugo Boss,Armaf

Brand	Title	Volume	Type	Store	Price (€)	Prev (€)	Δ%	Stock
Armaf	Armaf Art Du svaigs orient■ls koksnes 100ml	100ml	EDP	220.lv	36.90	36.90	—	Y
Armaf	Armaf Aura 100ml	100ml	EDP	220.lv	53.59	53.59	—	Y
Armaf	Armaf Blue 100ml	100ml	EDT	220.lv	25.69	25.69	—	Y
Armaf	Armaf Bucephalus XII Blue 100ml	100ml	EDP	220.lv	33.14	33.14	—	Y
Armaf	Armaf Caballo Citrus Woody 100ml	100ml	EDP	220.lv	34.42	34.42	—	Y
Armaf	Armaf Checkmate Knight Black Woody 100ml	100ml	EDP	220.lv	45.50	45.50	—	Y
Armaf	Armaf Club Nuit 105ml	105ml	EDT	220.lv	35.90	35.90	—	Y
Armaf	Armaf Club Nuit 105ml	105ml	EDT	220.lv	34.99	34.99	—	Y
Armaf	Armaf Club Nuit 105ml	105ml	EDT	Makeup.lv	42.59	42.59	—	Y
Armaf	Armaf Club Nuit 150ml	150ml	EDP	220.lv	52.99	52.99	—	Y
Armaf	Armaf Club Nuit 200ml	200ml	EDP	220.lv	67.90	67.90	—	Y
Armaf	Armaf Club Nuit 200ml	200ml	EDP	Makeup.lv	80.54	80.54	—	Y
Armaf	Armaf Club Nuit Blue Iconic 105ml	105ml	—	Makeup.lv	43.26	43.26	—	Y
Armaf	Armaf Club Nuit Blue Iconic 105ml	105ml	EDP	220.lv	48.99	48.99	—	Y
Armaf	Armaf Club Nuit Blue Iconic 200ml	200ml	—	Makeup.lv	91.19	91.19	—	Y
Armaf	Armaf Club Nuit Blue Iconic 30ml	30ml	—	Makeup.lv	33.69	33.69	—	Y
Armaf	Armaf Club Nuit Blue Iconic 30ml	30ml	EDP	220.lv	31.50	31.50	—	Y
Armaf	Armaf Club Nuit Iconic Blue 200ml	200ml	EDP	220.lv	105.00	105.00	—	Y
Armaf	Armaf Club Nuit Milestone 105ml	105ml	EDP	220.lv	38.90	38.90	—	Y
Armaf	Armaf Club Nuit Milestone 200ml	200ml	EDP	220.lv	91.63	91.63	—	Y
Armaf	Armaf Club Nuit Precieux 1 55ml	55ml	Parfum	Makeup.lv	45.20	45.20	—	Y

PDF fails (Produkta eksports)



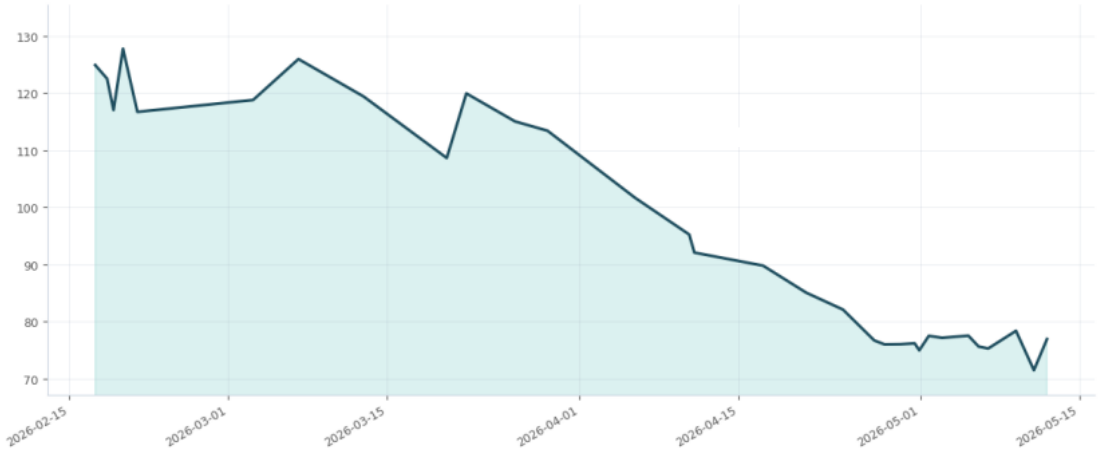
EISENBERG

EISENBERG Frai'che 30ml

30ml · EDP · Douglas.lv

75.99 € ~~was 115.00 €~~ **-33.9%**

Price history — 3 Months



Quick stats

Lowest	Highest	Average	Current	Observations	Spread
71.47 €	127.72 €	95.64 €	75.99 €	31	78.7%

Offers by store

Store	Price (€)	Previous (€)	Change
Douglas.lv	75.99	115.00	-33.9%

Programmas pirmteksts

models.py

```
from typing import Optional, List
from decimal import Decimal
from datetime import datetime
from sqlmodel import SQLModel, Field, Relationship, Column, JSON
from sqlalchemy import UniqueConstraint


# --- 1. PRODUKTU GRUPA (Apkopojums salīdzināšanai starp veikaliem) ---
class ProductGroup(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    # Piemērs: "Dior Sauvage EDP 100ml"
    name: str = Field(index=True, unique=True)
    category: Optional[str] = Field(default="General", index=True)
    created_at: datetime = Field(default_factory=datetime.utcnow)

    # Saistība ar konkrētiem piedāvājumiem dažādos veikalos
    products: List["Product"] = Relationship(back_populates="group")


# --- 2. IZSEKOŠANAS NOTEIKUMI (Domēna konfigurācija) ---
class ScrapingRule(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    domain: str = Field(unique=True, index=True)
    # Veikala nosaukums (Notino, Douglas, Makeup ...)
    site_name: str = Field(default="Nezināms veikals")
    emoji: str = Field(default="🛒")

    # CSS selektori
    container_css: str
    title_css: str
    price_css: Optional[str] = Field(default=None)
    image_css: Optional[str] = Field(default=None)
```

```
    # Pievienots, lai precīzāk noteiktu zīmolu (pirms tā tika izgūts no nosaukuma)
    brand_css: Optional[str] = Field(default=None)
    # Latviešu vietņu CONC elements (Douglas/Makeup); null vietnēm,
    # kur koncentrācija atrodas tieši nosaukumā
    concentration_css: Optional[str] = Field(default=None)
    stock_css: Optional[str] = Field(default=None)
    next_page_css: Optional[str] = Field(default=None)

    # Detaļlapas iegūšana (Deep Dive)
    use_detail_page: bool = Field(default=False)
    # Atslēgvārdi (atdalīti ar komatu), kas aktivizē padziļinātu iegūšanu
    # (piemēram, "sākot,from") — norāda uz vairākiem variantiem
    detail_trigger_text: Optional[str] = Field(default=None)
    # CSS katra varianta konteineram (piemēram, dažādiem tilpumiem)
    detail_variant_selector: Optional[str] = Field(default=None)
    # CSS varianta nosaukumam (piemēram, "50ml")
    detail_variant_name_css: Optional[str] = Field(default=None)
    # CSS konkrētā varianta cenai
    detail_variant_price_css: Optional[str] = Field(default=None)

    # Konteksta filtrs (JSON saraksts ar virknēm, kas norāda uz reklāmām)
    negative_keywords: Optional[str] = Field(default="[]")

    # Veselības rādītāji (autonomijai un pašatjaunošanai)
    last_success: Optional[datetime] = Field(default=None)
    last_error: Optional[str] = Field(default=None)
    success_count: int = Field(default=0)
    failure_count: int = Field(default=0)
    health_score: float = Field(default=1.0)  # 0.0 - 1.0

    def __repr__(self) -> str:
        return (
            f"<ScrapingRule domain='{self.domain}' "
            f"site='{self.site_name}' health={self.health_score}>"
        )

    # Atsākšanas iespēja (kontrolpunkti pēc kraha)
```

```
    last_page_url: Optional[str] = Field(default=None)
    items_scraped_current_run: int = Field(default=0)


# --- 3. PRODUKTS (Konkrēts piedāvājums veikalā) ---
class Product(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    url: str = Field(index=True)
    title: str
    image_url: Optional[str] = Field(default=None)
    description: Optional[str] = Field(default=None)

    # Meklēšanas termins (vai veikala domēns) — palīdz filtrēt katalogus
    search_term: str = Field(default="unknown", index=True)
    created_at: datetime = Field(default_factory=datetime.utcnow)

    # --- Normalizācija (Smaržu Ķēdēm — Perfume Chains) ---
    brand: Optional[str] = Field(default=None, index=True)
    volume: Optional[str] = Field(default=None, index=True)
    concentration: Optional[str] = Field(default=None, index=True)
    is_available: bool = Field(default=True)
    last_seen: datetime = Field(default_factory=datetime.utcnow)

    # Denormalizācija ātrākai piekļuvei (nav jāveic JOIN ar PriceHistory)
    last_price: Optional[Decimal] = Field(default=None)
    # Vecā cena — atlaižu izsekošanai
    old_price: Optional[Decimal] = Field(default=None)

    # Saistība ar grupu (cenu salīdzināšanai)
    group_id: Optional[int] = Field(default=None, foreign_key="productgroup.id")
    group: Optional[ProductGroup] = Relationship(back_populates="products")

    # Cenu vēsture (laikrindu dati)
    history: List["PriceHistory"] = Relationship(back_populates="product")

    def __repr__(self) -> str:
        return f"<Product title='{self.title[:30]}...' price={self.last_price} €>"
```

```
# --- 4. CENU VĒSTURE (Notikumu plūsma — Event Sourcing) ---
class PriceHistory(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    price: Decimal
    in_stock: bool
    parsed_at: datetime = Field(default_factory=datetime.utcnow)

    # Izmaiņu izsekošana — viena no šīm vērtībām:
    # initial, price_up, price_down, no_change, back_in_stock, out_of_stock
    change_type: str = Field(default="initial")

    product_id: Optional[int] = Field(default=None, foreign_key="product.id")
    product: Optional[Product] = Relationship(back_populates="history")


# --- 5. GLOBĀLIE MODEĻI (Sistēmas pieredze AI uzlabošanai) ---
class SharedPattern(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    # 'container', 'title', 'price', 'image', 'next_page'
    pattern_type: str = Field(index=True)
    css_selector: str = Field(unique=True)
    success_count: int = Field(default=1)
    last_used: datetime = Field(default_factory=datetime.utcnow)


# --- 6. LIETOTĀJS (Autentifikācija) ---
class User(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    email: str = Field(unique=True, index=True, max_length=255)
    full_name: Optional[str] = Field(default=None, max_length=100)
    hashed_password: str
    is_active: bool = Field(default=True)
    is_admin: bool = Field(default=False)
    created_at: datetime = Field(default_factory=datetime.utcnow)
# --- 7. LIETOTĀJA PREFERENCES (Onboarding + personalizācija) ---
```

```
class UserPreferences(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    user_id: int = Field(foreign_key="user.id", unique=True, index=True)
    # JSON masīvi — uzglabā elastīgus lietotāja izvēļu sarakstus
    monitored_stores: List[str] = Field(
        default_factory=list, sa_column=Column(JSON)
    )
    interested_brands: List[str] = Field(
        default_factory=list, sa_column=Column(JSON)
    )
    onboarding_completed: bool = Field(default=False)

    # Paziņojumu kanāli — Phase 4
    notification_channels: List[str] = Field(
        default_factory=list, sa_column=Column(JSON)
    )
    notification_threshold_percent: Decimal = Field(default=Decimal("5.0"))
    telegram_chat_id: Optional[str] = Field(default=None, max_length=64)
    # Vienreizējs tokens dziļās saites verifikācijai —
    # tiek dzēsts pēc veiksmīga /start komandas saņemšanas
    telegram_link_token: Optional[str] = Field(default=None, max_length=64)
    telegram_link_token_expires_at: Optional[datetime] = Field(default=None)



# --- 8. FAVORĪTS (Lietotāja vērojumu saraksta ieraksts) ---
class Favorite(SQLModel, table=True):
    # Unikāls ierobežojums datu bāzes līmenī —
    # novērš dublētus ierakstus pat sacensību apstākļos (race conditions)
    __table_args__ = (
        UniqueConstraint("user_id", "product_id",
name="uq_favorite_user_product"),
    )

    id: Optional[int] = Field(default=None, primary_key=True)
    user_id: int = Field(foreign_key="user.id", index=True)
    product_id: int = Field(foreign_key="product.id", index=True)
    alert_enabled: bool = Field(default=True)
```

```
# Cenas momentuzņēmums pievienošanas brīdī —
# ļauj aprēķināt ietaupījumus salīdzinājumā ar sākotnējo cenu
price_when_added: Optional[Decimal] = Field(default=None)
created_at: datetime = Field(default_factory=datetime.utcnow)
```

```
import json
from urllib.parse import urlparse
from typing import Optional
from google import genai
from google.genai import types
from bs4 import BeautifulSoup
from parsel import Selector
from loguru import logger
from sqlmodel import select, desc
from app.core.config import settings
from app.domain.models import ScrapingRule, SharedPattern


class AIGenerator:
    def __init__(self):
        # Gemini klients — aizstāj OpenAI dēļ zemākām izmaksām
        self.client = genai.Client(api_key=settings.GOOGLE_API_KEY)

    async def get_or_create_rule(
        self,
        session,
        url: str,
        html_content: str,
        force_refresh: bool = False,
        error_hint: Optional[str] = None
    ) -> Optional[ScrapingRule]:

        domain = urlparse(url).netloc.replace("www.", "")

        statement = select(ScrapingRule).where(ScrapingRule.domain == domain)
        result = await session.execute(statement)
        rule = result.scalar_one_or_none()

        if rule and force_refresh:
            logger.warning(f"♻️ Dzēšam veco noteikumu domēnam {domain}...")
```

```
                await session.delete(rule)
                await session.commit()
                rule = None

            if rule:
                logger.info(f"📍 Atrasts noteikums domēnam {domain}")
                return rule

            # --- VERIFICĒTS ĀTRAIS CEĻŠ: izlaist AI zināmiem domēniem ---
            # Šie selektori ir apstiprināti ar manuālu HTML izpēti.
            # Pārbaude PIRMĀ — novērš 3×15s = 45s tērēšanu AI izsaukumiem.
            VERIFIED_DETAIL_RULES = {
                # Verificēts 2026-04-17 ar HTML reverso inženieriju
                # PIEZĪME: klašu selektori (styled__*) mainās katrā Notino
izvietojumā.
                # Izmantojam data-testid atribūtus — stabili starp izvietojumiem.
                "notino.lv": {
                    "site_name": "Notino",
                    "emoji": "👃",
                    "container_css": (
                        "[data-testid='product-container']"
                        ":not([data-recommendation='promo-banner'])"
                    ),
                    # Notino renderē divus komponentus: ProductCard (Playwright)
                    # un ProductTile (īsts pārlūks). Abi selektori — caur komatu.
                    "title_css": (
                        "[data-testid='product-card-name'],"
                        " [data-testid='product-tile-name']"
                    ),
                    "brand_css": (
                        "[data-testid='product-card-brand'],"
                        " [data-testid='product-tile-brand']"
                    ),
                    "price_css": "[data-testid='price-component']",
                    "image_css": "img[loading='lazy']",
                    "next_page_css": ".nextPage[data-testid='page-button-wrapper']",
                    "use_detail_page": True,
```

```
    "detail_trigger_text": "sākot ar,no ",
},
# Verificēts 2026-04-17. Douglas Latvia mainīja HTML struktūru:
# vecais .product-tile → jaunais .product_element.
# Cena ir span.now (var saturēt "Šobrīd nav"/"Drīz tirdzniecībā!")
"douglas.lv": {
    "site_name": "Douglas Latvia",
    "emoji": " 💄 ",
    "container_css": "div.product_element",
    "title_css": "span.name",
    "brand_css": "span.brand_caps",
    "price_css": "span.price span.now",
    # Kataloga kartes koncentrācijas elements
    # ("Parfimērijas ūdens ...", "Tualetes ūdens ...");
    # detaļlapā izmanto span.product_type kā rezerves variantu.
    "concentration_css": "span.type, span.product_type",
    "image_css": "img.product-photo",
    "next_page_css": "a.page.next",
    "use_detail_page": True,
    "detail_trigger_text": "no ,sākot",
    "detail_variant_selector": "fieldset.items div.item",
    "detail_variant_name_css": "span.name",
    "detail_variant_price_css": "div.price span.now",
},
# Verificēts 2026-03-15 (scout_220.py)
# SPA (Pigu platforma) — produkti ielādēti ar pārlūka JS,
# parsēti caur widget-data JSON
"220.lv": {
    "site_name": "220.lv",
    "emoji": " ⚡ ",
    "container_css": "div.c-product-card",
    "title_css": ".c-product-card__title a",
    # Tikai rezerves variants; widget-data ir prioritāra
    "price_css": ".c-price:not(.h-price--loyalty)",
    "image_css": "img.c-product-card__image",
    "next_page_css": "[class*=pagination] a:last-child",
```

```
    },
    # Verificēts 2026-03-15 (analyze_1a.py)
    # Konteiners: 48/48 kartes apstiprinātas, BEM klases nosaukumi
    "1a.lv": {
        "site_name": "1a.lv",
        "emoji": "🛒",
        "container_css": "div.catalog-taxons-product",
        "title_css": "a.catalog-taxons-product__name",
        "price_css": "span.catalog-taxons-product-price__price-number",
        "image_css": "img.catalog-taxons-product__image",
        "next_page_css": "a.paginator__next",
    }
}

if domain in VERIFIED_DETAIL_RULES:
    # Izlaižam AI — veidojam noteikumu tieši no verificētajiem selektoriem
    logger.info(
        f"⚡ VERIFICĒTS ātrais ceļš domēnam {domain} — izlaižu AI"
    )
    override = VERIFIED_DETAIL_RULES[domain]

    # Atomiska pārbaude un izveide — lai izvairītos no IntegrityError
    stmt = select(ScrapingRule).where(ScrapingRule.domain == domain)
    existing = (await session.execute(stmt)).scalar_one_or_none()

    if existing:
        for field, value in override.items():
            setattr(existing, field, value)
        new_rule = existing
    else:
        new_rule = ScrapingRule(domain=domain)
        for field, value in override.items():
            setattr(new_rule, field, value)

    session.add(new_rule)
    await session.commit()
```

```
            await session.refresh(new_rule)
            logger.success(
                f"🍾 VERIFICĒTS noteikums izveidots/atjaunināts domēnam {domain}"
            )
            return new_rule

        new_rule = await self.generate_rule(
            session, url, html_content, error_hint=error_hint
        )

        # --- VALIDĀCIJAS CIKLS: pārbaudīt, ka noteikums atrod vairākus elementus
---
        if new_rule:
            sel = Selector(text=html_content)
            cards_found = len(sel.css(new_rule.container_css))

            attempts = 1
            max_attempts = 3
            while cards_found < 5 and attempts < max_attempts:
                logger.warning(
                    f"⚠️ Noteikums {domain} atrada tikai {cards_found} "
                    f"elementus. Atkārtoti vaicā AI..."
                )
                attempts += 1
                hint = (
                    f"Tavs iepriekšējais container_css "
                    f"'{new_rule.container_css}' atrada tikai "
                    f"{cards_found} elementus. Man vajag selektoru "
                    f"galvenajam produktu režģim (parasti 10-50 elementi)."
                )
                new_rule = await self.generate_rule(
                    session, url, html_content, error_hint=hint
                )
                if not new_rule:
                    break
                cards_found = len(sel.css(new_rule.container_css))
```

```
    if new_rule:
        new_rule.domain = domain
        session.add(new_rule)
        await session.commit()
        await session.refresh(new_rule)
        logger.success(f"💾 Jaunais noteikums domēnam {domain} saglabāts.")
        return new_rule

    return None

def simplify_html(self, soup: BeautifulSoup) -> str:
    """
    Samazina tokenu patēriņu, noņemot atkārtojošos brāļu elementus
    (saraksta vienumus). Saglabā struktūru, bet noņem masu.
    """
    for parent in soup.find_all(True):
        # Grupējam bērnus pēc taga nosaukuma, lai identificētu sarakstus
        children = [
            child for child in parent.find_all(recursive=False)
            if isinstance(child.name, str)
        ]
        if len(children) > 10:
            # Ja ir daudz viena taga bērnu (piem., daudz LI vai DIV),
            # uzskatām to par sarakstu.
            # Saglabājam pirmos 5 un pēdējo 1, lai parādītu struktūru.
            first_tag = children[0].name
            similar_children = [
                c for c in children if c.name == first_tag
            ]

            if len(similar_children) > 10:
                # Saglabājam pirmos 5 un pēdējo 1
                for child in similar_children[5:-1]:
                    child.decompose()

    return str(soup)
```

```
def clean_html(self, html: str, limit: int = 0) -> str:
    try:
        soup = BeautifulSoup(html, "html.parser")

        # Noņemam ne-vizuālus vai smagus tagus
        for element in soup(
            ["script", "style", "svg", "noscript",
             "meta", "link", "iframe", "head"]
        ):
            element.decompose()

        # Noņemam HTML komentārus
        from bs4 import Comment
        for comment in soup.find_all(
            string=lambda text: isinstance(text, Comment)
        ):
            comment.extract()

        cleaned = self.simplify_html(soup)

        # Ja ir norādīts skaidrs limits (piem., detaļlapām),
        # izmantojam vienkāršu apgriešanu
        if limit > 0:
            if len(cleaned) > limit:
                logger.warning(
                    f"⚠️ Apgriežu detaļu HTML no {len(cleaned)} līdz {limit}"
                )
                return cleaned[:limit]

        # VIEDĀ APGRIEŠANA KATALOGIEM
        # Mums vajag SĀKUMU (galvenes, pirmie elementi)
        # un BEIGAS (lapošana). Vidus ir atkārtojošās produktu kartes
        # — to var droši izgriezt.
        MAX_TOTAL = 150000
        if len(cleaned) > MAX_TOTAL:
```

```
                keep_size = MAX_TOTAL // 2
                head = cleaned[:keep_size]
                tail = cleaned[-keep_size:]
                logger.info(
                    f"✂️ Viedā apgriešana: saglabāti pirmie un pēdējie "
                    f"{keep_size} simboli."
                )
                return head + "\n... [VIDĒJAIS SATURS NOŅEMTS] ...\n" + tail

            return cleaned

        except Exception as e:
            logger.error(f"HTML tīrīšanas kļūda: {e}")
            return html[:30000]

    async def generate_rule(
        self,
        session,
        url: str,
        html_content: str,
        error_hint: str = None
    ) -> ScrapingRule:
        logger.info(f"🤖 Pieprasu AI palīdzību: {url}...")

        # Izmantojam viedo tīrīšanu
        clean_content = self.clean_html(html_content)

        # --- KOLEKTĪVĀ INTELIĢENCE: ielādē veiksmīgākos modeļus no DB ---
        stmt = (
            select(SharedPattern)
            .order_by(desc(SharedPattern.success_count))
            .limit(10)
        )
        db_patterns = (await session.execute(stmt)).scalars().all()

        knowledge_base = ""
```

```
if db_patterns:
    knowledge_base = (
        "\nVeiksmīgi modeļi no citām vietnēm "
        "(mēģini izmantot līdzīgu loģiku):\n"
    )
    for p in db_patterns:
        knowledge_base += f"- {p.pattern_type}: {p.css_selector}\n"

# Kļūdas konteksts — nosūtīts tikai pēc neveiksmīga mēģinājuma
error_context = (
    f"\nSVARĪGI: Tavs iepriekšējais mēģinājums neizdevās: "
    f"{error_hint}. NEIZMANTO tos pašus selektorus."
) if error_hint else ""

prompt = f"""
Rīkojies kā vecākais tīmekļa datu ieguves speciālists.
Mērķa URL: {url}

Uzdevums: Analizē vienkāršoto HTML zemāk un atklāj CSS selektorus
produktu datu iegūšanai.

KRITISKĀS PRASĪBAS:
1. Stabilitāte: dod priekšroku `data-testid`, `id`, `itemprop` vai
   jēgpilniem klašu nosaukumiem (piem., 'product-card', 'price')
   pār nejaušiem jaucējiem (piem., 'css-1x2y3z').
2. Konteiners: atrodi ATKĀRTOJOŠOS elementu, kas aptver vienu produktu
   GALVENAJĀ kataloga režģī (parasti 10-50 reizes).
3. Precizitāte: nosaukuma, cenas un attēla selektoriem
   jābūt RELATĪVIEM pret konteineru.
4. Prettroksnis: IGNORĒ atsevišķus elementus, "Ieteicamie" blokus,
   "Nesen skatīts" karuseļus, "Grozs" sānjoslas.
5. Varianti: pārbaudi, vai cenai ir prefikss "sākot no", "no", "from".
   Tas norāda uz vairākām opcijām. Atgriez šo atslēgvārdu.

LAPOŠANAS STRATĒĢIJA (KRITISKI):
- Atrodi pogu/saiti "Nākamā lapa".
- Meklē rel="next", aria-label="Next", aria-label="Nākamā".
```

```
        - Meklē tekstu "Next", ">", "»", "Nākamā".
        - Pārliecinies, ka tas ir NOKLIKŠĶINĀMS elements (a tag vai poga).
        - NEVIS atspējota poga.

        {knowledge_base}
        {error_context}

        HTML FRAGMENTS (Viedi apgriezts — saglabāta galva un aste):
        {clean_content}

        ATGRIEZ JSON:
        {{
            "site_name": "Veikala nosaukums",
            "emoji": "🛒",
            "container_css": "atšķirīgs selektors produkta kartei",
            "title_css": "selektors produkta nosaukumam konteinerā",
            "brand_css": "selektors zīmola nosaukumam",
            "price_css": "selektors cenas tekstam",
            "concentration_css": "selektors atsevišķam koncentrācijas elementam",
            "image_css": "selektors img tagam konteinerā",
            "next_page_css": "stabils selektors pogai 'Nākamā'",
            "detail_trigger_text": "teksta paraugs, kas norāda uz vairākiem
variantiem",
            "negative_keywords": ["sponsored", "ad", "promo"]
        }}
        """

        try:
            response = await self.client.aio.models.generate_content(
                model=settings.GEMINI_MODEL,
                contents=prompt,
                config=types.GenerateContentConfig(
                    response_mime_type="application/json"
                )
            )
            cleaned_text = (
                response.text.replace('```json', '').replace('```', '').strip()
```

```
)
parsed_response = json.loads(cleaned_text)
logger.success(f"🤖 AI ģenerēja: {parsed_response}")

# Dažreiz AI atgriež sarakstu ar vienu vārdnīcu — izlīdzinām
if isinstance(parsed_response, list) and len(parsed_response) > 0:
    parsed_response = parsed_response[0]

if not isinstance(parsed_response, dict):
    raise ValueError(
        f"AI atgrieza nederīgu struktūru: "
        f"{type(parsed_response)} (gaidījām dict)"
    )

if (not parsed_response.get("container_css")
        or not parsed_response.get("title_css")):
    return None

domain = urlparse(url).netloc.replace("www.", "")

return ScrapingRule(
    domain=domain,
    site_name=parsed_response.get("site_name", "Nezināms veikals"),
    emoji=parsed_response.get("emoji", "🛒"),
    # Drošas noklusētās vērtības, ja AI atgriež null
    container_css=parsed_response.get("container_css") or "div",
    title_css=parsed_response.get("title_css") or "h1",
    brand_css=parsed_response.get("brand_css"),
    concentration_css=parsed_response.get("concentration_css"),
    price_css=parsed_response.get("price_css"),
    image_css=parsed_response.get("image_css"),
    next_page_css=parsed_response.get("next_page_css"),
    negative_keywords=json.dumps(
        parsed_response.get("negative_keywords", [])
    ),
    # Īpaša loģika padziļinātai iegūšanai
```

```
            use_detail_page=bool(
                parsed_response.get("detail_trigger_text")
                or domain == "notino.lv"
            ),
            detail_trigger_text=parsed_response.get("detail_trigger_text")
        )
    except Exception as e:
        logger.error(f"AI kļūda: {e}")
        return None

async def generate_detail_rule(
    self, url: str, html_content: str
) -> dict:
    """
    Analizē atsevišķu produkta lapu, lai atrastu, kā ir strukturēti
    varianti (tilpumi/cenas).
    """
    logger.info(f"🧠 AI dziļā analīze produktu lapai: {url}...")
    # Stingrs limits detaļlapām — lai izvairītos no 429 un noildzes
    clean_content = self.clean_html(html_content, limit=50000)

    prompt = f"""
    Rīkojies kā vecākais tīmekļa datu ieguves speciālists.
    Mērķa produkta URL: {url}

    Uzdevums: analizē produkta lapas HTML un atrodi, kur atrodas
    dažādi varianti (piem., tilpumi 30ml, 50ml, 100ml) un to cenas.

    KRITISKĀS PRASĪBAS:
    1. Variantu konteiners: CSS selektors elementam, kas aptver
       KATRU varianta opciju (piem., saraksta vienums vai div režģī).
    2. Varianta nosaukums: selektors (relatīvs pret konteineru) tekstam,
       kas apraksta variantu (piem., "50 ml").
    3. Varianta cena: selektors (relatīvs pret konteineru) cenai
       šim konkrētajam variantam.
    4. Stabilitāte: dod priekšroku `data-testid` vai semantiskām klasēm.
```

```
HTML FRAGMENTS:
{clean_content}

ATGRIEZ TIKAI JSON:
{{
    "variant_container_css": "selektors atkārtotajam varianta elementam",
    "variant_name_css": "selektors tilpumam relatīvi pret konteineru",
    "variant_price_css": "selektors cenai relatīvi pret konteineru"
}}
"""

try:
    response = await self.client.aio.models.generate_content(
        model=settings.GEMINI_MODEL,
        contents=prompt,
        config=types.GenerateContentConfig(
            response_mime_type="application/json"
        )
    )
    cleaned_text = (
        response.text.replace('```json', '').replace('```', '').strip()
    )
    data = json.loads(cleaned_text)

    if isinstance(data, list) and len(data) > 0:
        data = data[0]

    if not isinstance(data, dict):
        logger.error(f"AI atgrieza nederīgu struktūru: {type(data)}")
        return {}

    logger.success(f"📌 AI detaļu noteikums ģenerēts: {data}")
    return data
except Exception as e:
    logger.error(f"AI detaļu analīzes kļūda: {e}")
```

```
            return {}


ai_generator = AIGenerator()
```

```
async def _scrape_network_variants(self, url: str) -> list[dict]:
    """
    Atver lapu, klikšķina variantus un pārtver JSON lietderīgās kravas,
    lai precīzi iegūtu cenas un tilpuma datus.
    """
    import json
    from playwright.async_api import async_playwright
    from app.core.browser import browser_manager
    from app.core.constants import (
        COOKIE_ACCEPT_BUTTONS,
        OUT_OF_STOCK_PHRASES,
        MEASUREMENT_UNITS,
    )

    logger.info(f"🕷️ [Tīkla pārtveršana] Dziļā analīze: {url}")
    variants = []

    context = await browser_manager.get_context()
    page = await context.new_page()

    # Divi atsevišķi tīkla pieprasījumu plūsmas avoti
    intercepted_requests = {"luigisbox": [], "detail": []}

    async def handle_request(request):
        try:
            req_url = request.url
            if request.method in ["POST", "PUT"]:
                # STINGRS NOTEIKUMS CENAI: luigisbox analītikas notikumi
                if "collector-lv.notino.luigisbox.com/events" in req_url:
                    post_data = request.post_data_json
                    if post_data:
                        intercepted_requests["luigisbox"].append(post_data)
                # NOTEIKUMS TILPUMAM UN PIEEJAMĪBAI: detail API
                elif "/api/tracking/web/view/detail" in req_url:
                    post_data = request.post_data_json
```

```
                if post_data:
                    intercepted_requests["detail"].append(post_data)
    except Exception:
        pass

page.on("request", handle_request)

try:
    # Bloķējam attēlus ātrumam — tāpat tos neizmantojam
    await page.route(
        "**/*.{png,jpg,jpeg,gif,webp,svg,ico}",
        lambda route: route.abort()
    )
    await page.goto(url, wait_until="domcontentloaded", timeout=60000)

    # Automātiski apstiprinām sīkdatnes — citādi tās bloķē klikšķus
    cookie_selectors = COOKIE_ACCEPT_BUTTONS

    # Notino lēni renderē ar React — gaidām līdz 3s
    await asyncio.sleep(1.0)

    for btn_sel in cookie_selectors:
        cookie_btn = page.locator(btn_sel).first
        if await cookie_btn.is_visible(timeout=1000):
            await cookie_btn.click(force=True)
            await asyncio.sleep(0.5)
            break

    # Gaidām sākotnējos izsekotājus
    await asyncio.sleep(0.5)

    # Atrast un noklikšķināt variantus.
    # Paplašināti selektori — ietver vizuālus attēlu flīžu variantus
    # (piem., Scandal Elixir stilā)
    variant_els = await page.locator(
        "div[data-testid*='variant'], "
        "span[class*='variant'], "
```

```
    ".pd-variant-label, "
    "[data-testid='product-variants'] > div, "
    "div[class*='VariantContainer'] > div"
).all()
if not variant_els:
    # Rezerves variants — meklējam jebkuru elementu ar "ml"
    variant_els = await page.locator(
        "div:has-text(' ml'), button:has-text(' ml')"
    ).all()

# 1. FĀZE: noklikšķināt visus variantus, lai radītu tīkla trafiku
dom_variants_map = {}
import re
for i, el in enumerate(variant_els):
    if await el.is_visible():
        try:
            text = await el.inner_text()
            vol = ""
            units_pattern = '|'.join(MEASUREMENT_UNITS)
            m_vol = re.search(
                rf'(\d+(?:[.,]\d+)?\s*({units_pattern}))',
                text, re.IGNORECASE
            )
            if m_vol:
                vol = m_vol.group(1).lower().replace(' ', '')
                unit_match = m_vol.group(2).lower()
                vol = vol.replace(unit_match, f' {unit_match}')

                text_lower = text.lower()
                is_avail = not any(
                    phrase in text_lower
                    for phrase in OUT_OF_STOCK_PHRASES
                )

                if (vol not in dom_variants_map
                        or dom_variants_map[vol]["price"] == 0.0):
                    dom_variants_map[vol] = {
```

```
                "price": 0.0,
                "available": is_avail
            }

    # Atkārtoti mēģinām, ja sīkdatnes bloķēja pirmo klikšķi
    if i == 0:
        for btn_sel in cookie_selectors:
            btn = page.locator(btn_sel).first
            if await btn.is_visible(timeout=100):
                await btn.click(force=True)
                await asyncio.sleep(0.5)

    await el.click(timeout=1500, force=True)
    # 0.8s — optimālais laiks: ļauj React atjaunināt cenu
    # un apiet stingru analītikas izmešanu
    await asyncio.sleep(0.8)

    # Ideālais rezerves variants:
    # nolasām vizuāli atjaunināto cenu tieši no ekrāna
    if (vol and dom_variants_map[vol]["price"] == 0.0
            and dom_variants_map[vol]["available"]):
        try:
            main_price = await page.locator(
                "[data-testid='price-component']"
            ).first.inner_text(timeout=500)
            p_val = (
                self.parse_price(main_price)
                if hasattr(self, 'parse_price') else 0.0
            )
            if not p_val:
                m_pr = re.search(r'(\d+[,.]\d+)', main_price)
                if m_pr:
                    p_val = float(
                        m_pr.group(1).replace(',', '.')
                    )
            if p_val:
                dom_variants_map[vol]["price"] = p_val
```

```
                except Exception:
                    pass
        except Exception:
            pass

# Gaidām nokavējušos tīkla pieprasījumus
await asyncio.sleep(1.0)

variants_map = {}

# 2. FĀZE: parsē visus /detail pieprasījumus tilpumam un pieejamībai
for req_data in intercepted_requests["detail"]:
    code = req_data.get("code")
    if code:
        size = req_data.get("size")
        sizeUnit = req_data.get("sizeUnit")
        is_available = req_data.get("stockQuantity") != "outOfStock"
        if size and sizeUnit:
            variants_map[code] = {
                "volume": f"{size} {sizeUnit}",
                "available": is_available
            }

# 3. FĀZE: parsē visus luigisbox pieprasījumus cenām
for req_data in intercepted_requests["luigisbox"]:
    events = req_data.get("events", [])
    for event in events:
        event_data = event.get("data", {})

        products = list(event_data.get("products", []))
        if "related_to" in event_data:
            products.extend(
                event_data["related_to"].get("products", [])
            )

        for p in products:
            p_code = p.get("product_code")
```

```
            # Saistām cenas TIKAI ar variantiem,
            # ko jau redzējām /detail pieprasījumos
            if p_code and p_code in variants_map:
                p_discount = p.get("discount_price")
                p_full = p.get("full_price")

                # Stingrs noteikums: izmantot discount_price,
                # rezerve — full_price
                found_price = (
                    p_discount if p_discount is not None else p_full
                )
                if found_price is not None:
                    variants_map[p_code]["price"] = found_price

# 4. FĀZE: salikt galīgo dedublēto sarakstu
seen_volumes = set()
for code, data in variants_map.items():
    if ("price" in data
            and data["price"] != 0.0
            and data["price"] is not None):
        vol = data["volume"].lower().replace(' ', '')
        vol = vol.replace('ml', ' ml').replace('g', ' g')
        if vol not in seen_volumes:
            seen_volumes.add(vol)
            variants.append({
                "title_suffix": vol,
                # Decimal(str(...)) — pareizs paņēmiens,
                # lai izvairītos no IEEE 754 noapaļošanas kļūdām
                "price": (
                    Decimal(str(data["price"])).quantize(
                        Decimal("0.01")
                    )
                ),
                "is_available": data["available"]
            })

# Pievienojam trūkstošos DOM variantus
```

```
# (piem., izpārdotos, ko tīkls ignorēja, vai cenas, kas neizdevās)
for vol, dom_data in dom_variants_map.items():
    if vol not in seen_volumes:
        seen_price = dom_data.get("price")
        if seen_price is None or seen_price == 0.0:
            continue  # Izlaižam pilnībā nederīgas DOM cenas

        seen_volumes.add(vol)
        variants.append({
            "title_suffix": vol,
            "price": Decimal(str(seen_price)).quantize(Decimal("0.01")),
            "is_available": dom_data.get("available", True)
        })

# 5. FĀZE: rezerves variants atsevišķiem produktiem (bez pogām)
# Notino vienā variantā: īpaša tilpuma vieta un akciju cenas
if not variants:
    try:
        # 1. Cenas iegūšana (prioritāte akciju cenai ar kodu)
        price_val = 0.0

        # Tieša JS pārbaude akciju blokam
        # (piem., "31,87 € ar kodu TOP")
        # Meklējam zemāko cenu BuyBox apgabalā
        price_val = await page.evaluate('''() => {
            // 1. prioritāte: sarkanais akciju cenas teksts
            let promoEl = document.querySelector(
                'div[class*="PromoPrice"], '
                + '[data-testid*="promo-price"], '
                + 'span[class*="promo-price"]'
            );
            if (promoEl && promoEl.innerText.includes('€')) {
                 let m = promoEl.innerText.match(/(\\d+[,.]\\d+)/);
                 if (m) return parseFloat(m[1].replace(',', '.'));
            }

            // 2. prioritāte: standarta cenas komponents
```

```
    let stdEl = document.querySelector(
        '[data-testid="price-component"]'
    );
    if (stdEl) {
         let m = stdEl.innerText.match(/(\\d+[,.]\\d+)/);
         if (m) return parseFloat(m[1].replace(',', '.'));
    }
    return 0;
}''')

# 2. Tilpuma iegūšana
# (prioritāte #pdSelectedVariant ar JS pārrakstīšanu)
# Playwright inner_text() dažreiz neredz tekstu,
# ko slēpj React, bet kas faktiski ir DOM
vol_text = await page.evaluate('''() => {
    let el = document.querySelector('#pdSelectedVariant');
    return el ? el.innerText : '';
}''')

vol = ""
units_pattern = '|'.join(MEASUREMENT_UNITS)
if vol_text:
    m_sv = re.search(
        rf'(\d+(?:[.,]\d+)?\s*({units_pattern}))',
        vol_text, re.IGNORECASE
    )
    if m_sv:
        vol = m_sv.group(1).lower().replace(' ', '')
        unit_m = m_sv.group(2).lower()
        vol = vol.replace(unit_m, f' {unit_m}')

if not vol:
    # Rezerves variants: skenējam visu lapu
    full_text = await page.evaluate("document.body.innerText")
    m_vol = re.search(
        rf'(\d+(?:[.,]\d+)?\s*({units_pattern}))',
        full_text, re.IGNORECASE
```

```
                    )
                    if m_vol:
                        vol = m_vol.group(1).lower().replace(' ', '')
                        unit_match = m_vol.group(2).lower()
                        vol = vol.replace(unit_match, f' {unit_match}')

                if price_val:
                    variants.append({
                        "title_suffix": vol,
                        "price": (
                            Decimal(str(price_val)).quantize(Decimal("0.01"))
                        ),
                        "is_available": True
                    })
            except Exception as e:
                logger.debug(f"Atsevišķa produkta rezerves variants neizdevās: {e}")
    except Exception as e:
        logger.error(f"Tīkla pārtveršana neizdevās {url}: {e}")
    finally:
        await page.close()

    return variants
```

```
import asyncio
import random
from playwright.async_api import (
    async_playwright, Browser, BrowserContext, Page, Playwright
)
from playwright_stealth import stealth_async  # Stealth ielāps pret botu atklāšanu
from loguru import logger
from app.core.proxy import proxy_manager
from app.core.humanizer import Humanizer  # Cilvēka uzvedības simulators


class NetworkInterceptor:
    """Pārtver JSON atbildes un POST pieprasījumus no tīkla."""

    def __init__(self):
        # Saraksti pārtvertajiem datiem
        self.intercepted_responses = []
        self.intercepted_requests = []

    async def handle_response(self, response):
        """Apstrādā servera atbildi — saglabā JSON saturu."""
        try:
            content_type = response.headers.get("content-type", "").lower()
            if "application/json" not in content_type:
                return
            url = response.url
            # Izslēdzam izsekošanas un reklāmu pakalpojumus
            junk = [
                'google', 'facebook', 'hotjar', 'metrics',
                'clarity', 'trustpilot', 'doubleclick', 'ads'
            ]
            if any(j in url.lower() for j in junk):
                return
            json_data = await response.json()
            self.intercepted_responses.append({"url": url, "data": json_data})
```

```
        except Exception:
            pass  # Klusā kļūdu apstrāde — pārtrauktas atbildes ir normālas

    async def handle_request(self, request):
        """Pārtver POST/PUT/PATCH pieprasījumus ar JSON datiem."""
        try:
            url = request.url
            junk = [
                'google', 'facebook', 'hotjar', 'metrics',
                'clarity', 'trustpilot', 'doubleclick', 'ads'
            ]
            if any(j in url.lower() for j in junk):
                return
            # Interesē tikai pieprasījumi ar datiem
            if request.method in ["POST", "PUT", "PATCH"]:
                post_data = request.post_data_json
                if post_data:
                    self.intercepted_requests.append(
                        {"url": url, "data": post_data}
                    )
        except Exception:
            pass


class BrowserManager:
    """
    Vienots pārlūka pārvaldnieks (singleton) ar stealth slāni.
    Pārvalda Playwright instanci un izveido izolētus kontekstus.
    """

    # Cloudflare Turnstile iframe URL shablons
    __CF_PATTERN__ = (
        r"^https?://cloudflare\.com/cdn-cgi/challenge-platform/.*"
    )

    def __init__(self) -> None:
        self.playwright: Playwright | None = None
```

```
        self.browser: Browser | None = None
        # Bloķējošā slēdzene paralēlai inicializācijai
        self.lock = asyncio.Lock()

    async def start(self):
        """Palaiž Playwright un Chromium pārlūku."""
        async with self.lock:
            if not self.playwright:
                self.playwright = await async_playwright().start()
                # Bezgalviņu režīms serverī; argumenti maskē automatizāciju
                self.browser = await self.playwright.chromium.launch(
                    headless=True,
                    args=[
                        # Paslēpj 'kontrolēts ar automatizāciju' pazīmi
                        "--disable-blink-features=AutomationControlled",
                        "--no-sandbox",
                        "--disable-setuid-sandbox",
                        "--disable-dev-shm-usage",
                        "--disable-gpu"
                    ]
                )
                logger.info("🚜 3. līmenis: Pārlūka dzinējs (Chromium)
palaists.")

    async def get_context(self) -> BrowserContext:
        """
        Izveido un atgriež jaunu pārlūka kontekstu ar stealth
        un cilvēkveidīgiem iestatījumiem.
        """
        if not self.browser:
            await self.start()

        assert self.browser is not None, "Pārlūka dzinējs neizdevās palaist."

        # Iegūstam starpniekserveri no rotācijas pārvaldnieka
        proxy_str = proxy_manager.get_proxy()
        proxy_config = None
```

```
if proxy_str:
    try:
        protocol, rest = proxy_str.split("://")
        creds, addr = rest.split("@")
        username, password = creds.split(":")
        proxy_config = {
            "server": f"{protocol}://{addr}",
            "username": username,
            "password": password
        }
    except Exception:
        proxy_config = {"server": proxy_str}

# Pārmaiņus lietojamie lietotāja aģenti
user_agents = [
    (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/121.0.0.0 Safari/537.36"
    ),
    (
        "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/120.0.0.0 Safari/537.36"
    ),
    (
        "Mozilla/5.0 (X11; Linux x86_64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/121.0.0.0 Safari/537.36"
    ),
    (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/120.0.0.0 Safari/537.36"
    ),
]
```

```
    # Izplatītākie ekrāna izmēri īstās lietotāju ierīcēs
    viewports = [
        {"width": 1920, "height": 1080},
        {"width": 1366, "height": 768},
        {"width": 1536, "height": 864},
        {"width": 1440, "height": 900},
    ]

    # Katrs konteksts — nejaušs UA un ekrāna izmērs
    context = await self.browser.new_context(
        user_agent=random.choice(user_agents),
        viewport=random.choice(viewports),
        proxy=proxy_config
    )
    return context

async def _solve_cloudflare(self, page: Page) -> bool:
    """
    Mēģina atrisināt Cloudflare Turnstile izaicinājumu
    ar koordinātu balstītu klikšķi.
    """
    try:
        content = await page.content()
        # Pārbaudam vai Cloudflare izaicinājums vispār ir klāt
        cf_present = (
            "Just a moment..." in content
            or "Verifying you are human" in content
        )
        if not cf_present:
            return False

        logger.info(
            "🛡️ Cloudflare izaicinājums atklāts. Mēģinām atrisināt..."
        )

        # Īsa pauze, kamēr tīkls nostabilizējas
```

```
await asyncio.sleep(2)

# Meklējam Turnstile iframe pēc URL shablona
iframe = None
for frame in page.frames:
    import re
    if re.match(self.__CF_PATTERN__, frame.url):
        iframe = frame
        break

if not iframe:
    logger.warning("⚠️ Cloudflare iframe nav atrasts.")
    return False

# Iegūstam iframe atrašanās vietu ekrānā
frame_element = await iframe.frame_element()
if not frame_element:
    return False

box = await frame_element.bounding_box()
if not box:
    return False

# Turnstile rūtiņa parasti atrodas augšējā kreisajā stūrī
# Nobīde: X+27, Y+26 (iedvesmots no Scrapling bibliotēkas)
click_x = box["x"] + random.randint(25, 30)
click_y = box["y"] + random.randint(25, 30)

logger.info(
    f"👆 Klikšķinām Turnstile ({click_x}, {click_y})"
)
await page.mouse.click(
    click_x, click_y, delay=random.randint(100, 200)
)

# Gaidām izaicinājuma atrisinājumu (līdz 10 sekundēm)
```

```
            for _ in range(10):
                await asyncio.sleep(1)
                page_content = await page.content()
                if "Just a moment..." not in page_content:
                    logger.success("✅ Cloudflare izaicinājums atrisināts!")
                    return True

            return False
        except Exception:
            return False

async def get_page_content(
    self,
    url: str,
    fast_mode: bool = False,
    disable_images: bool = True,
    context: BrowserContext | None = None,
    click_selector: str | None = None,
    verified_domain: bool = False,
    intercept_traffic: bool = False,
    solve_cf: bool = True
) -> dict | None:
    """
    Atver lapu, opcijas: klikšķina elementu, atgriež HTML un galīgo URL.

    Parametri:
        fast_mode: turbo režīms ar minimālu gaidīšanu
        verified_domain: ja True – izlaiž humanizāciju (3-5s ietaupījums)
        click_selector: CSS selektors, uz ko klikšķināt pēc ielādes
        intercept_traffic: ja True – pārtver XHR/Fetch trafiku
    """
    if not self.browser:
        await self.start()

    assert self.browser is not None, "Pārlūka dzinējs nav palaists."
```

```
# Ja konteksts nav nodots — izveidojam lokālu
local_context = context
if not local_context:
    local_context = await self.get_context()

page: Page | None = None
try:
    page = await local_context.new_page()

    # Bloķējam attēlus ātrākai ielādei
    if disable_images:
        await page.route(
            "**/*.{png,jpg,jpeg,gif,webp,svg,ico}",
            lambda route: route.abort()
        )

    # Uzstādām tīkla pārtvērēju, ja nepieciešams
    interceptor = None
    if intercept_traffic:
        interceptor = NetworkInterceptor()
        page.on("response", interceptor.handle_response)
        page.on("request", interceptor.handle_request)

    # Uzliekam stealth ielāpu — paslēpjam Playwright nospiedumu
    await stealth_async(page)

    # Ātrais režīms nezināmai vietnei: neliela sākotnējā pauze
    if fast_mode and not verified_domain:
        await asyncio.sleep(random.uniform(0.5, 2.0))

    mode_label = "Turbo" if fast_mode else "Stealth"
    logger.info(f"🚜 Ielādē lapu ({mode_label}): {url}")
    response = await page.goto(
        url, wait_until="domcontentloaded", timeout=60000
    )
```

```
# Mēģinām atrisināt Cloudflare izaicinājumu
if solve_cf:
    await self._solve_cloudflare(page)

# Automātiskā sīkdatņu piekrišana
try:
    cookie_selectors = [
        "button:has-text('Piekrītu')",
        "button:has-text('Apstiprināt')",
        "button:has-text('Piekrist visiem')",
        "button:has-text('Pieņemt')",
        "button:has-text('Pieņemt visu')",
        "button:has-text('Accept')",
        "button:has-text('Allow all')",
        "button:has-text('I agree')",
        "#onetrust-accept-btn-handler",
        ".cookie-consent-accept",
    ]
    for btn_sel in cookie_selectors:
        btn = page.locator(btn_sel).first
        if await btn.is_visible(timeout=500):
            await btn.click()
            logger.info(
                f"🍪 Sīkdatnes akceptētas: {btn_sel}"
            )
            await asyncio.sleep(0.5)
            break
except Exception:
    pass

# UserCentrics CMP (Notino) — darbojas shadow DOM, noņemam
# pārklājumu tieši caur JavaScript
try:
    removed = await page.evaluate("""() => {
        const el = document.querySelector('#usercentrics-cmp-ui');
        if (el) { el.remove(); return true; }
```

```
            return false;
        }""")
        if removed:
            logger.info("🍪 UserCentrics CMP pārklājums noņemts.")
    except Exception:
        pass

    # Klikšķa loģika (paginācija, pogas u.c.)
    if click_selector:
        logger.info(f"👆 Klikšķinām elementu: {click_selector}")
        try:
            # SPA vietnes (Notino, 220.lv): React renderē PĒC
            # domcontentloaded — gaidām networkidle un +4s React
            # hidratācijas buferi pirms elementa meklēšanas
            try:
                await page.wait_for_load_state(
                    "networkidle", timeout=10000
                )
            except Exception:
                pass
            await asyncio.sleep(4.0)  # React hidratācijas buferis
            # state='attached' — elements tikai jāatrodas DOM
            # 'visible' neizdodas ja CSS izmanto mainīgos redzamībai
            await page.wait_for_selector(
                click_selector, state="attached", timeout=15000
            )
            locator = page.locator(click_selector).first
            await locator.scroll_into_view_if_needed(timeout=5000)
            await asyncio.sleep(0.5)
            # force=True apiet CSS redzamības pārbaudi
            await locator.click(force=True, timeout=8000)
            try:
                await page.wait_for_load_state(
                    "networkidle", timeout=10000
                )
            except Exception:
```

```
                    await asyncio.sleep(3.0)
                except Exception as e:
                    logger.warning(
                        f"⚠️ Klikšķis neizdevās {click_selector}: {e}"
                    )

            status = response.status if response else "Nav zināms"
            logger.debug(f"🌐 Atbildes statuss: {status} | {url}")

            # 4. LĪMENIS: CILVĒKA UZVEDĪBAS SIMULĀCIJA
            if verified_domain:
                # Verificētas domēns: nav anti-botu, izlaižam humanizāciju
                await asyncio.sleep(random.uniform(0.3, 0.7))
            elif fast_mode:
                # Nezināms domēns turbo režīmā: minimāla simulācija
                await asyncio.sleep(random.uniform(3.0, 5.0))
                await page.mouse.move(
                    random.randint(100, 800),
                    random.randint(100, 800)
                )
                await page.mouse.wheel(0, random.randint(100, 300))
            else:
                logger.info("🤖 Humanizēšana (cilvēka uzvedības
atdarināšana)...")
                await Humanizer.smart_wait(2, 4)
                await Humanizer.random_mouse_move(page)
                await Humanizer.human_scroll(page)
                await Humanizer.smart_wait(2, 4)

            # Gaidām dinamisko saturu (React fragmenti, SPA hidratācija)
            try:
                await page.wait_for_load_state("networkidle", timeout=10000)
            except Exception:
                pass

            final_url = page.url
            content = await page.content()
```

```
            # Brīdinājums ja HTML ir pārāk mazs — iespējams bots bloks
            if len(content) < 30000:
                short_head = content[:200].replace("\n", " ")
                logger.warning(
                    f"⚠️ Neparasts HTML izmērs: {len(content)} baiti. "
                    f"Sākums: {short_head}"
                )

            # Veidojam atgriešanas rezultātu
            result = {"html": content, "url": final_url}
            if intercept_traffic and interceptor:
                result["network_data"] = {
                    "requests": interceptor.intercepted_requests,
                    "responses": interceptor.intercepted_responses,
                }

            return result

        except Exception as e:
            logger.error(f"❌ Pārlūka kļūda: {e}")
            return None
        finally:
            # Vienmēr aizverot lapu un lokālo kontekstu
            if page:
                try:
                    await page.close()
                except Exception:
                    pass
            # Ja konteksts bija lokāls (mēs to izveidojām) — aizverot
            if not context and local_context:
                try:
                    await local_context.close()
                except Exception:
                    pass

    async def stop(self):
```

```
        """Aptur pārlūku un Playwright instanci."""
        async with self.lock:
            if self.browser:
                await self.browser.close()
                self.browser = None
            if self.playwright:
                await self.playwright.stop()
                self.playwright = None


# Globālais singleton — viens pārlūks visam procesam
browser_manager = BrowserManager()
```

client.py

```
import asyncio
import random
from curl_cffi.requests import AsyncSession
from loguru import logger
from app.core.proxy import proxy_manager
from app.core.config import settings


class StealthClient:
    """
    Uzlabots HTTP klients: izmanto Keep-Alive (pastāvīgas sesijas)
    un TLS viltošanu, lai atdarinātu īstu pārlūku.
    """

    # Pieejamie TLS pirkstu nospiedumu mērķi (Chrome versijas)
    IMPERSONATE_TARGETS = ["chrome120", "chrome124", "safari15_5"]
    # Maksimālais mēģinājumu skaits apiet aizsardzību
    MAX_RETRIES = 5

    def __init__(self):
        self.session: AsyncSession | None = None  # Aktīvā HTTP sesija
        self.current_browser: str | None = None   # Aktīvais UA tips
        self.current_proxy: str | None = None     # Aktīvais starpniekserveris

    async def init_session(self):
        """
        Inicializē ilgmūžīgu HTTP sesiju ar Keep-Alive atbalstu.
        Konfigurē TLS viltošanu un rotē starpniekserverus.
        """
        if self.session:
            await self.close()

        # Nejauša pārlūka TLS profila izvēle
        self.current_browser = random.choice(self.IMPERSONATE_TARGETS)
        self.current_proxy = proxy_manager.get_proxy()
```

```
        proxies = (
            {"http": self.current_proxy, "https": self.current_proxy}
            if self.current_proxy
            else None
        )

        self.session = AsyncSession(
            impersonate=self.current_browser,
            proxies=proxies,
            headers={
                "User-Agent": (
                    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                    "AppleWebKit/537.36 (KHTML, like Gecko) "
                    "Chrome/124.0.0.0 Safari/537.36"
                ),
                "Accept-Language": "en-US,en;q=0.9",
                "Cache-Control": "no-cache",
            },
            timeout=30
        )
        proxy_log = (
            "Tiešs"
            if self.current_proxy is None
            else f"Starpnieks {self.current_proxy.split('@')[-1]}"
        )
        logger.info(
            f"🕴️ Jauna Stealth sesija: "
            f"{self.current_browser} | {proxy_log}"
        )

    async def get_page(self, url: str) -> str | None:
        """
        Iegūst lapas HTML saturu, izmantojot aktīvo stealth sesiju.
        Īsteno automātisku atkārtošanu, starpniekservera rotāciju
        un kļūdu apstrādi bieži sastopamiem botu blokiem.
        """
```

```
# Ja sesija vēl nav izveidota — inicializējam
if not self.session:
    await self.init_session()

attempt = 0
while attempt < self.MAX_RETRIES:
    attempt += 1

    proxy_log = (
        "Tiešs"
        if self.current_proxy is None
        else f"Starpnieks {self.current_proxy.split('@')[-1]}"
    )
    logger.info(
        f"🦹 [Mēģinājums {attempt}/{self.MAX_RETRIES}] "
        f"{self.current_browser} | {proxy_log} | {url}"
    )

    try:
        response = await self.session.get(url)

        # VEIKSMĪGS ATBILDES STATUSS
        if response.status_code == 200:
            return response.text

        # LAPA NAV ATRASTA
        if response.status_code == 404:
            logger.warning(f"Lapa nav atrasta (404): {url}")
            return None

        # 202 ACCEPTED — JavaScript izaicinājums (Cloudflare/Qrator)
        # Nekavējoties pārejam uz pārlūku (3. līmenis)
        if response.status_code == 202:
            logger.warning(
                f"🛡️ JS izaicinājums (202) uz {proxy_log}. "
                f"Pārejam uz pārlūku..."
```

```
        )
        return None

    # BLOĶĒŠANA VAI PĀRSLOGOJUMS
    if response.status_code in [429, 503]:
        wait_time = random.uniform(
            settings.RATE_LIMIT_COOLDOWN_MIN,
            settings.RATE_LIMIT_COOLDOWN_MAX
        )
        logger.warning(
            f"⛔ Bloķēšana {response.status_code} uz "
            f"{proxy_log}. Gaidīšana {wait_time:.0f}s..."
        )
        # Aizliedzam šo starpniekserveri
        if self.current_proxy:
            proxy_manager.ban_proxy(self.current_proxy)
        # Izveidojam jaunu sesiju ar citu identitāti
        await self.init_session()
        await asyncio.sleep(wait_time)
        continue

    # CIETĀ BLOĶĒŠANA — maināms identitāti
    if response.status_code == 403:
        logger.critical(
            f"🚫 Cietā bloķēšana 403 uz {proxy_log}. "
            f"Mainām identitāti..."
        )
        await self.init_session()
        await asyncio.sleep(random.uniform(5, 10))
        continue

    logger.warning(
        f"Neparasts statuss ({response.status_code}) "
        f"uz {proxy_log}"
    )
    return None
```

```
            except Exception as e:
                logger.error(f"Tīkla kļūda ({proxy_log}): {e}")
                # Tīkla kļūdas gadījumā arī aizliedzam starpniekserveri
                if self.current_proxy:
                    proxy_manager.ban_proxy(self.current_proxy)
                await self.init_session()
                await asyncio.sleep(random.uniform(1, 2))
                continue

        logger.critical(
            f"❌ Neizdevās apiet aizsardzību pēc "
            f"{self.MAX_RETRIES} mēģinājumiem."
        )
        return None

    async def close(self):
        """Aizver aktīvo HTTP sesiju."""
        if self.session:
            try:
                await self.session.close()
            except Exception:
                pass
            self.session = None


# Globālais klients atkārtotai izmantošanai
stealth_client = StealthClient()
```

```
from typing import Optional, Dict
from sqlmodel import select
from thefuzz import fuzz
from loguru import logger

from app.domain.models import ProductGroup, Product
from app.services.normalizer import perfume_normalizer


class MatcherService:
    """
    Operācijas 'smadzenes'.
    Saista produktus no dažādām vietnēm vienotās 'ķēdēs' (ProductGroup).
    """

    # Neskaidrās sakritības slieksnis procentos
    FUZZY_THRESHOLD = 90

    async def find_best_match(
        self,
        session,
        product_data: Dict
    ) -> Optional[ProductGroup]:
        """
        Mēģina atrast esošu ProductGroup dotajam parfimam.
        Saskaņo pēc zīmola, tilpuma, koncentrācijas un normalizēta nosaukuma.
        """
        # 1. Iegūstam produkta atribūtus
        title = product_data.get("title", "")
        explicit_brand = product_data.get("brand")
        norm = perfume_normalizer.extract_all(
            title, explicit_brand=explicit_brand
        )

        # Izmantojam explicit_brand, ja normalizators to neatrada nosaukumā
```

```
brand = explicit_brand or norm["brand"]
volume = norm.get("volume") or product_data.get("volume")
concentration = (
    norm.get("concentration")
    or product_data.get("concentration")
)
clean_name = norm["clean_name"]

if not brand or not clean_name:
    # BĪSTAMĀ ZONA: bez ticama zīmola un tīra nosaukuma
    # NEDRĪKST klasterizēt. Pārejam uz striktu pilna nosaukuma
    # neskaidro saskaņošanu kā pēdējo iespēju.
    return await self._fuzzy_fallback(session, title)

# 2. Striktas saskaņošanas fāze
# Meklējam grupu ar TĀDU PAŠU zīmolu un līdzīgu nosaukumu
stmt = select(ProductGroup).where(
    ProductGroup.category == "Perfume"
)
results = await session.execute(stmt)
groups = results.scalars().all()

for group in groups:
    # Normalizējam arī grupas nosaukumu salīdzināšanai
    group_norm = perfume_normalizer.extract_all(group.name)

    # 1. kārtula: zīmolam jāsakrīt 100%
    if (
        brand
        and group_norm["brand"]
        and brand.lower() != group_norm["brand"].lower()
    ):
        continue

    # PIEZĪME: Tilpuma un koncentrācijas ierobežojumi NOŅEMTI —
    # dažādi varianti (EDT/EDP, 50ml/100ml) var būt vienā smaržu
    # ķēdē. Filtrus var lietot UI slānī.
```

```
        # 2. kārtula: nosaukuma līdzība (neskaidrā saskaņošana)
        if not clean_name or not group_norm["clean_name"]:
            continue  # Tukšus nosaukumus nesaskaņojam

        score = fuzz.token_sort_ratio(
            clean_name.lower(),
            group_norm["clean_name"].lower()
        )

        if score >= self.FUZZY_THRESHOLD:
            logger.info(
                f"🔗 ĶĒDE SAVIENOTA! ({score}%): "
                f"'{title}' pievienots '{group.name}'"
            )
            return group

    return None

async def _fuzzy_fallback(
    self,
    session,
    title: str
) -> Optional[ProductGroup]:
    """Strikta neskaidrā saskaņošana kā pēdējā iespēja."""
    stmt = select(ProductGroup)
    results = await session.execute(stmt)
    groups = results.scalars().all()

    for group in groups:
        score = fuzz.token_set_ratio(
            title.lower(), group.name.lower()
        )
        if score >= 95:  # Ļoti strikts slieksnis bez zīmola
            return group
    return None
```

```
def get_canonical_name(
    self,
    title: str,
    brand: Optional[str] = None
) -> str:
    """Atgriež tīru, standartizētu nosaukumu ProductGroup."""
    norm = perfume_normalizer.extract_all(
        title, explicit_brand=brand
    )
    final_brand = brand or norm["brand"]

    parts = []
    if final_brand:
        parts.append(final_brand)

    if norm["clean_name"]:
        can_add = True
        for p in parts:
            # Neatkārtojam tekstu, kas jau ir citā daļā
            if (
                norm["clean_name"].lower() in p.lower()
                or p.lower() in norm["clean_name"].lower()
            ):
                can_add = False
                break
        if can_add:
            parts.append(norm["clean_name"])

    if norm["concentration"]:
        parts.append(norm["concentration"])

    # Galīgā tīrīšana: unikāli vārdi secībā (deduplicēšana)
    final_parts = []
    seen = set()
    for p in " ".join(parts).split():
        low = p.lower().strip("-,.")
```

```
            if low not in seen and len(low) > 1:
                final_parts.append(p)
                seen.add(low)

        if not final_parts or len(" ".join(final_parts)) < 3:
            # Rezerve: noņemam tikai tilpumu un vadošās pieturzīmes
            clean_title = perfume_normalizer.REGEX_VOLUME.sub(
                "", title
            )
            clean_title = clean_title.strip("-,. |/")

            # Ja palicis tikai vispārīgs vārds — atgriežam pilno
            generics = {
                "man", "men", "woman", "women",
                "pour", "for", "eau", "de", "set"
            }
            if clean_title.lower() in generics:
                return title

            return clean_title if len(clean_title) > 2 else title

        return " ".join(final_parts)


# Globālā instance
matcher_service = MatcherService()
```

```
import asyncio
import random
from playwright.async_api import Page
from loguru import logger


class Humanizer:
    """
    Atdarina cilvēka uzvedību pārlūkā: nelineāras peles kustības,
    nejaušas pauzes, 'cilvēcisks' ritināšanas veids.
    """

    @staticmethod
    async def human_scroll(page: Page):
        """
        Ritina lapu ar raustīgiem soļiem, atdarinot lasīšanu/apskati.
        """
        try:
            # Iegūstam lapas pilno augstumu
            total_height = await page.evaluate(
                "document.body.scrollHeight"
            )
            # viewport_size ir īpašība, nevis async metode
            viewport = page.viewport_size
            viewport_height = viewport['height'] if viewport else 1080

            current_scroll = 0

            # Ritinām ne līdz pašai beigai — cilvēki reti sasniedz
            # precīzu pēdējo pikseļu
            limit = total_height * random.uniform(0.7, 0.9)

            while current_scroll < limit:
                # Nejaušs ritināšanas solis
                scroll_step = random.randint(300, 700)
```

```
            current_scroll += scroll_step

            # Ritinām ar riteņa simulāciju
            await page.mouse.wheel(0, scroll_step)

            # Nejaušs apstāšanās brīdis — "lasām saturu"
            await asyncio.sleep(random.uniform(0.2, 1.2))

            # 30% iespēja papildus pakustināt peli
            if random.random() < 0.3:
                await Humanizer.random_mouse_move(page)

    except Exception as e:
        logger.warning(f"human_scroll kļūda: {e}")

@staticmethod
async def random_mouse_move(page: Page):
    """
    Pārvieto kursoru uz nejaušu ekrāna punktu pa liektu trajektoriju.
    """
    try:
        # viewport_size ir sinhrona īpašība, nevis metode
        viewport = page.viewport_size
        if not viewport:
            viewport = {"width": 1920, "height": 1080}

        end_x = random.randint(0, viewport['width'])
        end_y = random.randint(0, viewport['height'])

        # Liektas trajektorijas simulācija caur soļu skaitu
        # (atšķiras no lineārā robotu kustību veida)
        steps = random.randint(10, 25)
        await page.mouse.move(end_x, end_y, steps=steps)

    except Exception as e:
        logger.warning(f"human_mouse kļūda: {e}")
```

```
@staticmethod
async def smart_wait(min_s: float = 1.0, max_s: float = 3.0):
    """Nejaušs apstāšanās brīdis dotajā sekunžu diapazonā."""
    await asyncio.sleep(random.uniform(min_s, max_s))
```